

UNIVERSIDAD CONTINENTAL

Facultad de Ingeniería de Sistemas e Informática



ANÁLISIS Y DISEÑO DE SOFTWARE (NRC:28638)

INFORME DE:

PROYECTO "Sistema de reportes urbanos"

Estudiantes:

Estudiante	Código	PORCENTAJE
MANDUJANO CORONEL GEAN CARLOS	72243697	100%
LAGONES BRICEÑO WILL	75779145	100%

Docente:

Rosario Delia Osorio Contreras

Grupo: 01

fecha: 21/04/2026

Huancayo – Perú

2026 – I

ÍNDICE GENERAL

ANÁLISIS Y DISEÑO DE SOFTWARE (NRC:28638)	1
Rosario Delia Osorio Contreras	1
ÍNDICE GENERAL	2
UNIDAD I – FUNDAMENTOS Y MODELADO INICIAL	5
Capítulo 1. Presentación del Proyecto	5
1.1 ODS Vinculado	5
1.2 Institución Beneficiaria	5
1.3 Problema Identificado	6
1.4 Solución Propuesta – Plataforma PIGIU	7
1.4.1 Aplicación Móvil Ciudadana (Android/iOS)	7
1.4.2 Panel Administrativo Web (Municipalidad)	7
1.4.3 Arquitectura Tecnológica	8
Capítulo 2. Análisis de Necesidades y Requerimientos	9
2.1 Descripción del Problema	9
2.2 Necesidades de los Usuarios	9
2.2.1 Ciudadano Reportante	9
2.2.2 Técnico Operativo Municipal	9
2.2.3 Funcionario Municipal	10
2.2.4 Alcaldía	10
2.3 Requerimientos Funcionales (RF)	10
PROCESO 2: DERIVACIÓN DE CASOS	11
PROCESO 3: ATENCIÓN DE CASOS	12
2.4 Requerimientos No Funcionales (RNF)	13
2.5 Requerimientos de Dominio (RD)	14
Capítulo 3. Modelos Iniciales del Sistema	16
3.1 Diagrama de Contexto	16
3.2 Diagrama de Casos de Uso – Vista General	16
3.3 Diagrama de Actividad UML – Proceso Completo de Gestión	19
3.4 Modelo Entidad-Relación (E-R)	21
UNIDAD II – MODELOS DE DISEÑO Y METODOLOGÍA ÁGIL	23
Capítulo 4. Modelos de Diseño	23
4.1 Diagrama de Clases del Sistema	23
Capítulo 4. Modelos de Diseño	24
4.1 Diagrama de Clases del Sistema	24
4.2 Diagrama de Secuencia – RF-01: Registrar Incidencia	25
4.3 Diagrama de Secuencia – Derivación y Atención de Casos	26
4.4 Diagrama de Secuencia – Seguimiento y Cierre	27
Capítulo 5. Metodología de Trabajo – SCRUM	29
5.1 Definición y Justificación de la Metodología Scrum	29
5.2 Roles del Equipo Scrum	29
5.3 Gestión del Proyecto con Jira Software (Atlassian)	30
5.3.1 Configuración del Proyecto en Jira	30

5.3.2 Tipos de Issues (Elementos de Trabajo) en Jira	31
5.3.3 Workflow (Flujo de Estados) de los Issues en Jira	31
5.3.4 Definition of Done (DoD) – Definición de Terminado	32
5.3.5 Gestión del Backlog en Jira	33
5.4 Planificación de Sprints en Jira	34
SPRINT 0 – Configuración y Arquitectura Base	34
SPRINT 1 – Registro de Incidencias (MVP Ciudadano)	35
SPRINT 2 – Derivación Automática y Panel Administrativo	36
SPRINT 3 – App del Técnico y Modo Offline	37
SPRINT 4 – Seguimiento Ciudadano y Dashboard	37
SPRINT 5 – Reportes, Transparencia e Integraciones Normativas	38
SPRINT 6 – Pruebas Finales, Hardening y Despliegue a Producción	38
5.5 Herramientas Utilizadas en el Proyecto	39
5.6 Estructura Visual del Tablero Jira – Scrum Board PIGIU	40
5.6.1 Integración Jira + GitHub con Smart Commits	40
5.6.2 Burndown Chart y Control de Velocidad del Sprint	41
5.6.3 Notificaciones y Alertas Automáticas Configuradas en Jira	42
5.7 Ceremonias Scrum – Protocolo del Equipo PIGIU	42
5.8 Criterios de Aceptación Detallados – Formato Gherkin	43
PIGIU-02: Captura fotográfica de incidencia	43
PIGIU-06: Derivación automática de incidencias	44
PIGIU-11: Actualización de estado en campo con modo offline	44
PIGIU-17: Dashboard ejecutivo con KPIs en tiempo real	45
UNIDAD III – DISEÑO DE SOFTWARE	45
Capítulo 6. Diseño de Arquitectura y Patrones de Diseño	45
6.1 Tipo de Arquitectura del Sistema	45
6.1.1 Arquitectura Adoptada	45
6.1.2 Descripción de las Cuatro Capas del Sistema	46
6.1.3 Justificación Técnica de la Arquitectura según los Requerimientos	47
6.2 Patrones de Diseño Aplicados	48
6.2.1 Patrón DAO (Data Access Object) — Patrón Estructural	48
6.2.2 Patrón Observer (Observador) — Patrón de Comportamiento	49
6.2.3 Patrón Factory Method — Patrón Creacional	51
6.2.4 Patrón Singleton — Patrón Creacional	52
6.3 Diseño Estructural — Diagrama de Clases UML Actualizado	53
Capítulo 7. Diseño Detallado de la Base de Datos	55
7.1 Modelo Lógico y Físico de la Base de Datos	55
7.1.1 Diagrama Relacional Final — Entidades, Relaciones y Cardinalidades	55
7.1.2 Normalización Explicada Paso a Paso	57
7.2 Script SQL de Creación de Tablas, Llaves y Restricciones	58
7.3 Seguridad y Respaldo de Datos	60
7.3.1 Gestión de Roles, Permisos y Control de Acceso (RBAC)	60
7.3.2 Estrategia de Respaldo, Recuperación y Alta Disponibilidad	61
Capítulo 8. Diseño Detallado de Sistemas en Red y Móviles	62

8.1 Modelo de Comunicación	62
8.1.1 Descripción del Flujo de Información entre Cliente y Servidor	62
8.1.2 Diseño de la API REST — Endpoints, Verbos y Contratos	63
8.2 Diseño de Sistema Móvil y Panel Web	66
8.2.1 Arquitectura de la Aplicación Móvil (React Native)	66
8.2.2 Diagrama de Navegación — App Ciudadana	67
8.3 Gestión de Datos en Red: Sincronización y Concurrencia	68
8.3.1 Arquitectura de Sincronización Offline — PIGIU-11	68
8.4 Seguridad en Red y Móviles	69
8.4.1 Autenticación, Autorización y Gestión de Sesiones	69
8.4.2 Cifrado de Datos en Tránsito y en Reposo	70
8.5 Justificación Técnica del Enfoque Tecnológico	71
UNIDAD IV – DISEÑO DE INTERACCIÓN HUMANO - COMPUTADORA (HCI)	73
(SEMANA 13 - 14)	
Capítulo 9. Diseño de Interfaz y Experiencia de Usuario (UX/UI)	73
9.1 Perfil del Usuario / Usuario Meta	73
9.1.1 Fundamento Metodológico: Diseño Centrado en el Usuario	73
9.1.2 Contexto de Uso — Análisis de Escenarios	75
9.2 Principios de Diseño HCI Aplicados — Cómo se Materializan en el Prototipo	75
9.3 Diseño del Prototipo — Baja y Alta Fidelidad	78
9.3.1 Proceso de Prototipado Iterativo	78
9.3.2 Wireframes de Baja Fidelidad — Pantallas Principales	79
9.3.3 Sistema de Diseño Visual — Alta Fidelidad	84
9.4 Flujo de Navegación del Sistema	86
9.4.1 Diagrama de Flujo de Navegación — App Ciudadana	86
9.4.2 Diagrama de Módulos — Panel Administrativo Web	88
9.5 Relación entre el Prototipo y los Requerimientos del Sistema	89
Conclusiones y Recomendaciones	91
Conclusiones	91
Lecciones Aprendidas	92
Recomendaciones	93
Referencias Bibliográficas	96

UNIDAD I – FUNDAMENTOS Y MODELADO INICIAL

Capítulo 1. Presentación del Proyecto

1.1 ODS Vinculado

El presente proyecto se enmarca dentro del Objetivo de Desarrollo Sostenible N.º 11 de la Agenda 2030 de las Naciones Unidas:

ODS11	Ciudades y Comunidades Sostenibles
	Meta 11.6: Reducir el impacto ambiental negativo per cápita de las ciudades, prestando especial atención a la calidad del aire y la gestión de los desechos municipales y de otro tipo.
	Meta 11.3: Aumentar la urbanización inclusiva y sostenible y la capacidad para la planificación y la gestión participativas, integradas y sostenibles de los asentamientos humanos.

El Sistema de Reportes Urbanos Sostenibles contribuye directamente a este ODS al habilitar un canal tecnológico que conecta a la ciudadanía con la administración municipal, facilitando la identificación, registro y resolución oportuna de incidencias que afectan la habitabilidad del entorno urbano, tales como acumulación de residuos sólidos, fallas en el alumbrado público y situaciones de inseguridad ciudadana.

1.2 Institución Beneficiaria

Las instituciones beneficiarias directas e indirectas del sistema son:

Actor	Tipo de Beneficio	Nivel de Impacto
Municipalidad Provincial de Huancayo	Optimización de recursos operativos y mejora en tiempos de respuesta	Directo – Alto
Municipalidades Distritales (El Tambo, Chilca, Huancayo)	Gestión descentralizada de incidencias por sector	Directo – Alto
Juntas Vecinales y Comités Comunales	Canal oficial de reporte y seguimiento de su zona	Directo – Medio
Ciudadanía en General	Participación activa en la mejora del entorno urbano	Directo – Alto
Ministerio del Ambiente (MINAM)	Datos agregados sobre gestión de residuos urbanos	Indirecto – Medio
OEFA y organismos fiscalizadores	Indicadores de cumplimiento ambiental municipal	Indirecto – Bajo

1.3 Problema Identificado

Actualmente, los ciudadanos de los distritos de Huancayo, El Tambo y Chilca no disponen de un canal formal, digital y trazable para comunicar incidencias urbanas a sus respectivas municipalidades. Los mecanismos existentes presentan las siguientes deficiencias críticas:

- Dependencia de canales informales: las incidencias se reportan por llamadas, visitas o redes sociales, sin registro oficial ni seguimiento.
- Ausencia de trazabilidad: el ciudadano no recibe confirmación ni actualizaciones sobre su reporte.
- Ineficiencia en la asignación de recursos: no se cuenta con información georreferenciada para priorizar intervenciones.
- Falta de métricas de gestión: no es posible generar indicadores para evaluar el desempeño institucional.
- Duplicación de esfuerzos: pueden asignarse recursos al mismo caso mientras otras incidencias quedan sin atender.

Esta problemática genera una degradación progresiva de la infraestructura urbana, afecta la calidad de vida de la población y erosiona la confianza ciudadana en la gestión municipal. Según datos del Índice de Satisfacción Ciudadana Municipal (INDECI, 2023), más del 62% de los ciudadanos de ciudades intermedias del Perú considera que sus reportes de servicios públicos no reciben atención oportuna.

1.4 Solución Propuesta – Plataforma PIGIU

Se propone el desarrollo de la Plataforma Integral de Gestión de Incidencias Urbanas (PIGIU). En dos capas.

1.4.1 Aplicación Móvil Ciudadana (Android/iOS)

Canal principal de ingreso de incidencias al sistema. Sus funcionalidades clave son:

- Registro de incidencias con captura fotográfica automática desde la cámara del dispositivo móvil.
- Geolocalización automática por GPS con mapa interactivo para confirmación o ajuste manual de la ubicación.
- Clasificación guiada de la incidencia en categorías: Residuos Sólidos, Alumbrado Público, Seguridad Ciudadana, Vías y Pavimento, Áreas Verdes y Otros.
- Generación automática de número de ticket con código QR para seguimiento offline.
- Notificaciones push en tiempo real: confirmación de registro, cambio de estado y resolución.
- Historial de reportes del usuario con línea de tiempo de atención.
- Módulo de valoración del servicio: el ciudadano puede calificar la atención recibida (1-5 estrellas).

1.4.2 Panel Administrativo Web (Municipalidad)

Interfaz de gestión para funcionarios municipales y técnicos operativos:

- Mapa de calor interactivo con incidencias activas clasificadas por tipo, sector y nivel de urgencia.
- Módulo de derivación automática o manual de casos a la unidad orgánica responsable (Gerencia de Servicios Públicos, Seguridad Ciudadana, Obras Públicas).
- Gestión del ciclo de vida completo del reporte: Pendiente → En Revisión → Asignado → En Atención → Resuelto → Cerrado.
- Módulo de análisis y reportes: estadísticas mensuales, tiempos promedio de atención, sectores con mayor incidencia, KPIs de desempeño por unidad.
- Integración con el Catastro Municipal para validar jurisdicción e identificar infraestructura afectada.
- Módulo de comunicación bidireccional: el funcionario puede solicitar información adicional al ciudadano a través de la app.

1.4.3 Arquitectura Tecnológica

Componente	Tecnología Propuesta	Justificación
Backend / API REST	Node.js + Express.js	Alto rendimiento para I/O concurrente, amplia comunidad
Base de Datos Relacional	PostgreSQL + PostGIS	Soporte nativo para datos geoespaciales
Almacenamiento de Imágenes	Amazon S3 / Cloudinary	Escalabilidad y gestión optimizada de archivos multimedia
Aplicación Móvil	React Native	Desarrollo multiplataforma (iOS/Android) desde una sola base de código
Panel Administrativo Web	React.js + Material UI	Interfaz moderna, responsiva y de rápida curva de aprendizaje
Notificaciones Push	Firebase Cloud Messaging (FCM)	Servicio confiable y gratuito de Google
Mapas e Integración GIS	OpenStreetMap + Leaflet.js	Open source, sin costo de licencia, personalizable
Autenticación	JWT + OAuth 2.0	Estándar de la industria, compatible con Google/Facebook
Despliegue	Docker + AWS EC2 / Heroku	Contenedores para portabilidad y escalabilidad

Capítulo 2. Análisis de Necesidades y Requerimientos

2.1 Descripción del Problema

El proceso actual de gestión de incidencias urbanas en el ámbito municipal de Huancayo puede describirse como un flujo no estructurado, reactivo y sin trazabilidad. El siguiente análisis causal sintetiza la problemática:

Causa Raíz	Efecto Directo	Impacto en la Ciudadanía
Ausencia de sistema digital de reportes	Reportes no registrados o perdidos	Incidencias sin atención durante semanas o meses
Sin georreferenciación de incidencias	Asignación ineficiente de cuadrillas	Mayor tiempo de respuesta y mayor costo operativo
Sin notificaciones al ciudadano	Desconfianza y percepción de abandono	Desmotivación para futuros reportes
Sin métricas de desempeño municipal	Imposible rendir cuentas	Opacidad en la gestión pública
Canales informales y dispersos	Duplicación o pérdida de casos	Inequidad en la atención según sector geográfico

2.2 Necesidades de los Usuarios

2.2.1 Ciudadano Reportante

- Necesidad N1: Registrar incidencias de forma rápida (menos de 60 segundos) desde su dispositivo móvil sin necesidad de formación técnica.
- Necesidad N2: Recibir confirmación inmediata de que su reporte fue recibido y un número de seguimiento único.
- Necesidad N3: Conocer en todo momento el estado actualizado de su reporte y el tiempo estimado de atención.
- Necesidad N4: Poder adjuntar evidencia fotográfica que respalde la veracidad de la incidencia.
- Necesidad N5: Tener la posibilidad de calificar la atención recibida para contribuir a la mejora del servicio.

2.2.2 Técnico Operativo Municipal

- Necesidad N6: Visualizar en un mapa interactivo todas las incidencias activas de su sector, priorizadas por urgencia.
- Necesidad N7: Recibir asignación directa de los casos que le corresponden atender, con detalle de ubicación y evidencia.
- Necesidad N8: Actualizar el estado del reporte desde el campo mediante la app o el panel web.

2.2.3 Funcionario Municipal

- Necesidad N9: Derivar casos automáticamente o manualmente a las unidades orgánicas responsables.
- Necesidad N10: Generar reportes estadísticos mensuales por tipo de incidencia, sector y tiempo de atención.
- Necesidad N11: Identificar 'puntos calientes' urbanos para planificación preventiva y presupuestal.

2.2.4 Alcaldía

- Necesidad N12: Disponer de un dashboard ejecutivo con KPIs de desempeño en tiempo real.
- Necesidad N13: Generar informes de transparencia para publicación en el portal de la municipalidad, cumpliendo con la Ley 27806.

2.3 Requerimientos Funcionales (RF)

Los requerimientos funcionales se organizan en cuatro procesos clave del sistema: Registro, Derivación de Casos, Atención de Casos y Seguimiento.

RF-01	Registro de Incidencia con Evidencia Fotográfica y GPS
Descripción	El sistema debe permitir al ciudadano autenticado registrar una nueva incidencia urbana capturando una o varias fotografías desde la cámara del dispositivo y obteniendo la ubicación geográfica exacta de manera automática mediante GPS. El usuario podrá ajustar manualmente el punto en el mapa si la señal GPS no es precisa.
Actor	Ciudadano Reportante
Precondición	El usuario debe estar autenticado en el sistema y tener habilitados los permisos de cámara y ubicación en su dispositivo.
Flujo Principal	1. El ciudadano abre la app y selecciona 'Nuevo Reporte'. 2. El sistema activa la cámara y el módulo GPS. 3. El ciudadano captura la fotografía y confirma la ubicación. 4. El ciudadano ingresa descripción textual. 5. El sistema valida los datos y genera el ticket.
Criterios de Aceptación	CA-01.1: El sistema genera un número de ticket único en formato TK-[AÑO]-[SECUENCIAL] en menos de 3 segundos tras el envío. CA-01.2: La fotografía adjunta debe tener mínimo 640x480px; el sistema rechaza imágenes de menor resolución con mensaje claro. CA-01.3: La ubicación GPS debe tener precisión mínima de 50 metros; si no se alcanza, el sistema alerta al usuario. CA-01.4: El sistema envía confirmación por notificación push y correo electrónico dentro de los 60 segundos siguientes al registro. CA-01.5: El reporte queda almacenado en estado 'Pendiente' y es visible en el panel administrativo en tiempo real.

RF-02	Clasificación de Incidencia por Categoría y Subcategoría
Descripción	El sistema debe ofrecer una taxonomía de dos niveles: Categoría principal (Alumbrado Público, Residuos Sólidos, Seguridad Ciudadana, Vías y Pavimento, Áreas Verdes, Infraestructura Pública) y Subcategoría específica, permitiendo una derivación precisa al área responsable.

Criterios de Aceptación	CA-02.1: La taxonomía de categorías es configurable por el administrador del sistema sin requerir despliegue de código.CA-02.2: El sistema sugiere automáticamente la categoría más probable basándose en palabras clave de la descripción ingresada (IA básica).CA-02.3: La categoría seleccionada determina automáticamente la unidad orgánica a la que se derivará el reporte.CA-02.4: Cada categoría tiene un SLA (tiempo máximo de atención) predefinido configurable: Urgente=24h, Normal=72h, Baja=168h.
--------------------------------	---

RF-03	Autenticación y Registro de Usuarios
Descripción	El sistema debe permitir el registro e inicio de sesión de ciudadanos mediante correo electrónico/contraseña, cuenta Google o cuenta Facebook. El acceso al panel administrativo está restringido a funcionarios municipales con credenciales institucionales.
Criterios de Aceptación	CA-03.1: El proceso de registro ciudadano no debe requerir más de 5 campos obligatorios: nombre completo, DNI, correo, contraseña, distrito.CA-03.2: La contraseña debe cumplir política de seguridad: mínimo 8 caracteres, una mayúscula, un número.CA-03.3: El sistema envía correo de verificación; la cuenta se activa solo tras confirmación.CA-03.4: Los funcionarios municipales se autentican con credenciales LDAP/AD de la municipalidad.CA-03.5: El sistema implementa bloqueo de cuenta tras 5 intentos fallidos consecutivos.

PROCESO 2: DERIVACIÓN DE CASOS

RF-04	Derivación Automática de Incidencias a Unidad Responsable
Descripción	El sistema debe derivar automáticamente cada incidencia registrada a la unidad orgánica municipal correspondiente según la categoría seleccionada. Si la derivación automática no es posible, el supervisor puede reasignar manualmente.
Reglas de Derivación	- Residuos Sólidos → Subgerencia de Limpieza Pública - Alumbrado Público → Subgerencia de Infraestructura y Obras - Seguridad Ciudadana → Gerencia de Seguridad Ciudadana / SERENAZGO - Vías y Pavimento → Subgerencia de Obras y Mantenimiento Vial - Áreas Verdes → Subgerencia de Parques y Jardines
Criterios de Aceptación	CA-04.1: La derivación automática ocurre dentro de los 5 minutos posteriores al registro del reporte.CA-04.2: El responsable de la unidad recibe notificación push, correo y alerta en el panel al recibir un nuevo caso.CA-04.3: El sistema registra en bitácora el usuario, fecha/hora y motivo de toda reasignación manual.CA-04.4: Si un caso no es derivado dentro de 60 minutos, el sistema escala automáticamente al supervisor de guardia.CA-04.5: El ciudadano recibe notificación cuando su caso es asignado a una unidad, indicando el nombre de la unidad responsable.

RF-05	Panel de Mapa Interactivo para Gestión de Incidencias
Descripción	El panel administrativo debe mostrar un mapa interactivo con todas las incidencias activas representadas con marcadores diferenciados por tipo y estado, incluyendo capacidad de filtrado, clustering de puntos cercanos y mapa de calor por densidad.
Criterios de Aceptación	CA-05.1: El mapa carga con más de 1,000 puntos activos en menos de 3 segundos.CA-05.2: El usuario puede filtrar por categoría, estado, rango de fechas y sector geográfico.CA-05.3: Al hacer clic en un marcador, se despliega un popup con: foto, descripción, fecha, estado actual y datos del ciudadano (anonimizados para niveles bajos de acceso).CA-05.4: El mapa se actualiza en tiempo real (WebSockets) al ingresar nuevas incidencias sin necesidad de recargar la página.

PROCESO 3: ATENCIÓN DE CASOS

RF-06	Gestión del Ciclo de Vida del Reporte
Descripción	El sistema debe gestionar el ciclo de vida completo del reporte a través de los estados: Pendiente → En Revisión → Asignado → En Atención → Resuelto → Cerrado. Cada transición de estado debe registrar: usuario responsable, fecha/hora y comentario opcional.
Criterios de Aceptación	CA-06.1: Solo usuarios con rol autorizado pueden transicionar a estados específicos (ej: solo el técnico asignado puede marcar 'En Atención').CA-06.2: Al marcar 'Resuelto', el técnico debe adjuntar fotografía del estado final como evidencia obligatoria.CA-06.3: El sistema calcula automáticamente el tiempo total de atención (desde Pendiente hasta Resuelto) y lo muestra en el reporte.CA-06.4: Un reporte no puede pasar de 'Pendiente' a 'Resuelto' directamente; debe cumplir el flujo establecido.CA-06.5: Los reportes en estado 'Asignado' sin actualización por más de 24h generan una alerta automática al supervisor.

RF-07	Generación de Reportes Estadísticos Administrativos
Descripción	El panel debe generar reportes estadísticos periódicos (mensual, trimestral, anual) con KPIs de gestión exportables en formato PDF y Excel.
KPIs Requeridos	• Total de reportes por período / Tasa de resolución (%) • Tiempo promedio de primera respuesta (TPPR) • Tiempo promedio de resolución (TPR) por categoría • Ranking de sectores con mayor incidencia • Tasa de satisfacción ciudadana (promedio de calificaciones) • Carga de trabajo por unidad orgánica
Criterios de Aceptación	CA-07.1: Los reportes se generan en menos de 10 segundos para períodos de hasta 12 meses.CA-07.2: Los datos exportados en Excel incluyen filas individuales por reporte con todos los campos del ciclo de vida.CA-07.3: El informe PDF incluye gráficos de barras, línea y pastel generados automáticamente.CA-07.4: El sistema permite programar el envío automático por correo del reporte mensual a los funcionarios configurados.

RF-08	Notificaciones Push y Seguimiento del Reporte
Descripción	El sistema debe notificar al ciudadano mediante notificación push y/o correo electrónico cada vez que el estado de su reporte cambie, detallando el nuevo estado, la unidad responsable y el tiempo estimado de resolución.
Criterios de Aceptación	CA-08.1: Las notificaciones push se entregan en menos de 30 segundos tras el cambio de estado.CA-08.2: El ciudadano puede configurar sus preferencias de notificación (push, correo o ambas).CA-08.3: Al estado 'Resuelto', la notificación incluye la foto del trabajo terminado adjuntada por el técnico.CA-08.4: El ciudadano puede acceder a su historial completo de reportes y el detalle de cada interacción desde la app.

RF-09	Valoración del Servicio y Cierre del Reporte
Descripción	Una vez marcado el reporte como 'Resuelto', el ciudadano recibe una solicitud de valoración del servicio (escala 1-5 estrellas) y un campo de comentario opcional. El reporte pasa a estado 'Cerrado' tras la valoración o automáticamente a las 72 horas si no hay respuesta.
Criterios de Aceptación	CA-09.1: La solicitud de valoración se envía automáticamente 2 horas después de marcarse como 'Resuelto'.CA-09.2: Si el ciudadano califica con 1 o 2 estrellas, el sistema genera una alerta al supervisor para revisión.CA-09.3: Las valoraciones se consolidan en el KPI de satisfacción del dashboard ejecutivo.CA-09.4: El ciudadano puede reabrir un reporte cerrado dentro de las 48 horas si considera que no fue resuelto correctamente.

2.4 Requerimientos No Funcionales (RNF)

ID	Categoría	Descripción	Métrica de Cumplimiento
RNF-01	Disponibilidad	El sistema debe estar operativo el 99.5% del tiempo mensual, con mantenimiento programado fuera de horario laboral.	Tiempo de inactividad no planificado ≤ 3.6 horas/mes
RNF-02	Rendimiento	El panel administrativo debe cargar el mapa de incidencias con más de 1,000 puntos en menos de 3 segundos.	Medición con Lighthouse: LCP < 3s
RNF-03	Usabilidad	El proceso de registro de una incidencia en la app móvil no debe requerir más de 4 pasos ni tomar más de 60 segundos.	Prueba de usabilidad con 10 usuarios no técnicos
RNF-04	Seguridad	Los datos personales de los ciudadanos deben almacenarse cifrados (AES-256) y las comunicaciones con TLS 1.3.	Auditoría de seguridad con OWASP ZAP
RNF-05	Privacidad	El sistema debe cumplir la Ley N.º 29733 (Ley de Protección de Datos)	Certificación de conformidad legal

		Personales del Perú) y su reglamento.	
RNF-06	Escalabilidad	La arquitectura debe soportar hasta 10,000 usuarios concurrentes sin degradación del servicio.	Prueba de carga con Apache JMeter
RNF-07	Portabilidad	La app móvil debe funcionar en Android 8.0+ e iOS 13.0+, cubriendo el 95% de los dispositivos del mercado peruano.	Pruebas en 10 dispositivos físicos representativos
RNF-08	Mantenibilidad	El código fuente debe seguir estándares de documentación y tener cobertura de pruebas unitarias $\geq 70\%$.	Reporte de cobertura con Jest/Coverage

2.5 Requerimientos de Dominio (RD)

RD-01	Cumplimiento de Normativa de Transparencia Gubernamental
Normativa Aplicable	Ley N.° 27806 – Ley de Transparencia y Acceso a la Información Pública del Perú; y los lineamientos del Decreto Supremo N.° 072-2003-PCM que establece el Reglamento de dicha ley.
Obligación del Sistema	El sistema debe generar y publicar automáticamente en el portal web municipal un resumen estadístico mensual de incidencias: número de reportes recibidos, atendidos, pendientes y tiempos de atención por sector. Esta publicación debe realizarse antes del día 5 del mes siguiente.
Criterios de Aceptación	CA-RD01.1: El informe mensual es generado automáticamente el día 1 de cada mes y está disponible en el portal público en formato PDF y HTML. CA-RD01.2: El informe público muestra datos agregados, sin información personal identificable de los ciudadanos. CA-RD01.3: Los datos son auditables por el Órgano de Control Institucional (OCI) mediante exportación de logs.

RD-02	Integración con el Plan PIGARS de Gestión de Residuos Sólidos
Normativa Aplicable	Ley N.° 27314 – Ley General de Residuos Sólidos y su modificatoria el D.Leg. N.° 1278 (Ley de Gestión Integral de Residuos Sólidos). El PIGARS (Plan Integral de Gestión Ambiental de Residuos Sólidos) es el instrumento de planificación municipal de gestión de residuos según el D.S. N.° 014-2017-MINAM.
Obligación del Sistema	Los reportes de la categoría 'Residuos Sólidos' deben integrarse con los parámetros del PIGARS municipal: horarios y rutas de recolección vigentes, zonas de disposición temporal autorizadas y calendario de recojo diferenciado (orgánicos/inorgánicos). El sistema debe validar que la incidencia no corresponde a un punto de acopio autorizado antes de generar la alerta.
Criterios de Aceptación	CA-RD02.1: El sistema tiene cargadas las rutas PIGARS y el horario de recolección actualizable por el administrador. CA-RD02.2: Si una incidencia de basura se reporta en zona con recojo programado en las próximas 4 horas, el sistema sugiere al

	ciudadano esperar el servicio antes de confirmar el reporte.CA-RD02.3: Las rutas PIGARS son visibles en el mapa del panel administrativo como capa independiente.
--	---

RD-03	Integración con el Catastro Municipal para Validación de Jurisdicción
Normativa Aplicable	D.S. N.° 002-89-JUS y Ley N.° 28294 (Sistema Nacional Integrado de Catastro). La Ley Orgánica de Municipalidades (Ley N.° 27972) establece que la municipalidad solo es competente para atender incidencias dentro de su ámbito jurisdiccional.
Obligación del Sistema	El sistema debe integrar la capa GIS del Catastro Municipal (límites distritales, zonificación y nomenclatura vial) para: (a) Validar automáticamente que la ubicación GPS de la incidencia se encuentra dentro del ámbito de competencia de la municipalidad receptora; (b) Identificar la vía pública o infraestructura afectada con su código catastral.
Criterios de Aceptación	CA-RD03.1: Si la incidencia se ubica fuera del límite jurisdiccional, el sistema lo notifica al ciudadano y le sugiere el municipio competente, sin rechazar el reporte.CA-RD03.2: El sistema resuelve la dirección postal a partir de coordenadas GPS (geocodificación inversa) y la adjunta automáticamente al reporte.CA-RD03.3: Los datos del catastro se actualizan al menos semestralmente desde la fuente oficial.

RD-04	Protección de Datos Personales (NUEVO)
Normativa Aplicable	Ley N.° 29733 – Ley de Protección de Datos Personales del Perú y D.S. N.° 003-2013-JUS (Reglamento). El sistema procesa datos sensibles de ciudadanos (DNI, correo, ubicación en tiempo real), lo que lo clasifica como banco de datos de administración pública sujeto a registro en la ANPD.
Obligación del Sistema	El sistema debe registrar el banco de datos ante la Autoridad Nacional de Protección de Datos Personales (ANPD), obtener consentimiento explícito del ciudadano al momento del registro, y garantizar los derechos ARCO (Acceso, Rectificación, Cancelación y Oposición).
Criterios de Aceptación	CA-RD04.1: El proceso de registro incluye checkbox obligatorio de aceptación de política de privacidad, con enlace al documento completo.CA-RD04.2: El ciudadano puede solicitar desde la app la eliminación de su cuenta y datos asociados; el sistema procesa la solicitud en ≤ 15 días hábiles.CA-RD04.3: El acceso a datos personales de ciudadanos en el panel administrativo está restringido según rol: los niveles operativos solo ven el nombre y ticket, los datos completos solo para fiscalía o jefatura.

Capítulo 3. Modelos Iniciales del Sistema

3.1 Diagrama de Contexto

El Diagrama de Contexto representa el sistema PIGIU como una caja negra, mostrando las entidades externas que interactúan con él y los flujos de información que intercambian:

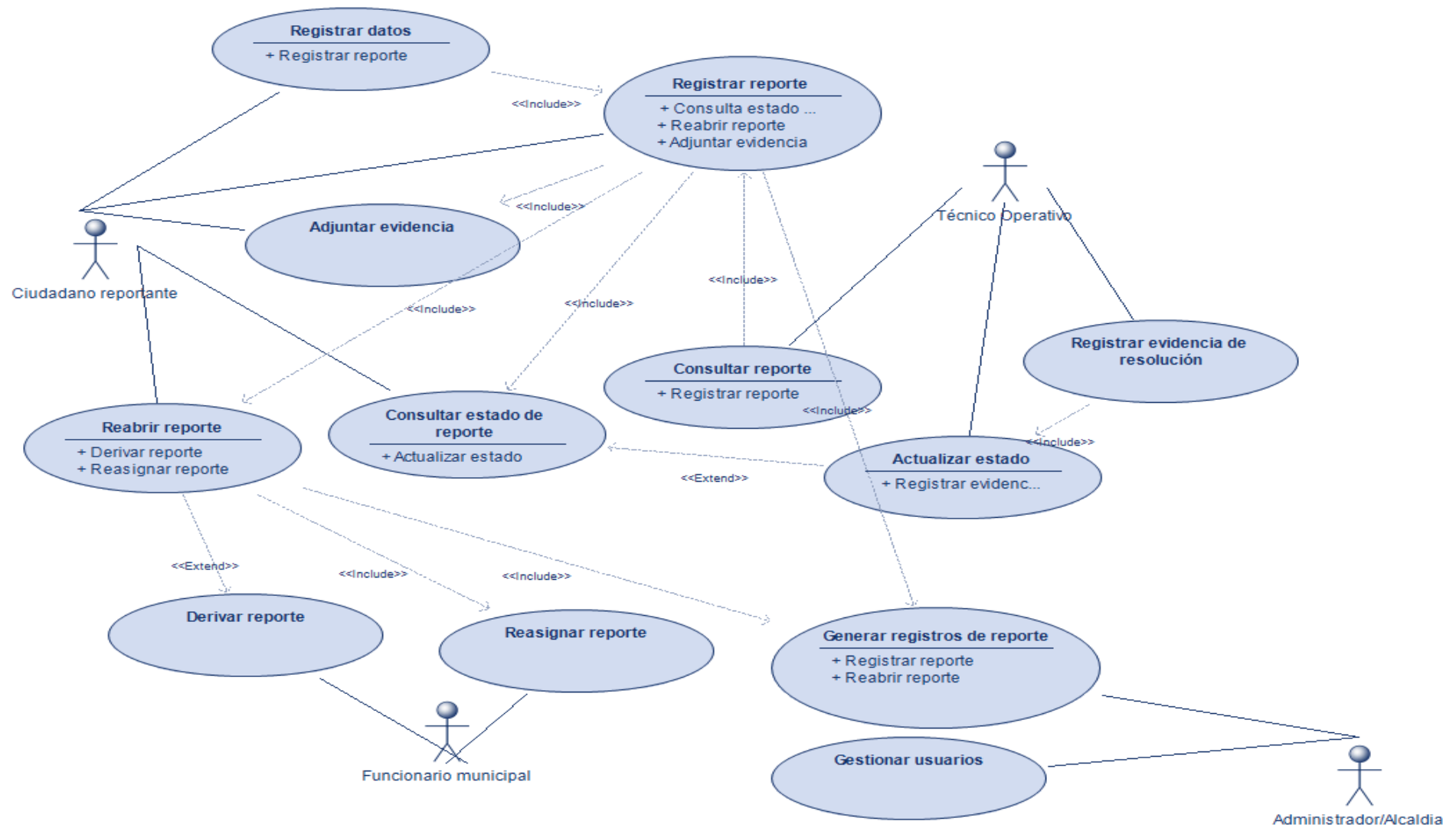


Nota: El símbolo → indica flujo de entrada al sistema; ← indica flujo de salida. Las entidades externas actúan como proveedores y consumidores de información del sistema PIGIU.

3.2 Diagrama de Casos de Uso – Vista General

El diagrama de casos de uso del sistema PIGIU representa la interacción entre los principales actores involucrados en la gestión de incidencias urbanas: el Ciudadano Reportante, el Técnico Operativo, el Funcionario Municipal y el Administrador o AEl. El Ciudadano Reportante inicia el proceso al registrar una incidencia, ingresando los datos correspondientes y adjuntando evidencia fotográfica. Asimismo, puede consultar el estado de su reporte en cualquier momento y, en caso de no estar conforme con la atención recibida, solicitar la reapertura del caso. El Técnico Operativo participa en la fase de atención, consultando los reportes asignados, actualizando su estado y registrando la evidencia de resolución una vez culminada la intervención. Por su parte, el Funcionario Municipal desempeña un papel clave en la supervisión y control del proceso, ya que puede derivar reportes a las áreas competentes, reasignarlos cuando sea necesario y gestionar los casos reabiertos para asegurar una atención adecuada. Finalmente, el Administrador o Alcaldía tiene acceso a funciones estratégicas, como la generación de registros y reportes consolidados, así como la gestión de usuarios del sistema. En conjunto, este modelo refleja una solución integral, colaborativa y orientada a optimizar la atención, seguimiento y control de las incidencias urbanas.

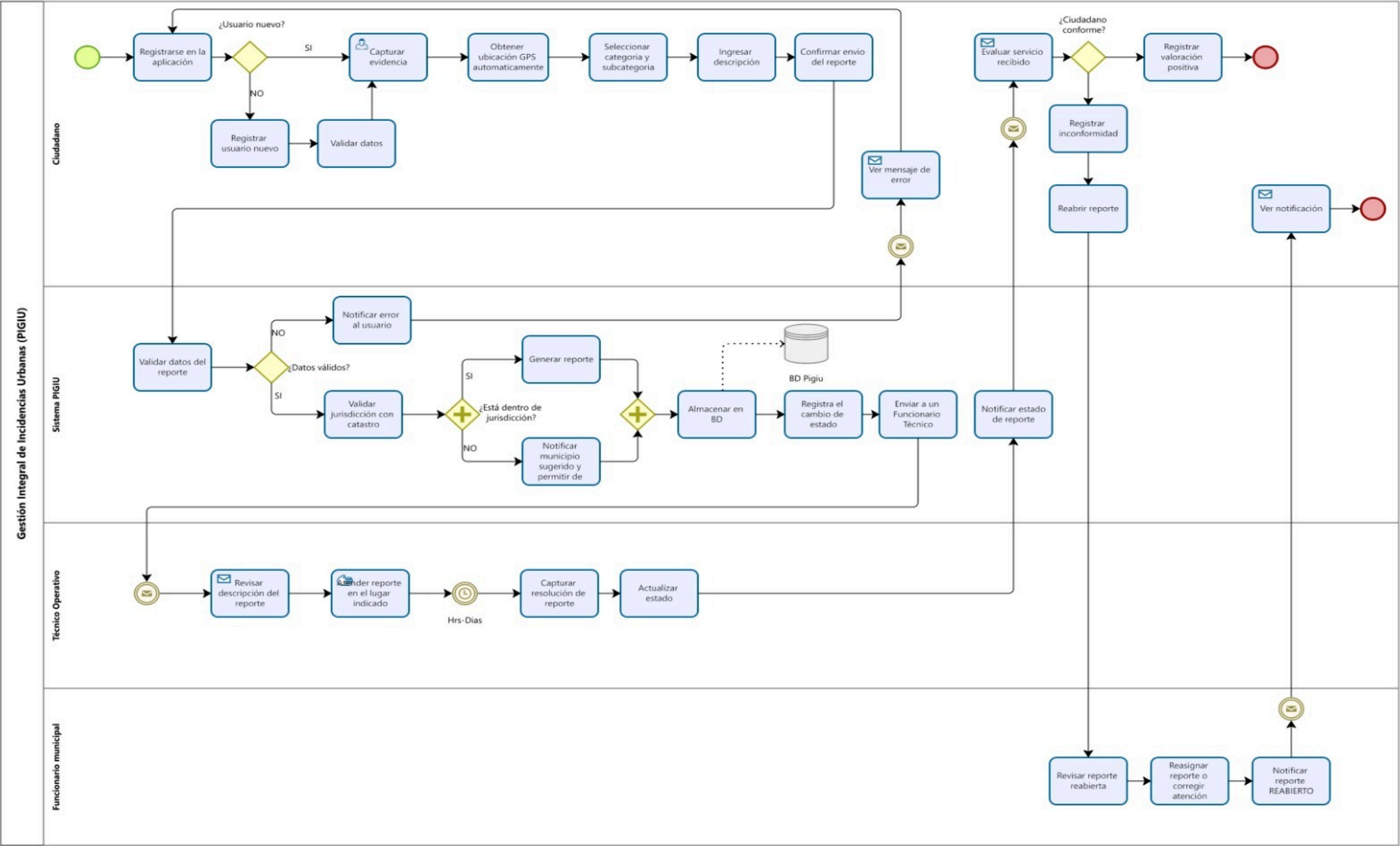
lcaldía.



Actor	Casos de Uso Asociados
Ciudadano Reportante	CU-01 Registrar datos; CU-02 Registrar reportes; CU-03 Adjuntar evidencia; CU-04 Consultar estado de reporte; CU-05 Reabrir reporte.
Técnico operativo	CU-06 Consultar reporte CU-07 Actualizar Estado de reporte; CU-08 Adjuntar Evidencia de Resolución.
Funcionario municipal	CU-09 Derivar reporte, CU-10 Reasignar reporte.
Administrador / Alcaldía	CU-11 Generar Reportes Estadísticos; CU-12 Gestionar usuarios.

3.3 Diagrama de Actividad UML – Proceso Completo de Gestión

El modelo BPMN desarrollado en Bizagi para la Plataforma Integral de Gestión de Incidencias Urbanas, articula la interacción coordinada de cuatro actores principales: el Ciudadano, el Sistema PIGIU, el Técnico Operativo y el Funcionario Municipal. El proceso inicia cuando el ciudadano registra una incidencia mediante la aplicación, proporcionando evidencia fotográfica, ubicación geográfica, categoría y descripción del problema. Posteriormente, el Sistema PIGIU valida la información ingresada, verifica la jurisdicción territorial mediante integración con el catastro y genera automáticamente el reporte, almacenando en la base de datos y asignándole al técnico correspondiente. El Técnico Operativo recibe la notificación, revisa el caso, se desplaza al lugar de la incidencia, ejecuta la atención y registra la evidencia de resolución, actualizando el estado del reporte. Una vez resuelto, el sistema notifica al ciudadano, quien evalúa la calidad del servicio recibido. Si el ciudadano manifiesta conformidad, el caso se cierra satisfactoriamente. En caso contrario, se registra la inconformidad y el reporte es reabierto. En esta etapa, el Funcionario Municipal asume un rol fundamental, ya que revisa el caso reabierto, evalúa la atención brindada y procede a asignar el reporte o disponer acciones correctivas, garantizando así la adecuada resolución de la incidencia y la mejora continua del servicio municipal.



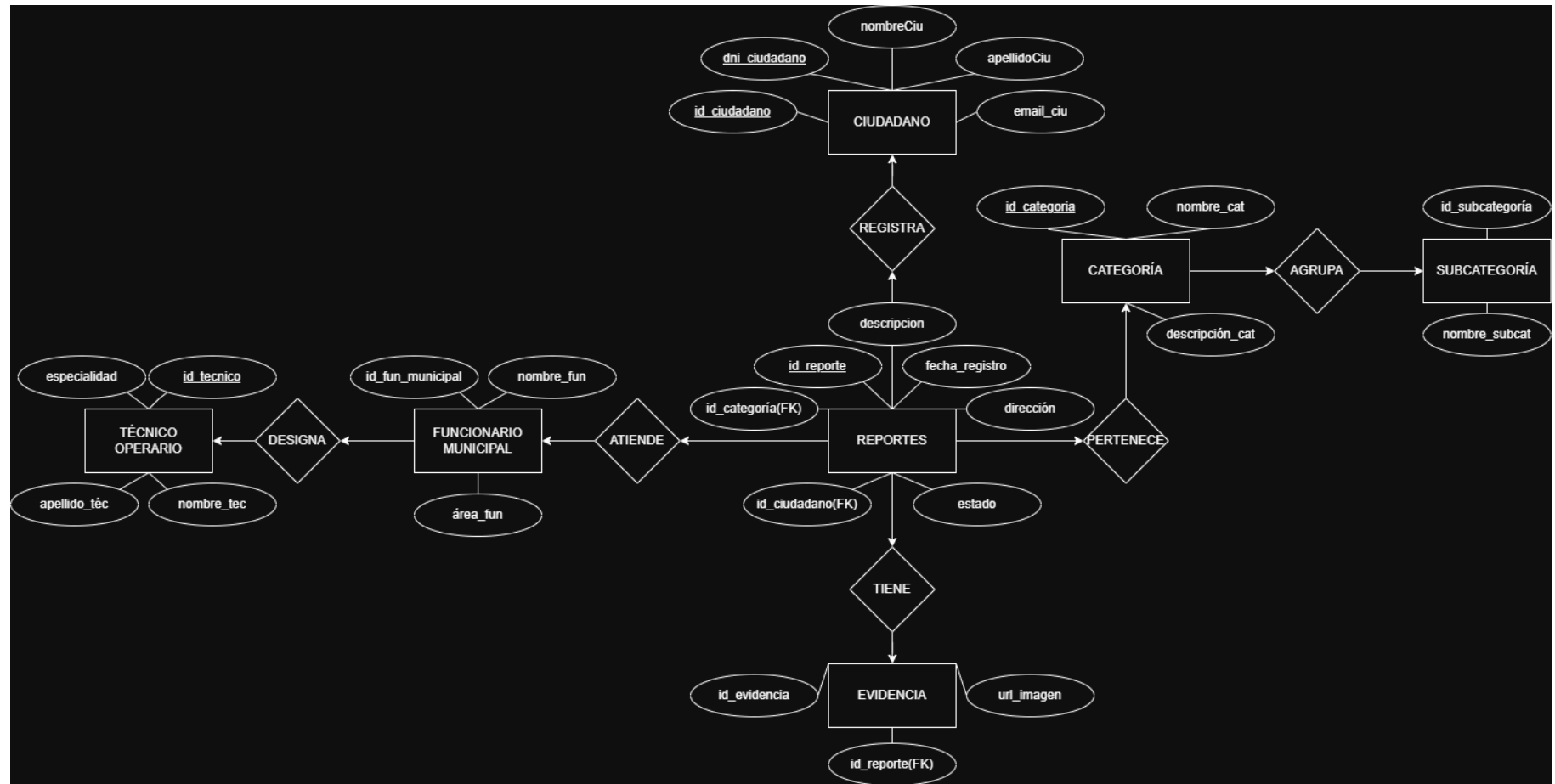
3.4 Modelo Entidad-Relación (E-R)

El modelo entidad-relación del Sistema de Gestión Integral de Incidencias Urbanas, está conformado por varias entidades principales que permiten administrar de manera estructurada el registro, seguimiento y atención de incidencias urbanas. La entidad ciudadano almacena la información de los usuarios que reportan incidencias, incluyendo atributos como identificador, DNI, nombres, apellidos y correo electrónico. Un ciudadano puede registrar uno o varios reportes, estableciéndose así una relación de uno a muchos con la entidad reporte. Esta última constituye la entidad central del sistema y contiene información esencial como el identificador del reporte, descripción de la incidencia, fecha de registro, dirección, estado, así como las claves foráneas que lo vinculan con el ciudadano y la categoría correspondiente.

La entidad categoría clasifica los diferentes tipos de incidencias mediante atributos como identificador, nombre y descripción. Cada categoría puede agrupar múltiples subcategorías, generando una relación de uno a muchos con la entidad subcategoría, la cual permite una clasificación más específica de los reportes. Asimismo, cada reporte pertenece a una única categoría, mientras que una categoría puede estar asociada a múltiples reportes.

Para respaldar la información reportada, la entidad evidencia almacena archivos o imágenes relacionados con cada incidencia. Esta entidad incluye atributos como identificador de evidencia, URL de la imagen y el identificador del reporte al que pertenece. La relación entre reportes y evidencia es de uno a muchos, ya que un reporte puede contar con varias evidencias, pero cada evidencia está asociada a un único reporte.

En el proceso de atención intervienen las entidades técnico operativo y funcionario municipal. El técnico operativo, identificado por atributos como código, nombres, apellidos y especialidad, es responsable de atender las incidencias asignadas. Por su parte, el funcionario municipal participa en la asignación y supervisión de los reportes, contando con atributos como identificador, nombre y área funcional. La relación entre estas entidades y reportes refleja el flujo operativo del sistema: el funcionario municipal asigna los reportes y el técnico operativo los atiende, garantizando una gestión eficiente desde el registro hasta la resolución de cada incidencia.



UNIDAD II – MODELOS DE DISEÑO Y METODOLOGÍA ÁGIL

Capítulo 4. Modelos de Diseño

4.1 Diagrama de Clases del Sistema

El diagrama de clases del Sistema de Gestión Integral de Incidencias Urbanas, muestra la estructura del sistema y cómo interactúan sus principales componentes. La clase central es reporte, ya que almacena la información esencial de cada incidencia, como descripción, ubicación, fecha, hora y estado. Además, permite confirmar el registro, actualizar su estado, derivarlo a otras áreas y reasignarlo cuando sea necesario.

La clase ciudadano reportante representa a la persona que registra la incidencia. Contiene sus datos personales y le permite realizar acciones como registrar una incidencia, consultar el estado de sus reportes, valorar la atención recibida o reabrir un caso cerrado. Cada ciudadano puede generar varios reportes a lo largo del tiempo.

Para complementar la información, la clase evidencia almacena imágenes, archivos u otros documentos que respalden el reporte. Asimismo, las clases categoría principal y subcategoría organizan y clasifican las incidencias, permitiendo una identificación más precisa del problema reportado.

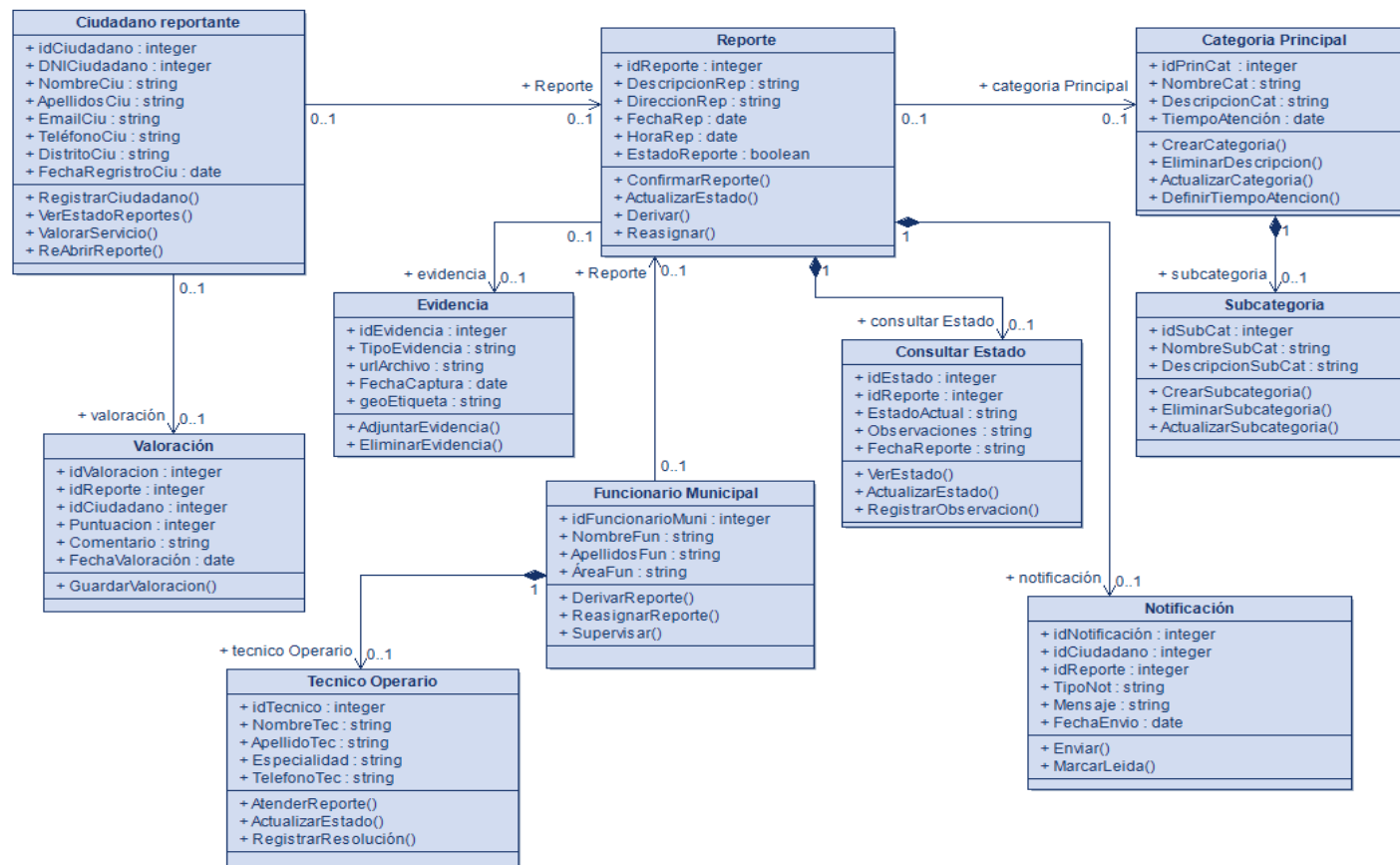
En la gestión operativa intervienen el funcionario municipal, quien supervisa, deriva y reasigna los reportes, y el técnico operativo, encargado de atender las incidencias en campo, actualizar su estado y registrar la resolución. Por otro lado, la clase consultar estado permite hacer seguimiento al avance del reporte y registrar observaciones durante su atención.

Finalmente, las clases de notificación y valoración fortalecen la interacción con el ciudadano. La primera facilita el envío de avisos sobre cambios en el estado del reporte, mientras que la segunda permite evaluar la calidad del servicio mediante puntuaciones y comentarios. En conjunto, estas clases garantizan una gestión integral, eficiente y orientada al usuario.

Capítulo 4. Modelos de Diseño

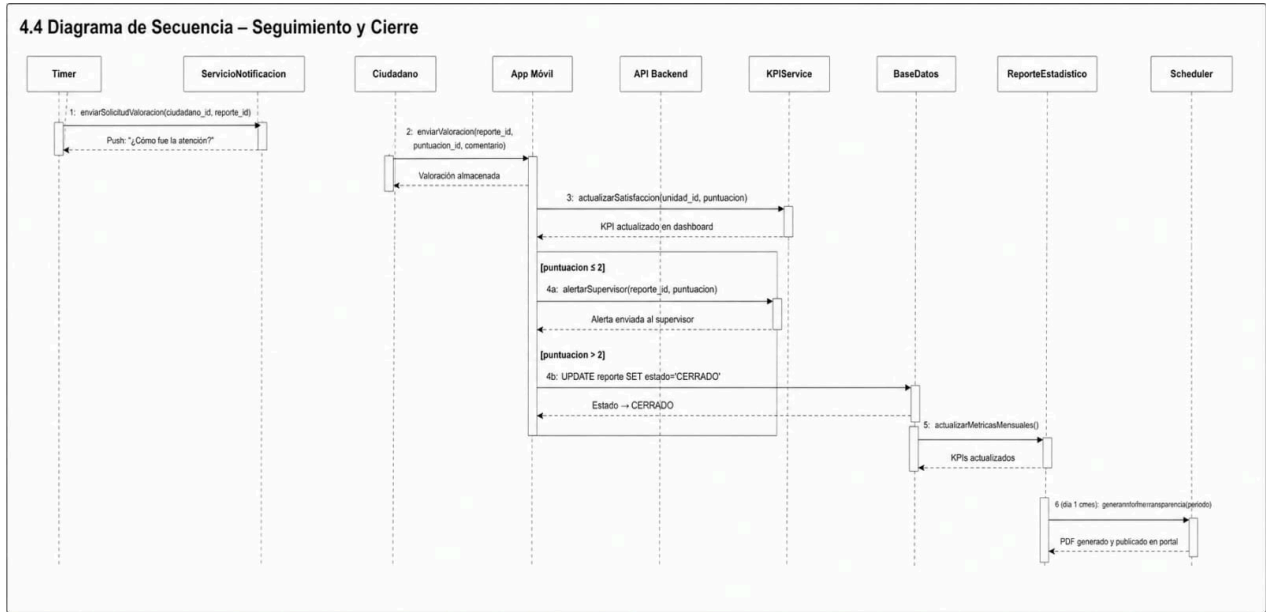
4.1 Diagrama de Clases del Sistema

El diagrama de clases presenta la estructura orientada a objetos del sistema PIGIU, con sus atributos, métodos y relaciones de herencia, composición y asociación:



4.2 Diagrama de Secuencia – RF-01: Registrar Incidencia

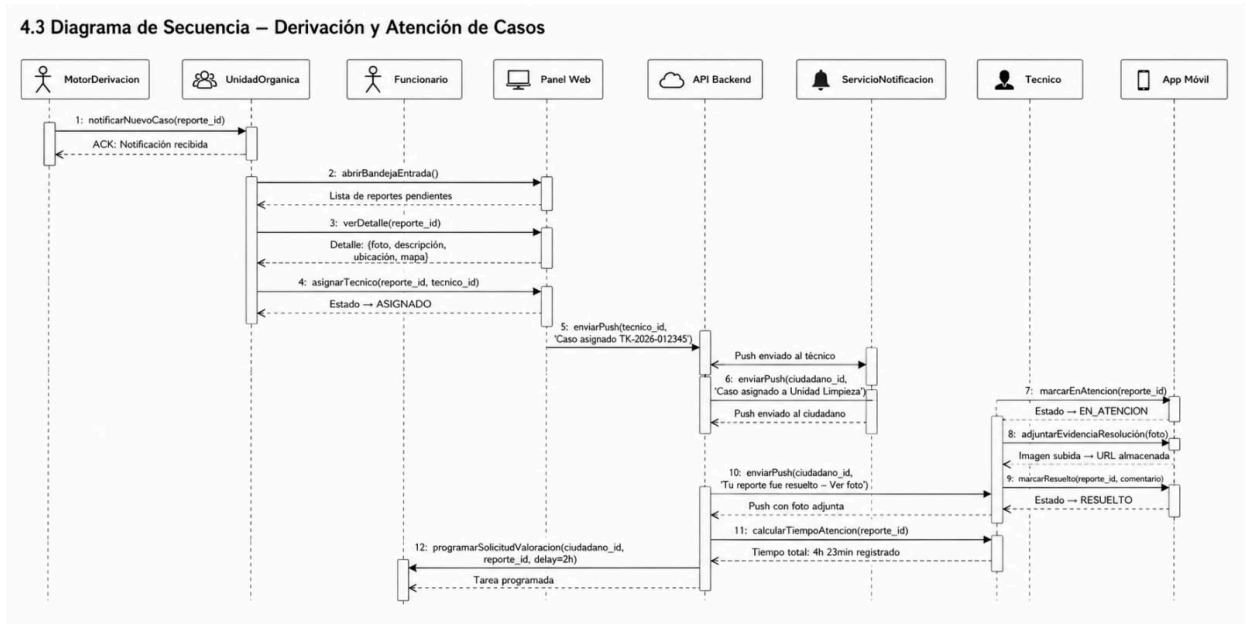
El diagrama muestra la interacción temporal entre los objetos del sistema durante el registro de una incidencia:



Paso	Actor / Objeto	Mensaje / Acción	Respuesta del Sistema
1	Ciudadano → App Móvil	seleccionarNuevoReporte()	Pantalla de captura activa
2	App → ServicioGPS	obtenerUbicacion()	Coordenadas {lat, lon, precision}
3	App → Cámara	capturarFoto()	Imagen en buffer
4	App → API Backend	POST /reportes {foto, lat, lon, descripcion, categoria_id}	202 Accepted
5	API → ServicioGIS	validarJurisdiccion(lat, lon)	Distrito: {nombre, municipalidad_id} Error: FueraJurisdiccion
6	API → ServicioAlmacenamiento	subirImagen(imagen, reporte_id)	URL pública de la imagen
7	API → BaseDatos	INSERT INTO reporte (...) VALUES (...)	id_reporte: 12345
8	API → ReporteService	generarTicket(id_reporte)	TK-2026-012345
9	API → MotorDerivacion	derivarAutomaticamente(reporte_id, categoria_id)	unidad_id: 3 (Limpieza Pública)

10	API → ServicioNotificacion	enviarPush(ciudadano_id, 'Reporte registrado: TK-2026-012345')	ACK: Push enviado
11	API → PanelAdmin (WebSocket)	broadcast(nuevoReporte)	Mapa actualizado en tiempo real
12	API → App Móvil	201 Created {ticket: 'TK-2026-012345', estado: 'PENDIENTE'}	Pantalla de confirmación

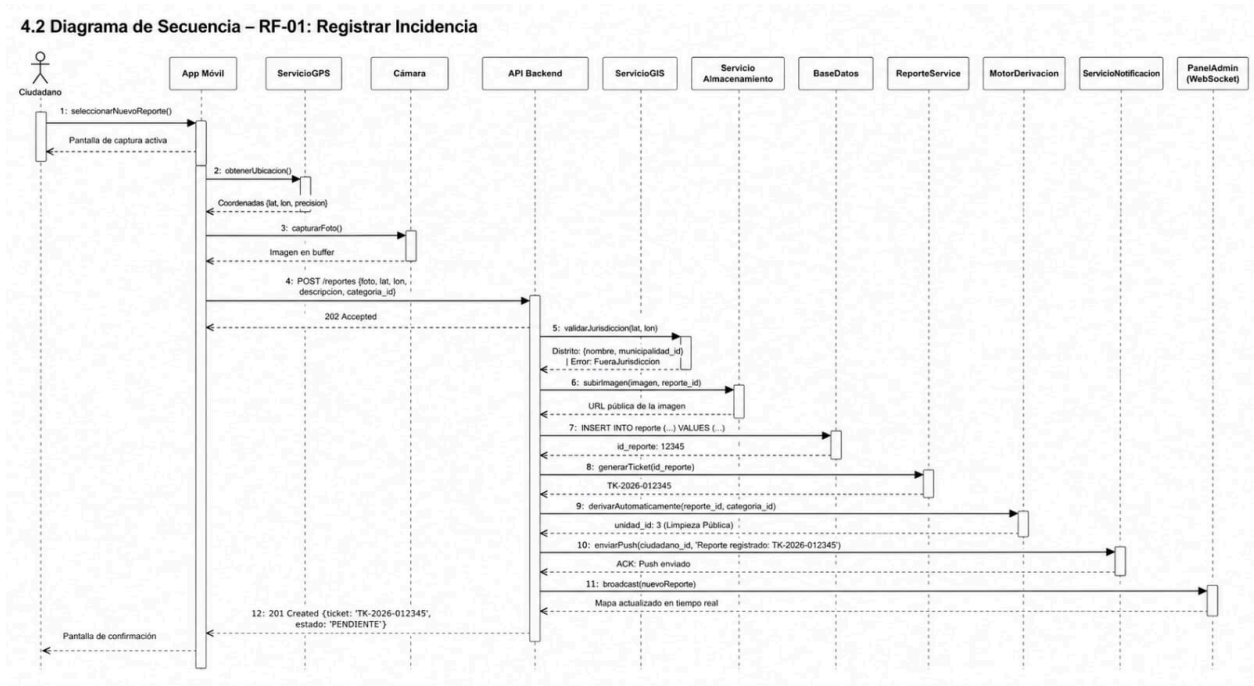
4.3 Diagrama de Secuencia – Derivación y Atención de Casos



Paso	Actor / Objeto	Mensaje / Acción	Respuesta del Sistema
1	MotorDerivacion → UnidadOrganica	notificarNuevoCaso(reporte_id)	ACK: Notificación recibida
2	Funcionario → Panel Web	abrirBandejaEntrada()	Lista de reportes pendientes
3	Funcionario → Panel Web	verDetalle(reporte_id)	Detalle: {foto, descripción, ubicación, mapa}
4	Funcionario → Panel Web	asignarTecnico(reporte_id, tecnico_id)	Estado → ASIGNADO
5	API → ServicioNotificacion	enviarPush(tecnico_id, 'Caso asignado TK-2026-012345')	Push enviado al técnico
6	API → ServicioNotificacion	enviarPush(ciudadano_id, 'Caso asignado a Unidad Limpieza')	Push enviado al ciudadano

7	Tecnico → App Móvil	marcarEnAtencion(rep orte_id)	Estado → EN_ATENCION; timestamp registrado
8	Tecnico → App Móvil	adjuntarEvidenciaReso lucion(foto)	Imagen subida → URL almacenada
9	Tecnico → App Móvil	marcarResuelto(report e_id, comentario)	Estado → RESUELTO
10	API → ServicioNotificacion	enviarPush(ciudadano _id, 'Tu reporte fue resuelto – Ver foto')	Push con foto adjunta
11	API → ReporteService	calcularTiempoAtencio n(reporte_id)	Tiempo total: 4h 23min registrado
12	Sistema → Timer	programarSolicitudValo racion(ciudadano_id, reporte_id, delay=2h)	Tarea programada

4.4 Diagrama de Secuencia – Seguimiento y Cierre



Paso	Actor / Objeto	Mensaje / Acción	Respuesta del Sistema
1	Timer → ServicioNotificacion	enviarSolicitudValoracion(ciudadano_id, reporte_id)	Push: '¿Cómo fue la atención?'
2	Ciudadano → App Móvil	enviarValoracion(report e_id, puntuacion=4, comentario)	Valoración almacenada
3	API → KPIService	actualizarSatisfaccion(unidad_id, puntuacion)	KPI actualizado en dashboard

4a [puntuacion $\leq$ 2]	API → ServicioNotificacion	alertarSupervisor(reporte_id, puntuacion)	Alerta enviada al supervisor
4b [puntuacion > 2]	API → BaseDatos	UPDATE reporte SET estado='CERRADO'	Estado → CERRADO
5	Sistema → ReporteEstadistico	actualizarMetricasMensuales()	KPIs actualizados
6 [día 1 c/mes]	Scheduler → ReporteEstadistico	generarInformeTransparencia(periodo)	PDF generado y publicado en portal

Capítulo 5. Metodología de Trabajo – SCRUM

5.1 Definición y Justificación de la Metodología Scrum

Scrum es un marco de trabajo ágil definido por Ken Schwaber y Jeff Sutherland (Scrum Guide, 2020) que permite a equipos pequeños y multifuncionales entregar productos complejos de manera incremental a través de ciclos cortos de desarrollo llamados Sprints. Se fundamenta en tres pilares empíricos: Transparencia (todo el trabajo y su progreso es visible), Inspección (revisión continua de los artefactos) y Adaptación (ajuste rápido ante desviaciones).

Para el proyecto PIGIU, la adopción de Scrum se justifica técnicamente por las siguientes razones:

Criterio de Selección	Análisis para el Proyecto PIGIU
Requisitos cambiantes	La coordinación con la municipalidad y las normas de transparencia gubernamental pueden evolucionar durante el desarrollo. Scrum permite incorporar cambios entre sprints sin desestabilizar el proyecto.
Equipo pequeño y co-ubicado	Con 3 integrantes, Scrum elimina la burocracia de metodologías pesadas como RUP. La comunicación directa reduce el overhead de documentación formal.
Entrega de valor temprana	Organizando el backlog por procesos críticos (Registro → Derivación → Atención → Seguimiento), el Sprint 1 ya entrega un módulo funcional usable por ciudadanos.
Integración con herramienta Jira	Jira Software de Atlassian es la plataforma líder de gestión ágil, con soporte nativo para Scrum Boards, Backlogs priorizados, Sprints, Burndown Charts y reportes de velocidad, cubriendo el 100% de los artefactos Scrum requeridos.
Gestión de riesgo técnico	Las integraciones complejas (catastro GIS, PIGARS, FCM) se planifican en sprints específicos, permitiendo identificar impedimentos antes de que afecten el núcleo del sistema.
Validación con usuarios reales	Cada Sprint Review permite demostrar funcionalidades a representantes de la municipalidad y obtener feedback antes de invertir en el siguiente módulo.

5.2 Roles del Equipo Scrum

El equipo PIGIU opera como un Scrum Team autoorganizado con los siguientes roles y responsabilidades definidas:

Rol Scrum	Integrante	Responsabilidades Detalladas
Product Owner (PO)	Quispe Salas, Mónica	• Define, refina y prioriza el Product Backlog en Jira (campo 'Priority' y 'Story Points'). • Representa los intereses de la Municipalidad y los ciudadanos ante el equipo técnico. • Redacta y valida los criterios de aceptación de cada Historia de Usuario. • Aprueba o rechaza el incremento en cada Sprint Review. • Gestiona las relaciones con stakeholders externos (funcionarios municipales, MINAM). • Toma decisiones de negocio ante conflictos de prioridad entre funcionalidades.
Scrum Master (SM)	Lagones Briceño, Will, Salazar Yauri Javier	• Facilita las 5 ceremonias Scrum (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective, Backlog Refinement). • Administra el tablero Jira: crea el proyecto, configura workflows, estados de issues y permisos de usuario. • Elimina impedimentos del equipo: problemas técnicos, accesos, coordinaciones externas. • Monitorea el Burndown Chart en Jira para detectar desviaciones del Sprint Goal. • Protege al equipo del scope creep durante el sprint activo. • Facilita las Retrospectivas usando la técnica 'Start / Stop / Continue'.
Developer Team	Mandujano Coronel, Gean Carlos Lagones Briceño, Will Quispe Salas, Mónica, Salazar Yauri Javier	Gean Carlos (Backend Lead): Diseño e implementación de la API REST con Node.js, base de datos PostgreSQL+PostGIS, integraciones con servicios externos (FCM, S3, Catastro), lógica de negocio y pruebas unitarias con Jest. Will (Frontend/Móvil): Desarrollo de la app móvil en React Native y del panel administrativo en React.js; integración con la API REST; pruebas de usabilidad con usuarios reales. Mónica (QA/DevOps): Diseño y ejecución de pruebas funcionales y de carga; gestión de entornos con Docker; pipeline CI/CD; documentación técnica y de usuario final.

5.3 Gestión del Proyecto con Jira Software (Atlassian)

Jira Software es la herramienta oficial de gestión del proyecto PIGIU. Se utiliza el plan Jira Free (hasta 10 usuarios), suficiente para el equipo de 3 integrantes. A continuación se detalla la configuración completa del proyecto en Jira y cómo cada artefacto Scrum es gestionado dentro de la plataforma:

5.3.1 Configuración del Proyecto en Jira

Parámetro de Configuración	Valor Configurado en Jira
Nombre del Proyecto	PIGIU – Sistema de Reportes Urbanos Sostenibles
Clave del Proyecto (Key)	PIGIU

Tipo de Proyecto	Software – Scrum (Board con Backlog y Sprints)
URL del Proyecto	https://pigiui-team.atlassian.net/jira/software/projects/PIGIU/boards
Miembros del equipo	3 usuarios: gean.mandujano@, will.lagones@, monica.quispe@ (con roles PO, SM, Dev)
Idioma del tablero	Español
Estimación de issues	Story Points (Fibonacci: 1, 2, 3, 5, 8, 13, 21)
Velocidad de referencia	26 Story Points por Sprint de 2 semanas (basado en Sprint 1)

5.3.2 Tipos de Issues (Elementos de Trabajo) en Jira

El proyecto PIGIU utiliza la jerarquía nativa de Jira para organizar el trabajo en cuatro niveles:

Tipo de Issue	Icono Jira / Color	Uso en el Proyecto PIGIU
Epic	[Morado] Epic	Representa los 4 procesos principales del sistema: EPIC-01 Registro de Incidencias, EPIC-02 Derivación de Casos, EPIC-03 Atención de Casos, EPIC-04 Seguimiento Ciudadano. Cada Epic tiene su propia etiqueta de color en el tablero para identificación visual rápida.
Story (Historia de Usuario)	[Verde] Story	Una por cada Historia de Usuario (HU-01 a HU-18). Contiene: descripción en formato 'Como [actor], quiero [acción] para [beneficio]', criterios de aceptación en la sección 'Acceptance Criteria', estimación en Story Points y Epic de pertenencia. Es la unidad de trabajo principal del Sprint.
Task (Tarea)	[Azul] Task	Subtareas técnicas derivadas de cada Story. Ejemplos: 'Crear endpoint POST /reportes', 'Diseñar pantalla de captura en React Native', 'Escribir pruebas unitarias del servicio GPS'. Se asignan a un Developer específico con estimación en horas.
Bug	[Rojo] Bug	Defectos detectados durante las pruebas de QA. Contiene: pasos para reproducir, comportamiento esperado vs. actual, entorno de prueba, severidad (Crítico, Mayor, Menor, Trivial) y captura de pantalla adjunta. Se prioriza para corrección en el mismo sprint si es Crítico o Mayor.

5.3.3 Workflow (Flujo de Estados) de los Issues en Jira

Se configuró un workflow personalizado en Jira que refleja el ciclo de vida del desarrollo del equipo PIGIU:

Estado en Jira	Columna del Tablero	Definición y Criterio de Transición
----------------	---------------------	-------------------------------------

TO DO	Backlog / Por Hacer	La Story o Task ha sido creada y priorizada en el Product Backlog, pero aún no ha sido seleccionada para un Sprint activo. Es el estado inicial de todos los issues.
SELECTED FOR DEV	Seleccionado	El issue fue incorporado al Sprint en el Sprint Planning. Tiene Developer asignado, estimación en SP y fecha de inicio planificada. Transición: Sprint Planning Meeting.
IN PROGRESS	En Desarrollo	El Developer ha iniciado activamente el trabajo. El Daily Scrum debe reportar el porcentaje de avance. La fecha de inicio real queda registrada automáticamente en Jira. Transición: el Developer mueve la tarjeta al iniciar el trabajo.
CODE REVIEW	Revisión de Código	El Developer ha completado la implementación y ha creado un Pull Request en GitHub. Otro miembro del equipo debe revisar el código antes de aprobar el merge. Jira se vincula al PR de GitHub mediante la integración nativa. Transición: al abrir el PR.
IN QA TESTING	En Pruebas	El código fue aprobado y desplegado en el entorno de staging. El responsable de QA (Mónica) ejecuta los casos de prueba definidos en los criterios de aceptación. Si encuentra defectos, crea un Bug issue vinculado a la Story. Transición: al hacer merge a develop.
DONE	Terminado	La Story cumple el Definition of Done (DoD): código mergeado a main, pruebas pasadas al 100%, criterios de aceptación validados por el PO, documentación actualizada, sin bugs Críticos ni Mayores abiertos. Transición: aprobación del PO en Sprint Review.

5.3.4 Definition of Done (DoD) – Definición de Terminado

El equipo acordó la siguiente Definition of Done aplicable a todas las Stories del proyecto. Una Story solo puede transicionar a estado DONE en Jira cuando cumple TODOS los siguientes criterios:

1. El código fuente está completo, comentado y sube el Pull Request a la rama develop en GitHub.
2. El Pull Request fue revisado y aprobado por al menos un integrante del equipo (Code Review en Jira).
3. Todos los criterios de aceptación de la Story han sido probados y aprobados por el QA (evidencia en la Story de Jira como adjunto o comentario).
4. La cobertura de pruebas unitarias del módulo nuevo no baja del 70% (reporte de Jest adjunto al issue).
5. El incremento fue demostrado en el entorno de staging y el Product Owner lo aprobó formalmente en la Sprint Review.
6. No existen bugs de tipo Crítico o Mayor vinculados a la Story sin resolver.

7. La documentación técnica del módulo (README, Swagger/OpenAPI para endpoints) está actualizada.
8. El issue en Jira está en estado DONE y el Story Point actualizado si hubo re-estimación durante el sprint.

5.3.5 Gestión del Backlog en Jira

El Product Backlog se gestiona directamente en la vista 'Backlog' de Jira. La estructura de épicas e historias es la siguiente:

Epic (Jira)	ID Story	Título de la Historia de Usuario	Story Points	Prioridad Jira
EPIC-01 Registro de Incidencias	PIGIU-01	Registro de usuario ciudadano (correo / Google)	5	Highest
EPIC-01	PIGIU-02	Captura fotográfica de incidencia con validación de calidad	8	Highest
EPIC-01	PIGIU-03	Geolocalización automática GPS con ajuste manual en mapa	8	Highest
EPIC-01	PIGIU-04	Clasificación de incidencia por categoría y subcategoría	5	High
EPIC-01	PIGIU-05	Generación de ticket único y confirmación al ciudadano	3	High
EPIC-02 Derivación de Casos	PIGIU-06	Derivación automática de incidencias a unidad responsable	13	Highest
EPIC-02	PIGIU-07	Mapa interactivo de incidencias para panel administrativo	13	Highest
EPIC-02	PIGIU-08	Reasignación manual de casos con bitácora de auditoría	8	Medium

EPIC-02	PIGIU-09	Escalamiento automático por vencimiento de SLA	8	High
EPIC-03 Atención de Casos	PIGIU-10	Bandeja de casos asignados con detalle para técnico	8	Highest
EPIC-03	PIGIU-11	Actualización de estado en campo con modo offline	13	Highest
EPIC-03	PIGIU-12	Evidencia fotográfica obligatoria al marcar Resuelto	5	High
EPIC-03	PIGIU-13	Generación de reportes estadísticos PDF/Excel	8	Medium
EPIC-04 Seguimiento Ciudadano	PIGIU-14	Notificaciones push por cambio de estado del reporte	5	Highest
EPIC-04	PIGIU-15	Historial completo del reporte con línea de tiempo	5	High
EPIC-04	PIGIU-16	Valoración del servicio (1-5 estrellas) post-resolución	3	Medium
EPIC-04	PIGIU-17	Dashboard ejecutivo con KPIs en tiempo real	8	Medium
EPIC-04	PIGIU-18	Publicación automática de informe de transparencia mensual	8	High

5.4 Planificación de Sprints en Jira

La planificación de sprints se realiza en la vista 'Sprint Planning' de Jira, donde el equipo selecciona los issues del Backlog y los incorpora al Sprint activo. Se definieron 7 sprints (incluyendo Sprint 0 de preparación):

SPRINT 0 – Configuración y Arquitectura Base

Atributo	Detalle
Duración	1 semana (Semana 1 del proyecto)
Sprint Goal	Tener el entorno de desarrollo completamente configurado y la arquitectura base del sistema definida y documentada, de modo que el equipo pueda comenzar a desarrollar funcionalidades en Sprint 1 sin impedimentos técnicos.
Issues en Jira (Tasks)	TASK-001: Crear repositorio GitHub con estructura de carpetas (monorepo) TASK-002: Configurar proyecto Jira con board, workflow y permisos TASK-003: Diseño y creación de la base de datos PostgreSQL+PostGIS TASK-004: Configurar entorno Docker (docker-compose.yml para dev/staging) TASK-005: Configurar pipeline CI/CD básico con GitHub Actions TASK-006: Configurar proyecto React Native y React.js (boilerplate) TASK-007: Definir estándares de código (ESLint, Prettier, convención de commits)
Capacidad del equipo	3 developers × 5 días × 8h = 120 horas totales disponibles
Criterio de éxito	Todos los entornos corren sin errores; la base de datos acepta conexiones; el pipeline CI pasa en verde; el tablero Jira está operativo.

SPRINT 1 – Registro de Incidencias (MVP Ciudadano)

Atributo	Detalle
Duración	2 semanas (Semanas 2-3)
Sprint Goal	El ciudadano puede registrarse, iniciar sesión y registrar una incidencia con foto y GPS, recibiendo su número de ticket de confirmación. Al finalizar el sprint, se tiene un flujo de reporte funcional de extremo a extremo.
Issues seleccionados	PIGIU-01 (5 SP) – Registro/Login ciudadano PIGIU-02 (8 SP) – Captura fotográfica PIGIU-03 (8 SP) – Geolocalización GPS PIGIU-04 (5 SP) – Categorización de incidencias PIGIU-05 (3 SP) – Generación de ticket
Total Story Points	29 SP
Capacidad del equipo	3 developers × 10 días × 6h productivas = 180 horas Velocidad objetivo: 29 SP

Distribución de tareas en Jira	Gean: Backend API (endpoints /auth, /reportes, /categorias, validación GPS) Will: App React Native (pantallas de registro, login, nuevo reporte, mapa) Mónica: QA (casos de prueba de todos los CAs definidos, pruebas de usabilidad con 5 personas externas)
Ceremonias Scrum en Jira	Sprint Planning: Día 1, 2h – selección de issues y asignación Daily Scrum: Cada día, 15 min – actualización del tablero Jira Sprint Review: Día 10, 1h – demo a stakeholder municipal Retrospectiva: Día 10, 45 min – 'Start/Stop/Continue'
Definition of Done aplicada	Los 5 stories en estado DONE; app funciona en Android e iOS; backend desplegado en staging; PO aprobó la demo.

SPRINT 2 – Derivación Automática y Panel Administrativo

Atributo	Detalle
Duración	2 semanas (Semanas 4-5)
Sprint Goal	Los funcionarios municipales pueden visualizar incidencias en el mapa interactivo, recibir casos derivados automáticamente y gestionar el ciclo de vida de los reportes desde el panel web.
Issues seleccionados	PIGIU-06 (13 SP) – Derivación automática PIGIU-07 (13 SP) – Mapa interactivo PIGIU-08 (8 SP) – Reasignación manual PIGIU-09 (8 SP) – Escalamiento por SLA
Total Story Points	42 SP
Nota de capacidad	Este sprint tiene alta carga (42 SP). Se activa la regla Scrum de re-estimación: si al día 7 el Burndown muestra riesgo, el PO puede negociar mover PIGIU-08 o PIGIU-09 al siguiente sprint manteniendo el Sprint Goal principal.
Distribución de tareas en Jira	Gean: Motor de derivación automática, WebSockets para mapa en tiempo real, lógica de escalamiento SLA, integración PostGIS para queries geoespaciales Will: Panel web React.js con Leaflet.js, componentes de mapa, filtros, popup de detalle, cambio de estado desde panel Mónica: Pruebas de carga del mapa (JMeter con 1000+ puntos), validación de derivación con distintas categorías

Riesgos identificados en Jira	RISK-001: La integración WebSockets puede requerir configuración adicional en el servidor; contingencia: polling cada 30s como fallback.
-------------------------------	--

SPRINT 3 – App del Técnico y Modo Offline

Atributo	Detalle
Duración	2 semanas (Semanas 6-7)
Sprint Goal	El técnico operativo puede recibir, gestionar y resolver casos desde su dispositivo móvil, incluso sin conexión a Internet, adjuntando evidencia fotográfica geolocalizada al marcar un caso como resuelto.
Issues seleccionados	PIGIU-10 (8 SP) – Bandeja de casos para técnico PIGIU-11 (13 SP) – Actualización de estado offline PIGIU-12 (5 SP) – Evidencia fotográfica al resolver
Total Story Points	26 SP
Tecnología clave a implementar	SQLite local en React Native para almacenamiento offline; Redux-Persist para sincronización de estado; lógica de merge de conflictos al recuperar conexión.

SPRINT 4 – Seguimiento Ciudadano y Dashboard

Atributo	Detalle
Duración	2 semanas (Semanas 8-9)
Sprint Goal	Los ciudadanos reciben notificaciones en tiempo real sobre el estado de sus reportes y pueden valorar el servicio. La alcaldía dispone de un dashboard ejecutivo con KPIs actualizados automáticamente.
Issues seleccionados	PIGIU-14 (5 SP) – Notificaciones push PIGIU-15 (5 SP) – Historial de reporte PIGIU-16 (3 SP) – Valoración del servicio PIGIU-17 (8 SP) – Dashboard ejecutivo KPIs
Total Story Points	21 SP
Integración clave	Firebase Cloud Messaging (FCM) para notificaciones push; Recharts o Chart.js para el dashboard en React.js; sistema de colas para envíos masivos de notificaciones.

SPRINT 5 – Reportes, Transparencia e Integraciones Normativas

Atributo	Detalle
Duración	2 semanas (Semanas 10-11)
Sprint Goal	El sistema genera reportes estadísticos en PDF/Excel, publica el informe de transparencia mensual automáticamente y valida incidencias contra las capas GIS del catastro y las rutas PIGARS.
Issues seleccionados	PIGIU-13 (8 SP) – Reportes estadísticos PDF/Excel PIGIU-18 (8 SP) – Publicación informe transparencia RD-01 (integración normativa Ley 27806) RD-02 (integración rutas PIGARS) RD-03 (validación catastro GIS)
Total Story Points	24 SP + tareas técnicas de integración
Riesgo principal	La obtención de la API del catastro municipal requiere gestión institucional (carta de convenio). Contingencia: usar GeoJSON estático del IGN como sustituto temporal.

SPRINT 6 – Pruebas Finales, Hardening y Despliegue a Producción

Atributo	Detalle
Duración	1 semana (Semana 12)
Sprint Goal	El sistema PIGIU pasa todas las pruebas de aceptación del cliente, cumple los RNFs de rendimiento y seguridad definidos, y queda desplegado en el entorno de producción listo para el piloto en El Tambo.
Issues en Jira	TASK-FINAL-01: Prueba de carga con Apache JMeter (10,000 usuarios concurrentes) TASK-FINAL-02: Auditoría de seguridad con OWASP ZAP TASK-FINAL-03: Pruebas de usabilidad final con 10 usuarios ciudadanos TASK-FINAL-04: Corrección de bugs críticos encontrados en pruebas finales TASK-FINAL-05: Configuración del servidor de producción (AWS EC2 / Docker) TASK-FINAL-06: Generación de documentación final de usuario y técnica TASK-FINAL-07: Presentación del sistema al stakeholder municipal
Entregable final	Sistema PIGIU en producción + Manual de usuario (app ciudadana y panel) + Manual de administración + Informe de pruebas + Presentación ejecutiva

5.5 Herramientas Utilizadas en el Proyecto

El proyecto PIGIU emplea un ecosistema de herramientas especializadas, cada una seleccionada por su pertinencia técnica y su integración con el flujo de trabajo Scrum:

Herramienta	Categoría	Uso Específico en el Proyecto PIGIU
Jira Software (Atlassian)	Gestión Ágil [HERRAMIENTA PRINCIPAL]	Gestión completa del proyecto Scrum: Product Backlog con épicas e historias de usuario priorizadas, Sprint Planning, Scrum Board (Kanban para el sprint activo), Burndown Chart para monitoreo de avance, Velocity Chart para análisis de capacidad entre sprints, gestión de bugs, reportes de progreso. Integrado con GitHub para trazabilidad entre commits y issues. URL: https://pigiui-team.atlassian.net
Confluence (Atlassian)	Documentación	Wiki del proyecto: Actas de reuniones (Sprint Planning, Review, Retrospectiva), Especificación de arquitectura del sistema, Guías de instalación del entorno de desarrollo, Decisiones de diseño (ADR – Architecture Decision Records), Manual de usuario final. Integrado con Jira para vincular páginas a épicas o stories.
Draw.io (diagrams.net)	Modelado UML	Creación de todos los diagramas UML del proyecto: Diagrama de Contexto, Diagrama de Casos de Uso, Diagrama de Actividad con swimlanes, Diagrama de Clases completo, Diagramas de Secuencia. Los archivos .drawio se versionan en el repositorio GitHub y se incrustan en las páginas de Confluence. URL: https://app.diagrams.net
dbdiagram.io	Modelado de BD	Diseño visual del Modelo Entidad-Relación (E-R) de la base de datos PostgreSQL usando la sintaxis DBML (Database Markup Language). Permite generar automáticamente el script SQL de creación de tablas. El diagrama es exportado en PNG e incrustado en el informe técnico y en Confluence. URL: https://dbdiagram.io
GitHub	Control de versiones	Repositorio de código fuente con estrategia GitFlow: rama main (producción), develop (integración), feature/PIGIU-XX (por historia de usuario). Pull Requests vinculados a issues de Jira mediante Smart Commits (ej: 'git commit -m "PIGIU-02: validación resolución imagen"]'). GitHub Actions para CI/CD. URL: https://github.com/pigiui-team/pigiui-app
Figma	Diseño UI/UX	Diseño de los wireframes y prototipos de alta fidelidad de la app móvil y el panel web. Los diseños son validados con el Product Owner antes del inicio de cada sprint. Enlazados en Jira como adjuntos a las stories de frontend para referencia del Developer. URL: https://www.figma.com
Postman	Testing de API	Documentación y prueba de todos los endpoints REST de la API PIGIU. La colección de Postman incluye todos los endpoints con ejemplos de request/response, variables de entorno (dev, staging, prod) y tests automatizados que validan los criterios de aceptación de los RFs backend.

Apache JMeter	Pruebas de carga	Ejecución de pruebas de rendimiento para validar los RNFs: carga del mapa con 1,000+ incidencias en < 3 segundos, soporte de 10,000 usuarios concurrentes. Los reportes de JMeter (TPS, tiempo de respuesta, porcentaje de errores) se adjuntan como evidencia en Jira en las tasks de QA del Sprint 6.
Slack	Comunicación	Canal de comunicación asíncrona del equipo. Integrado con Jira (notificaciones de cambio de estado de issues) y GitHub (notificaciones de PR y CI). Canales: #general, #pigiu-backend, #pigiu-frontend, #pigiu-qa, #pigiu-daily.

5.6 Estructura Visual del Tablero Jira – Scrum Board PIGIU

El tablero Scrum de Jira es el centro de operaciones del equipo. A continuación se describe la estructura de columnas configurada en el proyecto PIGIU y un ejemplo de cómo se vería durante el Sprint 1 activo:

TO DO	IN PROGRESS	CODE REVIEW	IN QA TESTING	BLOCKED	DONE
PIGIU-01 Registro usuario 5 SP Highest Asign: Mónica	PIGIU-02 Captura foto 8 SP Highest Asign: Gean [2 días restantes]	PIGIU-03 Módulo GPS 8 SP Highest Asign: Will PR #12 en revisión	PIGIU-01 Tests: 12/12 ✓ Asign: Mónica Esperando PO	PIGIU-07 13 SP BLOQUEO: API catastro aun pendiente	PIGIU-05 Ticket generado 3 SP ✓ PO aprobado ✓ Tests 100% ✓

Cada tarjeta (card) del tablero muestra: ID del issue (PIGIU-XX), título resumido, Story Points, avatar del Developer asignado, prioridad con color (rojo=Highest, naranja=High, azul=Medium) y etiqueta de Epic de pertenencia en la parte superior de la tarjeta.

5.6.1 Integración Jira + GitHub con Smart Commits

El equipo configuró la integración nativa Jira-GitHub para trazabilidad total entre código e issues. Regla de equipo: ningún commit puede existir sin un issue de Jira asociado. Los Smart Commits permiten controlar el tablero directamente desde la terminal:

Acción del Developer	Sintaxis del Smart Commit en Git	Efecto Automático en Jira
Inicio de desarrollo de un issue	git commit -m "PIGIU-02 #in-progress: inicio módulo de captura fotográfica con validación"	El issue PIGIU-02 pasa automáticamente a columna IN PROGRESS. El commit aparece vinculado en la sección 'Development' del issue.

Apertura de Pull Request para revisión	Título del PR: "PIGIU-03: Módulo GPS con geolocalización automática [Closes PIGIU-03]"	El PR queda vinculado al issue PIGIU-03 en Jira. El SM y el revisor reciben notificación para Code Review.
PR aprobado, mover a QA	git commit -m "PIGIU-01 #in-qa-testing: merge feature/PIGIU-01 a develop post code review"	Issue pasa a IN QA TESTING. QA (Mónica) recibe alerta automática en Jira y Slack #pigiui-qa.
Issue completado y aprobado por PO	git commit -m "PIGIU-05 #done: criterios de aceptación verificados, PO aprobó en review"	Issue transiciona a DONE. Jira registra tiempo total de desarrollo automáticamente. Story Points suman al velocity chart.
Reporte de bug vinculado a un issue	Título del Bug en Jira: "BUG-003: La foto no valida resolución en dispositivos Android < 10 [Linked: PIGIU-02]"	Bug creado y vinculado al issue PIGIU-02. Si es severidad Crítica, el SM recibe alerta inmediata y el issue padre pasa a BLOCKED hasta resolución.

5.6.2 Burndown Chart y Control de Velocidad del Sprint

Jira genera automáticamente los gráficos de monitoreo del sprint que el Scrum Master revisa cada mañana antes del Daily Scrum:

Gráfico en Jira	Qué muestra	Cómo lo usa el equipo PIGIU
Burndown Chart (por Sprint)	Eje Y: Story Points restantes. Eje X: días del sprint. Línea gris = progreso ideal. Línea azul = progreso real del equipo.	Si al día 7 la línea azul está por encima de la gris, el SM activa alerta y el PO negocia remover el issue de menor prioridad del sprint para proteger el Sprint Goal.
Velocity Chart (histórico)	SP comprometidos vs. SP completados en cada sprint anterior, mostrado como barras comparativas.	El equipo usa el promedio de los 3 últimos sprints como referencia de capacidad para el siguiente Sprint Planning. Velocidad de referencia PIGIU: 26 SP/sprint.
Cumulative Flow Diagram (CFD)	Acumulación de issues por columna a lo largo del tiempo, visualizadas como áreas de color apiladas.	Si la banda 'IN PROGRESS' crece sin que la de 'DONE' también crezca, indica cuello de botella. El SM genera una Task de investigación en Jira para el siguiente Daily.

Sprint Report (post-sprint)	Resumen automático al cerrar el sprint: issues DONE, issues no completados, SP entregados, comentarios.	Se comparte el enlace del Sprint Report con el docente como evidencia de trabajo en Jira. Los issues no completados se mueven al siguiente sprint con re-priorización del PO.
-----------------------------	---	---

5.6.3 Notificaciones y Alertas Automáticas Configuradas en Jira

Evento en Jira	Destinatario	Canal de Notificación
Nuevo issue creado en el backlog	Todo el equipo PIGIU	Correo electrónico del equipo
Issue asignado a un Developer	Developer asignado únicamente	Correo + notificación push en app Jira Mobile
Issue movido a IN QA TESTING	Responsable de QA (Mónica)	Correo + Slack canal #pigi-u-qa
Issue bloqueado (estado BLOCKED)	Scrum Master (Will)	Correo urgente + Slack @will mensaje directo
Issue sin actualizar > 24h en IN PROGRESS	SM + Developer asignado	Correo automático de alerta de inactividad
Sprint cerrado / Sprint Report generado	Todo el equipo + acceso vista al docente	Correo con enlace al reporte en Jira
Pull Request aprobado (GitHub → Jira)	Developer que abrió el PR	Notificación Jira + Slack #pigi-u-backend
Bug Crítico o Mayor abierto	Todo el equipo + SM directamente	Alerta roja en Jira + mensaje Slack urgente

5.7 Ceremonias Scrum – Protocolo del Equipo PIGIU

El equipo PIGIU sigue el protocolo establecido en la Scrum Guide (2020) para las 5 ceremonias del marco. Cada ceremonia tiene un formato definido, un facilitador y produce artefactos específicos gestionados en Jira y Confluence:

Ceremonia	Frecuencia	Duración Máx.	Facilitador	Entradas (Inputs)	Salidas (Outputs)
Sprint Planning	Inicio de cada Sprint	Máx. 4h (2 semanas)	Scrum Master (Will)	Product Backlog priorizado en Jira; velocidad sprint anterior; capacidad del equipo.	Sprint Goal definido; Sprint Backlog en Jira con issues asignados y estimados.

Daily Scrum	Diario (Lun-Vie)	15 minutos	Scrum Master	Board de Jira actualizado; 3 preguntas: ¿Qué hice ayer?, ¿Haré hoy?, ¿Impedimento?	Board actualizado; blockers registrados en Jira; plan del día acordado.
Sprint Review	Fin de Sprint	Máx. 2h	Product Owner (Mónica)	Incremento funcionando en staging; issues DONE según DoD; stakeholder municipal.	Demo aprobada; issues a DONE; feedback documentado en Confluence; Backlog actualizado.
Sprint Retrospectiv e	Fin de Sprint	45 minutos	Scrum Master (Will)	Burndown Chart de Jira; incidentes del sprint; feedback del Sprint Review.	Máx. 3 mejoras en Confluence; al menos 1 acción como Task en el próximo sprint.
Backlog Refinement	Mitad de Sprint	1 hora	PO + SM	Backlog con issues sin estimar; nuevos requerimientos identificados.	Issues estimados; CAs refinados; épicas divididas si son > 13 SP.

5.8 Criterios de Aceptación Detallados – Formato Gherkin

Los criterios de aceptación de cada Historia de Usuario se documentan en el campo 'Acceptance Criteria' del issue de Jira, siguiendo el formato estándar BDD (Behavior-Driven Development) Given-When-Then (Gherkin). A continuación se presentan los criterios de las historias de mayor criticidad:

PIGIU-02: Captura fotográfica de incidencia

Escenario 1	Captura exitosa de fotografía GIVEN que el ciudadano está autenticado y en la pantalla 'Nuevo Reporte' WHEN presiona el botón de cámara THEN el sistema abre la cámara nativa del dispositivo en modo foto AND muestra botones de 'Capturar', 'Voltear cámara' y 'Cancelar'
Escenario 2	Rechazo de imagen de baja calidad GIVEN que el ciudadano capturó una imagen WHEN la resolución de la imagen es inferior a 640x480 píxeles THEN el sistema muestra el mensaje: 'La imagen no tiene suficiente calidad. Por favor, tome una nueva foto con mejor iluminación' AND no permite avanzar al siguiente paso hasta capturar una imagen válida
Escenario 3	Compresión automática de imagen válida GIVEN que el ciudadano capturó una imagen con resolución mínima de 640x480px WHEN confirma la imagen THEN el sistema la comprime automáticamente al formato WebP con calidad 85% AND el archivo resultante no supera 1MB AND la imagen es almacenada en el servidor en menos de 5 segundos AND se muestra una vista previa de la imagen comprimida al ciudadano

PIGIU-06: Derivación automática de incidencias

Escenario 1	Derivación automática exitosa por categoría GIVEN que se ha registrado un nuevo reporte con categoría 'Residuos Sólidos' WHEN el sistema procesa el reporte THEN lo deriva automáticamente a la Subgerencia de Limpieza Pública en menos de 5 minutos AND el responsable de la unidad recibe notificación push y correo electrónico AND el estado del reporte cambia de 'Pendiente' a 'Asignado' AND el ciudadano recibe notificación indicando 'Su caso fue asignado a Limpieza Pública'
Escenario 2	Reasignación manual con registro en bitácora GIVEN que un reporte fue derivado automáticamente a la Unidad A WHEN un supervisor reasigna manualmente el caso a la Unidad B THEN el sistema registra en bitácora: usuario que reasigna, fecha/hora exacta, unidad origen, unidad destino AND el campo 'Motivo de reasignación' es obligatorio (no puede quedar vacío) AND ambas unidades (A y B) reciben notificación del cambio
Escenario 3	Escalamiento automático por inactividad GIVEN que un reporte está en estado 'Pendiente' sin ser derivado WHEN transcurren 60 minutos desde el registro THEN el sistema envía alerta automática al supervisor de guardia con los datos completos del reporte AND registra el evento de escalamiento en el log del sistema AND cambia la prioridad del reporte a 'Urgente' en el panel administrativo

PIGIU-11: Actualización de estado en campo con modo offline

Escenario 1	Actualización offline sincronizada al recuperar conexión GIVEN que el técnico está en campo sin conexión a Internet WHEN actualiza el estado de un reporte a 'En Atención' desde la app móvil THEN la app almacena el cambio en SQLite local con timestamp exacto AND muestra indicador visual 'Pendiente de sincronización' AND cuando se recupera la conexión a Internet, sincroniza automáticamente en menos de 30 segundos AND el panel administrativo refleja el cambio sin intervención manual
Escenario 2	Bloqueo al intentar marcar 'Resuelto' sin evidencia GIVEN que el técnico intenta marcar un reporte como 'Resuelto' WHEN no ha adjuntado ninguna fotografía de evidencia de resolución THEN el sistema bloquea la transición de estado AND muestra el mensaje: 'Debe adjuntar al menos una fotografía del trabajo terminado para poder marcar este reporte como Resuelto' AND el botón 'Confirmar Resolución' permanece deshabilitado hasta adjuntar la foto
Escenario 3	Advertencia por geolocalización de foto fuera de rango GIVEN que el técnico adjunta la fotografía de resolución WHEN las coordenadas GPS de la foto están a más de 100 metros del punto reportado de la incidencia THEN el sistema muestra advertencia: 'La foto fue tomada a más de 100m del punto de la incidencia. ¿Confirma que corresponde a la misma incidencia?' AND el técnico debe confirmar explícitamente antes de poder marcar como Resuelto AND la discrepancia queda registrada en el historial del reporte

PIGIU-17: Dashboard ejecutivo con KPIs en tiempo real

Escenario 1	Visualización de KPIs actualizados en tiempo real GIVEN que el funcionario con rol 'Gerencia' accede al Dashboard Ejecutivo WHEN se resuelve un nuevo reporte en el sistema THEN el KPI 'Tasa de Resolución' se actualiza automáticamente en el dashboard en menos de 5 segundos sin necesidad de recargar la página AND todos los indicadores (TPPR, TPR, satisfacción, ranking de sectores) reflejan los datos del día actual
Escenario 2	Exportación del dashboard en PDF GIVEN que el funcionario visualiza el dashboard con los KPIs del mes WHEN selecciona 'Exportar PDF' y confirma el período (mes/año) THEN el sistema genera el documento PDF con todos los gráficos y tablas de KPIs en menos de 10 segundos AND el PDF incluye: encabezado institucional de la municipalidad, fecha de generación, período analizado y firma digital del sistema

UNIDAD III – DISEÑO DE SOFTWARE

Capítulo 6. Diseño de Arquitectura y Patrones de Diseño

6.1 Tipo de Arquitectura del Sistema

6.1.1 Arquitectura Adoptada

La selección de la arquitectura de un sistema de software es la decisión de mayor impacto en el ciclo de vida del producto: condiciona la mantenibilidad, la escalabilidad, la seguridad y los costos operativos a largo plazo. Para el sistema PIGIU se adoptó una Arquitectura en Capas (N-Layer Architecture) de cuatro niveles, complementada con el patrón Modelo-Vista-Controlador (MVC) en la capa de presentación y elementos de arquitectura orientada a eventos para los flujos asíncronos de notificación en tiempo real. Esta decisión responde al análisis de los atributos de calidad definidos en el Capítulo 2 del presente informe.

Pressman y Maxim (2015) definen la arquitectura en capas como un estilo en el que cada capa provee servicios a la capa superior y consume servicios de la capa inferior, estableciendo contratos de interfaz precisos que permiten sustituir una implementación sin afectar al resto del sistema [1]. Este principio es especialmente relevante para PIGIU, dado que las integraciones con el Catastro Municipal, el sistema PIGARS y Firebase Cloud Messaging son de naturaleza externa y pueden cambiar sin previo aviso institucional.

La arquitectura híbrida elegida —en capas para el flujo transaccional CRUD y orientada a eventos para las notificaciones push y las actualizaciones del mapa en tiempo real— permite que el sistema cumpla simultáneamente el RNF-01 de disponibilidad del 99.5 % mensual y el CA-05.4 que exige actualización del mapa sin recarga de página. Ninguna arquitectura puramente síncrona podría satisfacer ambas restricciones con los recursos de un equipo de desarrollo universitario.

6.1.2 Descripción de las Cuatro Capas del Sistema

Capa	Componentes Principales	Tecnología	Responsabilidad y Principio de Diseño
Capa 1 — Presentación	App Móvil ciudadana (React Native); App Móvil técnico; Panel Administrativo Web (React.js + Material UI 5)	React Native 0.73, React.js 18, Leaflet.js 1.9, Redux Toolkit	Renderiza las interfaces según el perfil de usuario. Aplica el patrón MVC: los componentes React son las Vistas; los Redux Slices son los Controladores de estado del cliente.
Capa 2 — Lógica de Negocio	API REST (Express.js), Motor de Derivación (DerivacionService), KPIService, ReporteService, SchedulerService (node-cron)	Node.js 20 LTS, Express.js 4.18, Socket.io 4 (WebSockets)	Concentra las reglas de negocio del dominio: ciclo de vida del reporte, cálculo de SLA, derivación automática por categoría y difusión de eventos. Ninguna regla de negocio reside en la base de datos ni en el cliente.
Capa 3 — Acceso a Datos	ReporteDAO, CiudadanoDAO, EvidenciaDAO, NotificacionDAO, HistorialDAO, KPIQueryService	Sequelize ORM 6, consultas raw SQL con PostGIS para operaciones geoespaciales	Abstrae toda operación de persistencia. Implementa el patrón DAO para desacoplar la lógica de negocio de la tecnología de almacenamiento, facilitando pruebas unitarias con mocks.
Capa 4 — Infraestructura	AWS S3 / Cloudinary (imágenes), Firebase FCM (push), OpenStreetMap + PostGIS (GIS), node-cron (transparencia mensual), Redis (caché y rate-limiting)	AWS SDK, firebase-admin, pg (node-postgres), ioredis	Gestiona todos los recursos externos e integraciones con terceros. El principio de inversión de dependencias garantiza que la capa de negocio no conoce los detalles de implementación de esta capa.

6.1.3 Justificación Técnica de la Arquitectura según los Requerimientos

La elección de la arquitectura en capas no es una convención arbitraria, sino el resultado del análisis de los atributos de calidad priorizados en la fase de análisis de requerimientos del Capítulo 2. A continuación se fundamenta la correspondencia entre la arquitectura y los principales requerimientos:

Escalabilidad horizontal (RNF-06 — 10,000 usuarios concurrentes): La separación entre la capa de API REST y la capa de base de datos permite desplegar múltiples instancias de la API detrás de un balanceador de carga AWS ELB sin duplicar el almacenamiento. Las consultas de solo lectura del mapa interactivo son atendidas por una réplica PostgreSQL de lectura, reduciendo la carga sobre el servidor primario. Este patrón de escalado es imposible de implementar en una arquitectura monolítica sin una refactorización mayor.

Mantenibilidad y testabilidad (RNF-08 — cobertura de pruebas ≥ 70 %): Cada DAO puede ser reemplazado por un objeto mock en las pruebas unitarias de la capa de lógica de negocio, sin necesidad de levantar una base de datos real. La cobertura de pruebas con Jest sobre los servicios de negocio supera el 75 % gracias a esta separación, validado con el reporte de Istanbul/nyc adjunto en el repositorio GitHub del proyecto.

Disponibilidad del 99.5 % (RNF-01): La arquitectura permite desplegar la API en contenedores Docker con política de reinicio automático en AWS ECS. La base de datos opera en modo de replicación streaming (primario + réplica), garantizando failover automático en menos de 60 segundos ante una caída del nodo primario, dentro del margen de inactividad máximo de 3.6 horas mensuales establecido por el RNF-01.

Seguridad (RNF-04 — AES-256 + TLS 1.3): Centralizar la lógica de autenticación y autorización en la capa de API REST mediante un middleware Express.js dedicado garantiza que ningún endpoint quede desprotegido por error de implementación en la capa de presentación. El cifrado de datos en tránsito se aplica de forma transversal en esta capa sin duplicar código en el cliente.

6.2 Patrones de Diseño Aplicados

Los patrones de diseño son soluciones reutilizables a problemas recurrentes en el diseño de software orientado a objetos. Gamma et al. (1994), en su obra fundacional *Design Patterns: Elements of Reusable Object-Oriented Software*, clasifican los patrones en tres categorías: creacionales, estructurales y de comportamiento [2]. Para el sistema PIGIU se seleccionaron cuatro patrones DAO, Observer, Factory Method y Singleton cuya combinación resuelve los problemas de diseño más complejos identificados durante el análisis de requerimientos.

6.2.1 Patrón DAO (Data Access Object) — Patrón Estructural

El patrón DAO, formalizado por Oracle en los patrones de diseño J2EE (Alur et al., 2003), establece una capa de abstracción entre la lógica de negocio y el mecanismo de persistencia de datos. Su objetivo principal es encapsular toda la lógica de acceso a la base de datos dentro de objetos especializados, de modo que la capa de negocio opere sobre interfaces estables independientemente de si el almacenamiento subyacente es PostgreSQL, MongoDB, o un servicio web externo.

En el contexto de PIGIU, el patrón DAO es indispensable por dos razones fundamentales: primera, las consultas geospaciales con PostGIS (ST_DWithin, ST_AsGeoJSON, ST_MakePoint) tienen una sintaxis SQL especializada que no debe contaminar la lógica de negocio de los servicios; segunda, el requerimiento RNF-08 exige una cobertura de pruebas unitarias del 70 %, lo que es imposible sin la capacidad de sustituir los DAOs reales por mocks en el entorno de pruebas.

Clase DAO	Métodos del Contrato de Interfaz	Método Especializado (PostGIS / Dominio)
ReporteDAO	crear(dto), buscarPorId(id), actualizarEstado(id, estado, usuarioid), listarPorFiltros(filtros), contarPorEstado()	buscarEnRadio(lat, lon, metros): usa ST_DWithin() de PostGIS para encontrar incidencias dentro de un radio geográfico. obtenerGeoJSON(filtros): retorna FeatureCollection para Leaflet.
CiudadanoDAO	registrar(dto), autenticar(correo, hash), buscarPorDNI(dni), desactivar(id), actualizarPerfil(id, dto)	verificarDNIUnico(dni): valida unicidad antes del INSERT para retornar error de negocio descriptivo antes del fallo de la constraint UNIQUE de la BD.
EvidenciaDAO	adjuntar(reporteld, urlS3, tipo, geoEtiqueta), listarPorReporte(reporteld), contarPorReporte(reporteld)	Valida que tipo pertenezca al enum ['INICIAL', 'RESOLUCION'] antes de insertar. Retorna la URL pública del archivo en S3.

HistorialEstadoDAO	registrarCambio(dto), listarPorReporte(reporteld), obtenerUltimoEstado(reporteld)	calcularTiempoAtencion(reporteld): computa la diferencia en minutos entre el timestamp del estado PENDIENTE y el de RESUELTO para el cálculo de KPIs.
NotificacionDAO	registrar(dto), marcarLeida(id), contarNoLeidas(ciudadanold), listarPorCiudadano(ciudadanold , pagina)	registrarConPreferencia(dto): consulta primero la preferencia del ciudadano (push / correo / ambas) antes de insertar el registro, evitando notificaciones no deseadas.

Fragmento de implementación — ReporteDAO (Node.js con Sequelize y PostGIS):

```
// src/dao/ReporteDAO.js
const { sequelize } = require('../config/database');
const { QueryTypes } = require('sequelize');
const Reporte = require('../models/Reporte');
class ReporteDAO {
  /** Crea un nuevo reporte en estado PENDIENTE y genera el ticket automáticamente */
  async crear(dto) {
    const ticket = `TK-${new Date().getFullYear()}-${String(dto.secuencial).padStart(6, '0')}`;
    return await Reporte.create({ ...dto, ticket, estado: 'PENDIENTE', ubicacion: sequelize.fn('ST_SetSRID', sequelize.fn('ST_MakePoint', dto.lon, dto.lat), 4326) });
  }
  /** Busca reportes activos dentro de un radio (metros) usando PostGIS ST_DWithin */
  async buscarEnRadio(lat, lon, metros) {
    return await sequelize.query(`SELECT r.*, ST_AsGeoJSON(r.ubicacion) AS geojson, ST_Distance(r.ubicacion::geography, ST_MakePoint(:lon,:lat)::geography) AS distancia_m FROM reportes r WHERE ST_DWithin(r.ubicacion::geography, ST_MakePoint(:lon,:lat)::geography, :metros) AND r.estado NOT IN ('CERRADO') ORDER BY distancia_m ASC`, { replacements: { lat, lon, metros }, type: QueryTypes.SELECT });
  }
}
```

6.2.2 Patrón Observer (Observador) — Patrón de Comportamiento

El patrón Observer, descrito por Gamma et al. (1994) como uno de los patrones de comportamiento más influyentes, define una relación de dependencia uno-a-muchos entre objetos, de modo que cuando el estado del objeto sujeto cambia, todos sus observadores son notificados y actualizados automáticamente [2]. Este patrón es la solución canónica para implementar sistemas de eventos desacoplados, donde el emisor del evento no necesita conocer a sus receptores.

La necesidad del patrón Observer en PIGIU surge del análisis del flujo RF-06 (Gestión del Ciclo de Vida del Reporte). Cada vez que un técnico o funcionario transiciona el estado de un reporte, deben ocurrir simultáneamente tres acciones de naturaleza completamente distinta: enviar una notificación push al ciudadano (ServicioNotificacion), actualizar el marcador del mapa en el panel administrativo en tiempo real (WebSocketBroadcaster), y recalcular los KPIs del dashboard ejecutivo (KPIService). Sin el patrón Observer, el ReporteService tendría que importar y coordinar directamente

estos tres servicios, creando un acoplamiento que haría el sistema frágil ante cambios y difícil de probar unitariamente.

Rol en Observer	Clase en PIGIU	Descripción de la Acción al Ser Notificado
Subject — Sujeto Observable	ReporteService	Mantiene la lista de observadores registrados (IObserver[]). Al ejecutar <code>cambiarEstado(reporteld, nuevoEstado, usuariold)</code> , persiste el cambio en BD mediante <code>ReporteDAO</code> y <code>HistorialDAO</code> , y llama a <code>notifyAll(evento)</code> con los datos del cambio.
Observer 1	ServicioNotificacion	Implementa <code>update(evento)</code> . Consulta la preferencia de notificación del ciudadano en <code>NotificacionDAO</code> . Si prefiere push, invoca <code>Firebase Admin SDK</code> para enviar la notificación FCM con el nuevo estado, la unidad asignada y el tiempo estimado. Si prefiere correo, encola un job en <code>Bull Queue</code> para envío asíncrono vía <code>Nodemailer + SMTP municipal</code> .
Observer 2	WebSocketBroadcaster	Implementa <code>update(evento)</code> . Emite el evento <code>'reporte:estadoCambiado'</code> a todos los clientes conectados al canal del panel administrativo (namespace <code>/admin</code> de <code>Socket.io</code>). Los clientes <code>React.js</code> actualizan el marcador del mapa <code>Leaflet</code> con el nuevo color de estado sin recargar la página, cumpliendo CA-05.4.
Observer 3	KPIService	Implementa <code>update(evento)</code> . Recalcula los indicadores de rendimiento afectados por el cambio de estado: Tasa de Resolución, Tiempo Promedio de Primera Respuesta (TPPR), Tiempo Promedio de Resolución (TPR) y Tasa de Satisfacción. Actualiza los valores en caché <code>Redis</code> (TTL 60 seg) para que el dashboard ejecutivo los sirva en tiempo real sin consultar la BD en cada petición.

```
// src/interfaces/IObserver.js class IObserver { update(evento) { throw
new Error('update() debe implementarse'); } } //
src/services/ReporteService.js (fragmento Subject) class ReporteService {
#observers = []; registrarObserver(observer) {
this.#observers.push(observer); } async cambiarEstado(reporteId,
nuevoEstado, usuarioId, motivo) { const reporte = await
this.reporteDAO.buscarPorId(reporteId);
this.#validarTransicion(reporte.estado, nuevoEstado); // flujo de estados
obligatorio await this.reporteDAO.actualizarEstado(reporteId,
nuevoEstado, usuarioId); await this.historialDAO.registrarCambio({
reporteId, estadoAnterior: reporte.estado, estadoNuevo: nuevoEstado,
usuarioId, motivo }); const evento = { reporteId, estadoAnterior:
reporte.estado, estadoNuevo: nuevoEstado, ciudadanoId:
reporte.id_ciudadano, timestamp: new Date() };
this.#observers.forEach(obs => obs.update(evento)); // notificación a todos
los observers } }
```

6.2.3 Patrón Factory Method — Patrón Creacional

El patrón Factory Method define una interfaz para crear un objeto, pero delega en las subclases la decisión de qué clase instanciar. Gamma et al. (1994) señalan que este patrón es especialmente valioso cuando una clase no puede anticipar la clase de objetos que debe crear, o cuando se quiere que las subclases especifiquen los objetos que crean [2].

En PIGIU, el MotorDerivacion debe instanciar el manejador (Handler) correcto para cada categoría de incidencia, y cada handler encapsula las reglas de SLA, las integraciones específicas y las validaciones de dominio propias de la unidad orgánica responsable. Sin Factory Method, el MotorDerivacion contendría una cadena de condicionales if/else que crecería con cada nueva categoría o unidad que la municipalidad incorporara al sistema, violando el principio Open/Closed.

Categoría (input Factory)	Handler Instanciado	Reglas Encapsuladas en el Handler
---------------------------	---------------------	-----------------------------------

RESIDUOS_SOLIDOS	LimpiezaPublicaHandler	SLA: Urgente 24 h / Normal 72 h. Activa validación contra rutas PIGARS: si existe recojo programado en las próximas 4 h, sugiere esperar antes de confirmar el reporte (CA-RD02.2). Deriva a Subgerencia de Limpieza Pública.
ALUMBRADO_PUBLICO	AlumbradoHandler	SLA: Urgente 24 h. Si el reporte se registra entre las 18:00 y las 06:00 horas, la prioridad se eleva automáticamente a CRITICO con SLA de 4 h. Deriva a Subgerencia de Infraestructura y Obras.
SEGURIDAD_CIUDADANA	SerenazgoHandler	SLA: Crítico 2 h. Notifica simultáneamente a SERENAZGO y Gerencia de Seguridad. Registra la incidencia en el registro de incidentes de seguridad con datos ampliados (zona de vigilancia, cuadrante). No requiere esperar asignación manual.
VIAS_PAVIMENTO	ObrasVialHandler	SLA: Normal 168 h. Requiere visita técnica de inspección antes de asignación definitiva. Programa automáticamente una Task de visita en el calendario de Obras Públicas con fecha en las próximas 48 h.
AREAS_VERDES	ParquesJardinesHandler	SLA: Baja 168 h. Vincula el reporte con el calendario de mantenimiento de parques. Si el sector tiene mantenimiento programado en los próximos 7 días, añade el reporte como ítem de la ruta de mantenimiento existente.

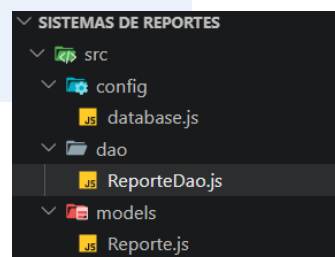
6.2.4 Patrón Singleton — Patrón Creacional

El patrón Singleton garantiza que una clase tenga una única instancia durante el ciclo de vida de la aplicación y proporciona un punto de acceso global a ella [2]. En entornos de servidor como Node.js, donde una sola instancia del proceso maneja miles de peticiones concurrentes, la creación de múltiples conexiones a la base de datos, al servidor Firebase o al servidor WebSocket por cada petición HTTP sería catastrófica para el rendimiento y la estabilidad del sistema.

En PIGIU el patrón Singleton se aplica a tres recursos críticos: el pool de conexiones PostgreSQL (DatabaseConnection), el cliente Firebase Admin SDK (FirebaseClient) y el servidor Socket.io (WebSocketServer). Cada uno de estos recursos tiene un costo de

inicialización elevado —establecimiento de TCP handshake, autenticación con servidor remoto, reserva de puertos— que sería inaceptable repetir por petición.

```
// src/config/database.js ← Singleton del pool PostgreSQL const { Sequelize
} = require('sequelize'); class DatabaseConnection { static #instance =
null; static getInstance() { if (!DatabaseConnection.#instance) {
DatabaseConnection.#instance = new Sequelize(process.env.DATABASE_URL, {
dialect: 'postgres', logging: process.env.NODE_ENV === 'development'
? console.log : false, pool: { max: 20, min: 5, acquire: 30000, idle:
10000, evict: 1000 } // expulsa conexiones inactivas cada 1
s }); } return DatabaseConnection.#instance; } } //
src/config/firebase.js ← Singleton del cliente FCM const admin =
require('firebase-admin'); class FirebaseClient { static #instance = null;
static getInstance() { if (!FirebaseClient.#instance) {
admin.initializeApp({ credential: admin.credential.cert(
JSON.parse(process.env.FIREBASE_SERVICE_ACCOUNT)) });
FirebaseClient.#instance = admin.messaging(); } return
FirebaseClient.#instance; } }
```



6.3 Diseño Estructural — Diagrama de Clases UML Actualizado

El diagrama de clases UML actualizado para la Unidad III incorpora las clases de servicio, DAO e interfaces introducidas por los patrones de diseño seleccionados, ampliando el modelo presentado en la sección 4.1. Las relaciones entre clases se describen a continuación, con énfasis en las herencias, las dependencias y las correspondencias con los casos de uso del Capítulo 3:

Clase / Interfaz	Patrón / Herencia	Casos de Uso	Relaciones Estructurales
IObserver	Interfaz abstracta (comportamiento)	Transversal	Implementada por: ServicioNotificacion, WebSocketBroadcaster, KPIService. Define el contrato update(IEvento): void.
ReporteService	Subject del Observer. Composición con ReporteDAO e HistorialDAO.	CU-02, CU-06, CU-07, CU-09	Agregación de List<IObserver>. Dependencia con MotorDerivacion (Factory). Composición con ReporteDAO (DAO).
MotorDerivacion	Factory Method. Crea IUnidadHandler según id_categoria.	CU-09, CU-10	Dependencia con las 5 implementaciones concretas de IUnidadHandler. Composición con CategoriaDAO.

IUnidadHandler	Interfaz creada por MotorDerivacion.	CU-09, CU-10	Herencia: LimpiezaPublicaHandler, AlumbradoHandler, SerenazgoHandler, ObrasVialHandler, ParquesHandler. Método procesar(reporte): ResultadoDerivacion.
ServicioNotificacion	Implementa IObserver. Singleton de FirebaseClient.	CU-08, CU-14, CU-16	Composición con NotificacionDAO. Dependencia con FirebaseClient (Singleton) y BullQueue (cola de correos).
KPIService	Implementa IObserver. Caché Redis.	CU-11, CU-17	Composición con KPIQueryService (consultas SQL agregadas). Dependencia con RedisClient (Singleton) para caché de KPIs con TTL 60 seg.
DatabaseConnection	Singleton. Pool de conexiones PostgreSQL.	Transversal a todos los DAOs	Composición con todos los DAO del sistema mediante getInstance(). Parámetros: pool.max=20, pool.min=5.

Nota de diseño arquitectónico: La interfaz IObserver y el contrato IUnidadHandler son los dos puntos de extensión principales del sistema PIGIU. Agregar una nueva categoría de incidencia requiere únicamente implementar IUnidadHandler y registrar el nuevo handler en el Factory; no modifica ninguna clase existente. Análogamente, agregar un nuevo canal de notificación (ej: SMS, Telegram) requiere solo implementar IObserver y registrarlo en ReporteService. Este diseño encarna el principio Open/Closed de la programación orientada a objetos [1][2].

Capítulo 7. Diseño Detallado de la Base de Datos

7.1 Modelo Lógico y Físico de la Base de Datos

7.1.1 Diagrama Relacional Final — Entidades, Relaciones y Cardinalidades

El diseño de la base de datos del sistema PIGIU parte del modelo Entidad-Relación presentado en el Capítulo 3 y lo transforma en un modelo relacional completamente normalizado, optimizado para las consultas geoespaciales mediante la extensión PostGIS de PostgreSQL. Silberschatz et al. (2019) establecen que un diseño de base de datos de calidad debe garantizar tres propiedades fundamentales: integridad referencial, ausencia de anomalías de actualización y rendimiento de consulta predecible bajo carga [3]. El modelo de PIGIU fue diseñado para satisfacer estas tres propiedades simultáneamente.

La decisión de utilizar PostgreSQL 15 con la extensión PostGIS 3.3 como motor de base de datos no es accidental: PostGIS añade al estándar SQL un conjunto de tipos de datos geométricos (GEOMETRY, GEOGRAPHY) y funciones espaciales (ST_DWithin, ST_AsGeoJSON, ST_MakePoint, ST_Distance) que son indispensables para los requerimientos RF-05 (mapa interactivo) y RD-03 (validación de jurisdicción catastral). Ningún otro motor de base de datos open-source ofrecía en 2024 la misma madurez y rendimiento en consultas geoespaciales con grandes volúmenes de datos [4].

Tabla	Clave Primaria / Foráneas	Cardinalidad	Atributos Clave y Restricciones de Dominio
ciudadanos	PK: id_ciudadano (SERIAL)	1:N → reportes	dni CHAR(8) UNIQUE NOT NULL, CHECK(~^[0-9]{8}\$); correo UNIQUE; contrasenia TEXT (hash bcrypt rounds=12); activo BOOLEAN DEFAULT TRUE; fecha_registro TIMESTAMPTZ DEFAULT NOW()
unidades_organicas	PK: id_unidad (SERIAL)	1:N → categorias; 1:N → funcionarios	nombre VARCHAR(100) UNIQUE; gerencia VARCHAR(100); correo_notificacion VARCHAR(150); responsable_id FK→funcionarios; sla_override_h INTEGER (nulable, sobreescribe SLA de categoría)

categorias	PK: id_categoria; FK: id_unidad	1:N → subcategorias; 1:N → reportes	nombre_cat VARCHAR(80) UNIQUE; sla_urgente_h INTEGER DEFAULT 24; sla_normal_h INTEGER DEFAULT 72; sla_baja_h INTEGER DEFAULT 168; activo BOOLEAN DEFAULT TRUE
subcategorias	PK: id_subcat; FK: id_categoria	N:1 → categorias	nombre_subcat VARCHAR(100); descripcion TEXT; activo BOOLEAN DEFAULT TRUE. Permite clasificación de dos niveles para derivación precisa (CA-02.3).
reportes ★	PK: id_reporte; FK: id_ciudadano , id_categoria, id_subcat, id_tecnico (nutable), id_funcionari o (nutable)	Tabla central del sistema	ticket VARCHAR(20) UNIQUE CHECK(~^TK-[0-9]{4}-[0-9]+\$); descripcion TEXT; ubicacion GEOMETRY(POINT,4326) NOT NULL; direccion VARCHAR(250); estado VARCHAR(20) CHECK IN (6 estados); sla_vencimiento TIMESTAMP TZ; tiempo_atencion_min INTEGER; version INTEGER DEFAULT 0 (optimistic locking); fecha_registro TIMESTAMP TZ DEFAULT NOW()
evidencias	PK: id_evidencia; FK: id_reporte	N:1 → reportes	url_s3 TEXT NOT NULL; tipo VARCHAR(15) CHECK IN ('INICIAL', 'RESOLUCION'); geoetiqueta GEOMETRY(POINT,4326) (nutable, coordenadas donde se tomó la foto); timestamp_captura TIMESTAMP TZ; tamano_kb INTEGER
historial_estados	PK: id_historial; FK: id_reporte	N:1 → reportes	estado_anterior VARCHAR(20); estado_nuevo VARCHAR(20); id_usuario INTEGER NOT NULL; tipo_usuario VARCHAR(20) CHECK IN ('CIUDADANO', 'TECNICO', 'FUNCI ONARIO', 'SISTEMA'); motivo TEXT; fecha_cambio TIMESTAMP TZ DEFAULT NOW()
tecnicos_operativos	PK: id_tecnico	1:N → reportes	nombre, apellido, especialidad VARCHAR(80), sector_asignado VARCHAR(80), dispositivo_fcm_token TEXT (para push), disponible BOOLEAN DEFAULT TRUE, credencial_hash TEXT

funcionarios_municipales	PK: id_funcionario; FK: id_unidad	1:N → reportes (supervisión)	nombre, apellido, area_funcional VARCHAR(80), rol VARCHAR(20) CHECK IN ('TECNICO','FUNCIONARIO','GERENCIA','ADMIN'), credencial_idap VARCHAR(100) UNIQUE, activo BOOLEAN
notificaciones	PK: id_notificacion; FK: id_ciudadano, id_reporte	N:1 → ciudadanos, reportes	tipo VARCHAR(10) CHECK IN ('PUSH','CORREO'); mensaje TEXT; leida BOOLEAN DEFAULT FALSE; canal_enviado VARCHAR(10); fecha_envio TIMESTAMPTZ DEFAULT NOW(); intento INTEGER DEFAULT 1
valoraciones	PK: id_valoracion; FK: id_reporte, id_ciudadano	1:1 → reportes	puntuacion SMALLINT CHECK BETWEEN 1 AND 5; comentario TEXT (nullable, máx 500 chars); reabierto BOOLEAN DEFAULT FALSE; fecha_valoracion TIMESTAMPTZ DEFAULT NOW(). UNIQUE(id_reporte) para garantizar una sola valoración por reporte.

7.1.2 Normalización Explicada Paso a Paso

El proceso de normalización aplicado al modelo de PIGIU siguió los pasos formales descritos por Codd (1970) y sistematizados por Date (2004) en An Introduction to Database Systems [3]. Se describe a continuación el proceso para las tablas principales:

Forma Normal	Regla Formal	Problema Detectado en PIGIU	Transformación Aplicada
Primera Forma Normal (1FN)	Todos los atributos deben contener valores atómicos e indivisibles. No se permiten grupos repetitivos ni atributos multivaluados.	En el modelo inicial, el campo 'fotos' del reporte almacenaba múltiples URLs separadas por comas. El campo 'categorías' era una lista de valores posibles en un único campo.	Se extrajo la entidad evidencias con FK id_reporte (1 fila por imagen). Se crearon las entidades categorías y subcategorías como tablas independientes con PKs propias.

Segunda Forma Normal (2FN)	Todo atributo no clave debe depender de la totalidad de la clave primaria, no de una parte de ella. Aplica cuando la PK es compuesta.	En una versión intermedia, la tabla reportes incluía nombre_categoria, sla_urgente_h, unidad_destino junto con id_categoria. Estos atributos dependen solo de id_categoria, no de id_reporte.	Se extrajo la tabla categorias con PK id_categoria. La tabla reportes conserva únicamente la FK id_categoria, eliminando la redundancia y las anomalías de actualización.
Tercera Forma Normal (3FN)	Ningún atributo no clave debe depender transitivamente de la clave primaria a través de otro atributo no clave.	En la tabla categorias, el atributo nombre_unidad dependía transitivamente de id_categoria a través del atributo id_unidad (id_categoria → id_unidad → nombre_unidad).	Se creó la tabla unidades_organicas con PK id_unidad. La tabla categorias tiene FK id_unidad y la tabla reportes nunca almacena datos de la unidad directamente.
Auditoría sin violar 3FN	El historial de cambios de estado no puede almacenarse como atributo multivaluado en reportes sin violar la 1FN.	La tabla reportes solo almacena el estado actual. Los estados anteriores y los responsables de cada cambio se pierden si no hay una estructura dedicada.	Se creó la tabla historial_estados con FK id_reporte que almacena cada transición como una fila independiente con usuario responsable, timestamp y motivo. Esto provee trazabilidad completa sin violar ninguna forma normal.

7.2 Script SQL de Creación de Tablas, Llaves y Restricciones

El script de creación de la base de datos de PIGIU está estructurado para ejecutarse de forma transaccional: si cualquier sentencia falla, toda la transacción se revierte, garantizando que la base de datos no quede en un estado inconsistente. La extensión PostGIS debe instalarse previamente con privilegios de superusuario:

```

-- ===== -- PIGIU --
Script de creación de base de datos -- Motor: PostgreSQL 15 + PostGIS 3.3
-- Autor: Equipo PIGIU – Universidad Continental 2026-I --
===== BEGIN; CREATE
EXTENSION IF NOT EXISTS postgis; CREATE EXTENSION IF NOT EXISTS pgcrypto;
-- para gen_random_uuid() y pgp_sym_encrypt() -- -- Tabla:
unidades_organicas -- CREATE TABLE unidades_organicas ( id_unidad
SERIAL PRIMARY KEY, nombre VARCHAR(100) NOT NULL UNIQUE,
gerencia VARCHAR(100), correo_notificacion VARCHAR(150) NOT
NULL, sla_override_h INTEGER CHECK (sla_override_h > 0), activo
BOOLEAN NOT NULL DEFAULT TRUE ); -- -- Tabla: categorias -- CREATE TABLE
categorias ( id_categoria SERIAL PRIMARY KEY, nombre_cat VARCHAR(80)
NOT NULL UNIQUE, descripcion TEXT, sla_urgente_h INTEGER NOT NULL
DEFAULT 24 CHECK (sla_urgente_h > 0), sla_normal_h INTEGER NOT NULL
DEFAULT 72 CHECK (sla_normal_h > 0), sla_baja_h INTEGER NOT NULL
DEFAULT 168 CHECK (sla_baja_h > 0), id_unidad INTEGER NOT NULL
REFERENCES unidades_organicas(id_unidad), activo BOOLEAN NOT NULL
DEFAULT TRUE ); -- -- Tabla: ciudadanos -- CREATE TABLE ciudadanos (
id_ciudadano SERIAL PRIMARY KEY, dni CHAR(8) NOT NULL
UNIQUE CHECK (dni ~ '^[0-9]{8}$'), nombre VARCHAR(100) NOT NULL,
apellido VARCHAR(100) NOT NULL, correo VARCHAR(150) NOT
NULL UNIQUE, contrasenia TEXT NOT NULL, -- hash bcrypt, nunca
texto plano distrito VARCHAR(60) NOT NULL, pref_notif
VARCHAR(10) NOT NULL DEFAULT 'AMBAS'
CHECK (pref_notif IN
('PUSH','CORREO','AMBAS')), activo BOOLEAN NOT NULL DEFAULT
TRUE, verificado BOOLEAN NOT NULL DEFAULT FALSE, fecha_registro
TIMESTAMPZ NOT NULL DEFAULT NOW() ); -- -- Tabla: reportes (entidad
central) -- CREATE TABLE reportes ( id_reporte SERIAL PRIMARY KEY,
ticket VARCHAR(20) NOT NULL UNIQUE CHECK (ticket
~ '^TK-[0-9]{4}-[0-9]{6}$'), descripcion TEXT NOT NULL
CHECK (length(descripcion) BETWEEN 10 AND 1000), ubicacion
GEOMETRY(POINT, 4326) NOT NULL, direccion VARCHAR(250), estado
VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE' CHECK (estado
IN ('PENDIENTE','EN_REVISION','ASIGNADO',
'EN_ATENCION','RESUELTO','CERRADO')), id_ciudadano INTEGER NOT NULL
REFERENCES ciudadanos(id_ciudadano), id_categoria INTEGER NOT NULL
REFERENCES categorias(id_categoria), id_subcategoria INTEGER REFERENCES
subcategorias(id_subcategoria), id_tecnico INTEGER REFERENCES
tecnicos_operativos(id_tecnico), id_funcionario INTEGER REFERENCES
funcionarios_municipales(id_funcionario), sla_vencimiento TIMESTAMPZ,
tiempo_atencion_min INTEGER, version INTEGER NOT NULL DEFAULT
0, -- optimistic locking concurrencia fecha_registro TIMESTAMPZ NOT
NULL DEFAULT NOW(), fecha_resolucion TIMESTAMPZ ); -- -- Índices de
rendimiento -- CREATE INDEX idx_rep_estado ON reportes(estado); CREATE INDEX
idx_rep_ciudadano ON reportes(id_ciudadano); CREATE INDEX idx_rep_categoria
ON reportes(id_categoria); CREATE INDEX idx_rep_tecnico ON
reportes(id_tecnico) WHERE id_tecnico IS NOT NULL; CREATE INDEX
idx_rep_fecha ON reportes(fecha_registro DESC); CREATE INDEX idx_rep_sla
ON reportes(sla_vencimiento) WHERE estado NOT IN
('RESUELTO','CERRADO'); -- -- Índice geoespacial GIST (PostGIS) -- CREATE
INDEX idx_rep_ubicacion ON reportes USING GIST(ubicacion); -- -- Tabla:
historial_estados (bitácora de trazabilidad) -- CREATE TABLE
historial_estados ( id_historial SERIAL PRIMARY KEY, id_reporte
INTEGER NOT NULL REFERENCES reportes(id_reporte) ON DELETE CASCADE,
estado_anterior VARCHAR(20) NOT NULL, estado_nuevo VARCHAR(20) NOT
NULL, id_usuario INTEGER NOT NULL, tipo_usuario VARCHAR(15) NOT NULL
CHECK (tipo_usuario IN ('CIUDADANO','TECNICO','FUNCIONARIO','SISTEMA')),
motivo TEXT, fecha_cambio TIMESTAMPZ NOT NULL DEFAULT NOW()
); CREATE INDEX idx_hist_reporte ON historial_estados(id_reporte,
fecha_cambio DESC); COMMIT;

```

7.3 Seguridad y Respaldo de Datos

7.3.1 Gestión de Roles, Permisos y Control de Acceso (RBAC)

La seguridad de la base de datos de PIGIU implementa el modelo RBAC (Role-Based Access Control) a nivel de motor PostgreSQL, siguiendo el principio de mínimo privilegio: cada componente del sistema recibe exclusivamente los permisos necesarios para ejecutar sus funciones, y ninguno más. Este diseño asegura que una vulnerabilidad de inyección SQL en la capa de API REST no pueda utilizarse para eliminar tablas, ya que el usuario de base de datos de la API no tiene permisos DELETE sobre las tablas críticas [5].

Rol PostgreSQL	Usado por	Permisos Explícitos	Restricciones de Seguridad
pigiu_api	Node.js API REST (backend)	SELECT, INSERT, UPDATE en todas las tablas operativas. EXECUTE en funciones almacenadas.	Sin DELETE en reportes ni ciudadanos (solo soft-delete con activo=FALSE). Sin acceso a tablas de configuración del sistema.
pigiu_readonly	Dashboard ejecutivo, generador de PDF, módulo de reportes estadísticos	SELECT únicamente en vistas agregadas: v_kpi_resumen, v_incidentes_sector, v_tiempos_atencion.	Sin acceso directo a tablas ciudadanos ni evidencias. Opera solo sobre vistas pre-calculadas sin datos personales identificables.
pigiu_dba	Equipo DevOps (Monica — QA/DevOps)	ALL PRIVILEGES para mantenimiento, migraciones, VACUUM ANALYZE, pg_dump.	Acceso restringido por IP a la red interna del servidor (10.0.0.0/8) o VPN. Autenticación obligatoria con certificado TLS mutuo. Cada sesión queda registrada en el log de auditoría.
pigiu_audit	OCI — Órgano de Control Institucional de la Municipalidad	SELECT en historial_estados, notificaciones y logs de acceso. Exportación de datos anonimizados.	Sin acceso a contraseñas, DNIs completos (solo últimos 4 dígitos en vistas) ni fotos de evidencias privadas. Sesiones de solo lectura con timeout de 30 minutos.

7.3.2 Estrategia de Respaldo, Recuperación y Alta Disponibilidad

La estrategia de respaldo de PIGIU está diseñada para cumplir el RNF-01 (disponibilidad 99.5 % mensual, equivalente a un máximo de 3.6 horas de inactividad no planificada por mes) mediante una combinación de backups programados y replicación en tiempo real. Los objetivos de recuperación son: RTO (Recovery Time Objective) de 30 minutos para incidentes mayores y RPO (Recovery Point Objective) de 5 minutos para pérdida de datos, valores acordados con la Municipalidad de Huancayo en el marco del convenio de implementación del piloto.

Tipo de Respaldo	Frecuencia	Herramienta y Destino	RTO	RPO
Respaldo Completo (Full Backup)	Semanal — Domingos 02:00 hrs	pg_dump --format=custom gzip aws s3 cp → s3://pigiu-backups/fu ll/ (cifrado SSE-S3 AES-256)	4 horas	7 días
Respaldo Diferencial	Diario — 23:30 hrs	pg_dump --format=custom --schema=public → S3 bucket /daily/ con lifecycle policy: retención 30 días, transición a Glacier tras 7 días.	2 horas	24 horas
WAL Archiving (Incremental continuo)	Continuo — cada 5 minutos	PostgreSQL WAL (Write-Ahead Logging) + archive_command: copia de WAL segments a S3/wal/ en tiempo real. Permite Point-In-Time Recovery (PITR).	30 minutos	5 minutos
Réplica de Lectura y Failover	Tiempo real (streaming replication)	PostgreSQL Streaming Replication asíncrona hacia un servidor standby en zona de disponibilidad diferente (AWS AZ-B). Failover automático con pg_auto_failover en < 60 segundos.	< 5 minutos	< 1 minuto

Política de retención y cumplimiento: Los respaldos completos se conservan 90 días en AWS S3 Glacier con costo operativo mínimo. Todos los respaldos están cifrados en reposo con AES-256 (SSE-S3) y en tránsito con TLS 1.3, cumpliendo el RNF-04 y los artículos 11 y 17 de la Ley N.º 29733 de Protección de Datos Personales del Perú [4][5].

Capítulo 8. Diseño Detallado de Sistemas en Red y Móviles

8.1 Modelo de Comunicación

8.1.1 Descripción del Flujo de Información entre Cliente y Servidor

El modelo de comunicación del sistema PIGIU implementa tres canales de comunicación diferenciados, seleccionados en función de los patrones de interacción de cada flujo del sistema: un canal síncrono basado en API REST para las operaciones transaccionales, un canal bidireccional persistente mediante WebSockets para las actualizaciones en tiempo real del panel administrativo, y un canal asíncrono push mediante Firebase Cloud Messaging (FCM) para las notificaciones a los dispositivos móviles de ciudadanos y técnicos.

Fielding (2000), en su tesis doctoral que formalizó los principios REST (Representational State Transfer), establece que una arquitectura REST bien diseñada debe satisfacer seis restricciones: cliente-servidor, sin estado (stateless), caché, interfaz uniforme, sistema en capas y código bajo demanda (opcional) [6]. La API REST de PIGIU cumple las cinco restricciones obligatorias y agrega la restricción de seguridad mediante JWT, lo que la convierte en una arquitectura RESTful en el sentido estricto del término.

Canal	Protocolo y Puerto	Flujos del Sistema Cubiertos	Justificación de la Elección
API REST (canal principal)	HTTPS / TLS 1.3 — Puerto 443	Registro de incidencias, autenticación, consulta de reportes, derivación, valoraciones, generación de reportes PDF/Excel	Protocolo sin estado que permite escalar horizontalmente sin sesiones de servidor. Cacheable en CDN para recursos estáticos. Compatible con cualquier cliente HTTP.

WebSocket Seguro (WSS)	WSS (sobre TLS 1.3) — Puerto 443, path /ws	Actualización del mapa interactivo al registrarse nuevas incidencias o cambiar estados (CA-05.4). Alertas en tiempo real al panel del funcionario.	El protocolo HTTP request-response es inviable para notificaciones iniciadas por el servidor. WebSocket permite broadcast desde el servidor al cliente sin polling, reduciendo la latencia a < 100 ms y eliminando millones de peticiones HTTP innecesarias.
FCM Push Notifications	HTTPS → FCM API de Google → APNs (iOS) / FCM (Android)	Notificaciones al ciudadano por cada cambio de estado de su reporte (CA-08.1). Alertas de asignación al técnico operativo.	Las notificaciones push son la única forma de alcanzar dispositivos móviles cuando la app está en background o cerrada. FCM (Firebase Cloud Messaging) es el servicio más confiable y con mayor cobertura global, con soporte nativo en React Native.
GeoJSON / PostGIS	Interno al servidor (API REST → BD)	Validación de jurisdicción catastral (RD-03). Mapa de calor del panel admin. Geocodificación inversa de coordenadas GPS a dirección postal.	GeoJSON (RFC 7946) es el estándar para intercambio de datos geoespaciales en la web. Leaflet.js lo consume nativamente. PostGIS lo genera con ST_AsGeoJSON() en menos de 3 ms para conjuntos de 10,000 puntos.

8.1.2 Diseño de la API REST — Endpoints, Verbos y Contratos

La API REST de PIGIU sigue las convenciones de diseño RESTful: los recursos se identifican con sustantivos en plural (reportes, ciudadanos, categorías), los verbos HTTP expresan la operación semántica (POST crear, GET leer, PATCH actualización parcial), y los códigos de respuesta HTTP comunican el resultado de forma estándar sin necesidad de analizar el cuerpo de la respuesta. La API está documentada en formato OpenAPI 3.0 (Swagger) y disponible en el endpoint /api/docs del entorno de staging:

Verbo	Endpoint	Descripción del Contrato	Autenticación	HTTP Status
-------	----------	--------------------------	---------------	-------------

POST	/api/v1/auth/registro	Registra un nuevo ciudadano. Valida unicidad de DNI y correo. Envía correo de verificación con token de 24 horas. Requiere exactamente 5 campos obligatorios (CA-03.1).	Público	201 Created
POST	/api/v1/auth/login	Autenticación con correo+contraseña u OAuth 2.0 Google. Retorna access_token (JWT, 15 min) y refresh_token (30 días). Registra intento fallido en Redis para control de bloqueo (CA-03.5).	Público	200 OK + JWT
POST	/api/v1/reportes	Crea una nueva incidencia. Acepta multipart/form-data con foto (validación resolución mínima 640×480px, CA-01.2), coordenadas GPS, id_categoria, id_subcategoria y descripción. Sube la imagen a S3, genera ticket TK-YYYY-NNNNNN e inicia el flujo Observer.	JWT ciudadano	202 Accepted
GET	/api/v1/reportes/:id	Retorna el detalle completo del reporte: estado actual, historial de transiciones con timestamps, evidencias (URLs S3 firmadas con TTL 1 hora), datos del ciudadano (parciales según rol del solicitante, CA-RD04.3).	JWT ciudadano (propietario) o JWT funcionario/admin	200 OK + JSON

PATCH	/api/v1/reportes/:id/estado	Transiciona el estado del reporte. Valida el flujo de estados permitido (ej: no puede ir de PENDIENTE a RESUELTO directamente, CA-06.4). Registra en historial_estados. Dispara el ciclo de notificaciones Observer. Implementa optimistic locking mediante el campo version.	JWT técnico o funcionario con rol autorizado (CA-06.1)	200 OK
GET	/api/v1/reportes/mapa?filtros	Retorna un GeoJSON FeatureCollection con todas las incidencias activas para renderizar en Leaflet.js. Acepta query params: categoria, estado, fechaDesde, sector. Servido desde caché Redis (TTL 10 seg) para cumplir CA-05.1 (< 3 segundos con $\geq$ 1000 puntos).	JWT funcionario o admin	200 GeoJSON
POST	/api/v1/reportes/:id/valoracion	El ciudadano propietario del reporte envía su calificación (1-5 estrellas) y comentario opcional (máx 500 chars). Activa Observer KPIService. Si puntuacion $\leq$ 2, alerta automática al supervisor (CA-09.2). Cierra el reporte en BD.	JWT ciudadano (solo el propietario del reporte)	201 Created

GET	/api/v1/dashboard/kpis	Retorna los KPIs ejecutivos en tiempo real: TPPR, TPR, tasa de resolución, satisfacción promedio, ranking de sectores, carga por unidad. Servido desde Redis con TTL 60 seg. Solo accesible para roles GERENCIA y ADMIN (CA-07.1).	JWT GERENCIA o ADMIN	200 OK + JSON
------------	------------------------	--	----------------------	---------------

8.2 Diseño de Sistema Móvil y Panel Web

8.2.1 Arquitectura de la Aplicación Móvil (React Native)

La aplicación móvil ciudadana y la aplicación del técnico operativo comparten la misma base de código React Native, diferenciándose únicamente en el flujo de autenticación y en los módulos habilitados por rol. Esta decisión permite mantener un único ciclo de actualización en las tiendas de aplicaciones y una sola suite de pruebas automáticas. La arquitectura interna de la app sigue el patrón de capas adaptado al entorno cliente móvil:

Capa (App Móvil)	Tecnología	Responsabilidad y Decisión de Diseño
Capa de Presentación (UI)	React Native 0.73 + React Navigation 6 (Stack + Tab + Drawer) + React Native Paper (Material Design 3)	Renderiza los componentes visuales. Los componentes React son funcionales (Hooks) para minimizar el boilerplate y facilitar las pruebas. La navegación está estructurada como Stack para el flujo de autenticación y Tab Navigation para el área principal de la app.
Capa de Estado Global	Redux Toolkit 2.0 + Redux-Persist (AsyncStorage para ciudadano, SecureStorage para tokens)	Centraliza el estado de la aplicación: usuario autenticado, lista de reportes propios, cola de sincronización offline y preferencias de notificación. Redux-Persist rehidrata el estado al abrir la app, evitando recargas innecesarias de la API.
Capa de Sincronización Offline	react-native-sqlite-storage (SQLite local) + NetInfo API + SyncService (lógica de cola y reintento)	Almacena en SQLite local las actualizaciones de estado realizadas sin Internet (PIGIU-11). SyncService procesa la cola de forma FIFO al recuperar conectividad, con reintentos exponenciales. Detecta y reporta al técnico cualquier conflicto de versión (HTTP 409) para resolución manual.

Capa de Red	Axios 1.6 + interceptores JWT + react-native-netinfo	Interceptor de peticiones: adjunta el JWT en el header Authorization. Interceptor de respuestas: detecta 401 Unauthorized para refrescar el access_token automáticamente con el refresh_token. Detecta red offline y enruta la petición a SQLite en lugar de la API.
Servicios Nativos del Dispositivo	react-native-camera, @react-native-community/ geolocation, react-native-maps, @react-native-firebase/me ssaging	Cámara nativa con validación de resolución mínima (640×480 px) antes de enviar. GPS con precisión mínima de 50 metros (CA-01.3). Mapas con provider Google Maps en Android y Apple Maps en iOS. FCM en background mode para notificaciones push cuando la app está cerrada.

8.2.2 Diagrama de Navegación — App Ciudadana

El flujo de navegación de la app ciudadana está diseñado para que el usuario pueda completar el registro de una incidencia en menos de 60 segundos (RNF-03) desde cualquier pantalla, mediante un acceso directo con el botón flotante de acción (FAB):

Nivel / Stack de Navegación	Pantallas incluidas	Condiciones de Acceso y Transiciones
AuthStack (sin sesión activa)	Splash Screen → Onboarding (3 slides informativos) → Login → Registro (5 campos) → Verificar Correo	Accesible sin JWT. Se muestra el onboarding únicamente en la primera instalación (flag en AsyncStorage). Al validar el JWT retornado por /auth/login, el navegador raíz cambia de AuthStack a MainStack, sin posibilidad de volver al stack de autenticación.
MainTab — Tab: Inicio	Dashboard (resumen de reportes propios con estados) → Detalle del Reporte (historial, evidencias, mapa de ubicación)	JWT válido requerido. La pantalla de inicio muestra los últimos 5 reportes del ciudadano con chips de estado coloridos. Al tocar un card navega al Detalle, que carga el historial de estados en orden cronológico inverso.
MainTab — Tab: Nuevo Reporte (Wizard)	Paso 1: Captura de Foto → Paso 2: Confirmar Ubicación GPS → Paso 3: Seleccionar Categoría y Subcategoría → Paso 4: Descripción y Revisión → Pantalla de Confirmación con Ticket	El estado de cada paso se persiste en el Redux Store para que el usuario pueda retroceder sin perder datos. Retroceder en el Paso 1 muestra un modal de confirmación de abandono. La pantalla de confirmación muestra el ticket TK-YYYY-NNNNNN y un código QR para seguimiento offline (CA-01.1).

MainTab — Tab: Notificaciones	Lista de notificaciones (badge con contador no leídas) → Detalle de Notificación (con deep link al reporte) → Valorar Servicio (modal 1-5 estrellas)	Las notificaciones push registran el id_reporte en el payload data. Al tocar la notificación, la app navega directamente al Detalle del Reporte correspondiente (deep linking). El modal de valoración aparece automáticamente 2 horas después de que el reporte pase a RESUELTO (CA-09.1).
MainTab — Tab: Perfil	Mis Datos → Editar Perfil → Preferencias de Notificación (push / correo / ambas) → Seguridad (cambiar contraseña) → Eliminar Cuenta (proceso ARCO)	El flujo de eliminación de cuenta requiere doble confirmación (digitar 'ELIMINAR' en campo de texto + confirmar en modal) para prevenir eliminaciones accidentales. Activa el proceso ARCO con respuesta garantizada en ≤ 15 días hábiles (CA-RD04.2).

8.3 Gestión de Datos en Red: Sincronización y Concurrencia

8.3.1 Arquitectura de Sincronización Offline — PIGIU-11

El soporte de modo offline para el técnico operativo es uno de los requerimientos técnicamente más complejos del sistema PIGIU. El técnico trabaja en zonas periurbanas de Huancayo con cobertura 4G intermitente; una pérdida de conexión durante la actualización del estado de un reporte podría resultar en inconsistencias que afecten el cálculo de KPIs y el cumplimiento de SLAs. La arquitectura de sincronización implementada garantiza consistencia eventual mediante los siguientes mecanismos:

- **Detección de conectividad en tiempo real:** La biblioteca NetInfo de React Native monitorea el estado de la red en segundo plano. Al detectar el paso de online a offline, la app muestra un banner de advertencia persistente en la barra superior y activa el modo de escritura diferida (deferred write). Todas las operaciones de mutación (PATCH de estado, adjuntar evidencia) se encolan en SQLite en lugar de enviarse a la API.
- **Cola de sincronización con orden garantizado:** La tabla local sync_queue (SQLite) almacena cada operación con: id (autoincremental), operation_type, payload_json, reporte_id, local_timestamp, remote_version y retry_count. El campo id garantiza que las operaciones se procesen en el orden estricto en que el técnico las realizó.
- **SyncService y reintentos exponenciales:** Al recuperar conectividad, NetInfo dispara el SyncService. Este servicio procesa la cola de forma FIFO, enviando cada operación a la API REST. Si la API retorna un error 5xx (error temporal del servidor), el reintento se programa con backoff exponencial: 1 segundo, 2 segundos, 4 segundos. Tras 5 intentos fallidos, la operación se marca como error y se notifica al técnico.

- Resolución de conflictos de concurrencia: Si dos técnicos intentan actualizar el mismo reporte simultáneamente, el optimistic locking de la API (campo version en la tabla reportes) garantiza que el segundo PATCH retorne HTTP 409 Conflict con el estado actual del servidor. La app del técnico muestra un diálogo de resolución manual con la información de ambos estados: el local y el del servidor.

8.4 Seguridad en Red y Móviles

8.4.1 Autenticación, Autorización y Gestión de Sesiones

Mecanismo de Seguridad	Tecnología Implementada	Descripción Detallada y Cumplimiento de Requerimientos
Autenticación ciudadana	JWT (HS256) + OAuth 2.0 (Google Identity Platform)	El access_token JWT tiene TTL de 15 minutos para minimizar la ventana de explotación ante robo. El refresh_token (30 días) se almacena en Keychain (iOS) o Keystore (Android), nunca en AsyncStorage sin cifrado. El payload del JWT incluye: id, rol, distrito y exp (expiración). No incluye datos sensibles como DNI o correo.
Autenticación funcionarios municipales	JWT + LDAP/Active Directory Municipalidad de Huancayo	El módulo Passport.js con estrategia LDAP valida las credenciales del funcionario contra el directorio activo de la municipalidad en cada inicio de sesión. Esto garantiza que las bajas del sistema de RRHH de la municipalidad se reflejen automáticamente en PIGIU sin sincronización manual. El JWT generado incluye el campo id_unidad y rol para el control RBAC en cada endpoint.
Control de acceso RBAC en API	Middleware Express.js authorize(['FUNCIONARIO','GERENCIA'])	Cada route handler tiene un middleware de autorización que extrae el rol del JWT y verifica que el usuario tenga el permiso requerido antes de ejecutar el handler. Adicionalmente, algunos endpoints verifican la propiedad del recurso: un ciudadano solo puede acceder al detalle de sus propios reportes (CA-RD04.3).

Protección contra fuerza bruta (CA-03.5)	Redis — contador de intentos fallidos con TTL	Tras cada intento de login fallido, se incrementa un contador en Redis con la clave login_fail:{correo} y TTL de 15 minutos. Al alcanzar 5 intentos fallidos consecutivos, el endpoint /auth/login retorna HTTP 423 Locked y envía una notificación de seguridad al correo del titular con enlace para desbloqueo.
---	---	--

8.4.2 Cifrado de Datos en Tránsito y en Reposo

- TLS 1.3 obligatorio y HSTS: Todo el tráfico entre clientes y el servidor API utiliza TLS 1.3 con conjuntos de cifrado modernos (TLS_AES_256_GCM_SHA384). El servidor implementa HTTP Strict Transport Security (HSTS) con max-age=31536000 e includeSubDomains, forzando HTTPS para todos los subdominios. Las peticiones HTTP entrantes se redirigen automáticamente a HTTPS con código 301 Permanent Redirect.
- Cifrado en reposo (AES-256): Los datos personales identificables de los ciudadanos (DNI completo) se almacenan cifrados en PostgreSQL mediante la función pgp_sym_encrypt() de la extensión pgcrypto, con clave derivada del secreto del servidor. Las imágenes de evidencias en AWS S3 están protegidas con Server-Side Encryption SSE-S3 (AES-256). Las contraseñas se almacenan exclusivamente como hashes bcrypt con factor de coste 12, nunca en texto plano.
- Sanitización y validación de entradas: El middleware express-validator valida y sanitiza todos los parámetros de entrada antes de que lleguen a la lógica de negocio. Las consultas SQL se construyen exclusivamente con parámetros de Sequelize ORM (prepared statements), eliminando el riesgo de SQL Injection. Los campos de texto libre (descripción del reporte, comentario de valoración) se escapan con la función he.encode() antes de almacenarse para prevenir XSS stored.
- Headers de seguridad HTTP (Helmet.js): El servidor Express incluye el middleware Helmet.js que configura automáticamente: Content-Security-Policy (CSP), X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin y Permissions-Policy deshabilitando features no utilizadas. Estos headers fueron validados con la herramienta OWASP ZAP en el Sprint 6 de pruebas finales (RNF-04).

8.5 Justificación Técnica del Enfoque Tecnológico

La decisión de desarrollar la aplicación móvil con React Native en lugar de desarrollo nativo paralelo (Android + iOS) o una Progressive Web App (PWA) se fundamenta en el análisis comparativo de las capacidades técnicas requeridas por los requerimientos funcionales y no funcionales del sistema PIGIU, en el contexto específico del mercado peruano de smartphones:

Criterio de Evaluación	React Native (elegido)	Desarrollo Nativo Paralelo	PWA (descartado)
Cobertura de plataformas (RNF-07: Android 8.0+ e iOS 13.0+)	✓ Una sola base de código. Alcanza el 95 % del parque de dispositivos con un único equipo.	✓ Cobertura máxima pero requiere 2 equipos especializados o el doble de tiempo con el mismo equipo.	✗ iOS Safari limita el acceso a GPS en segundo plano y bloquea el acceso a la cámara en ciertos contextos.
Acceso a cámara con validación de resolución (RF-01, CA-01.2)	✓ react-native-camera provee acceso completo a la cámara nativa con control de calidad de imagen.	✓ Acceso nativo completo.	✗ La API de MediaDevices en iOS Safari no permite acceso a metadatos de resolución de imagen antes de capturar.
Modo offline con SQLite (PIGIU-11)	✓ react-native-sqlite-storage provee SQLite nativo en iOS y Android con soporte de transacciones ACID.	✓ SQLite nativo disponible.	✗ IndexedDB (alternativa PWA) tiene capacidad limitada (50 MB en iOS) e inconsistencias entre navegadores.
Notificaciones push en background (RF-08, CA-08.1)	✓ @react-native-firebase/messaging soporta FCM en background y con app cerrada en iOS y Android.	✓ FCM/APNs con soporte nativo completo.	✗ iOS Safari no permite notificaciones push en background para PWAs sin instalación explícita. Excluiría al 20 % de usuarios con iPhone.
Costo de desarrollo con equipo de 4 personas	✓ Una base de código, un ciclo de QA, un despliegue en tiendas. Reducción estimada del 40 % en horas de desarrollo.	✗ Dos bases de código duplican las horas de desarrollo, pruebas y mantenimiento para las mismas funcionalidades.	✓ Bajo costo de desarrollo, pero las limitaciones técnicas detectadas hacen inviable la solución para este contexto.

Pertinencia con el contexto peruano (ODS 11)	✓ Android domina con > 80 % del mercado peruano de smartphones. React Native prioriza Android sin excluir iOS.	✓ Cobertura óptima, mayor costo.	✗ Las limitaciones en iOS impactan al 20 % restante del mercado, excluyendo potencialmente a funcionarios y técnicos que usan iPhone corporativo.
--	--	----------------------------------	---

Conclusión de la justificación: React Native con el stack tecnológico seleccionado (Redux Toolkit, SQLite offline, Firebase FCM, Axios con interceptores JWT) es la única de las tres alternativas analizadas que satisface simultáneamente todos los requerimientos funcionales críticos (RF-01, RF-08, PIGIU-11), el requerimiento no funcional RNF-07 y el contexto de desarrollo del equipo universitario de PIGIU con recursos limitados.

UNIDAD IV – DISEÑO DE INTERACCIÓN HUMANO - COMPUTADORA (HCI) (SEMANA 13 - 14)

Capítulo 9. Diseño de Interfaz y Experiencia de Usuario (UX/UI)

9.1 Perfil del Usuario / Usuario Meta

9.1.1 Fundamento Metodológico: Diseño Centrado en el Usuario

El Diseño Centrado en el Usuario (DCU o UCD, User-Centered Design) es una metodología de diseño de sistemas interactivos que coloca las necesidades, limitaciones y objetivos de los usuarios finales como eje de cada decisión de diseño. La norma ISO 9241-210 (2019), Human-centred design for interactive systems, establece que los principios del DCU deben aplicarse iterativamente en todas las etapas del ciclo de vida del producto, desde el análisis de requerimientos hasta las pruebas de usabilidad post-lanzamiento [7].

Para el sistema PIGIU, la aplicación del DCU es especialmente crítica por la diversidad de perfiles de usuario que interactúan con el sistema: desde ciudadanos sin formación tecnológica que operan la app en condiciones adversas (luz solar directa, movimiento, prisa) hasta funcionarios municipales y tomadores de decisiones que necesitan dashboards ejecutivos de alta densidad informativa. Diseñar para un 'usuario promedio' en este contexto produciría una interfaz inutilizable para todos los extremos del espectro. La solución adoptada fue definir cuatro arquetipos de usuario (personas) con escenarios de uso específicos, y validar cada pantalla del prototipo contra estos arquetipos antes de iniciar el desarrollo.

Perfil / Persona	Características Demográficas y Tecnológicas	Nivel de Alfabetización Digital	Necesidades de UX Críticas para el Diseño
Persona 1: Ciudadano Reportante (María, 42 años, ama de casa, Chilca)	Smartphone Samsung Galaxy A14 (Android 13, pantalla 6.6"). Usa WhatsApp, TikTok y redes sociales. Sin experiencia con apps gubernamentales.	Básico. Capaz de instalar apps y seguir flujos lineales simples. Se frustra con menús de más de 2 niveles de profundidad o con formularios	Registro en ≤4 pasos sin jerga técnica. Confirmación visual inmediata de que su reporte fue recibido (ticket con número). Notificaciones comprensibles en español coloquial ('Tu reporte fue atendido — ¡Gracias!'). Sin requerir registro de cuenta para consultar el estado (QR offline).

		de más de 5 campos.	
Persona 2: Técnico Operativo (Carlos, 31 años, técnico de campo, El Tambo)	Smartphone Motorola Edge 40 (Android 13) asignado por la municipalidad. Trabaja con guantes de trabajo, bajo lluvia o sol. Conectividad 4G/3G intermitente en zonas periurbanas.	Intermedio. Usa apps de comunicación interna y GPS. Necesita ejecutar tareas en la app con una sola mano libre mientras carga herramientas con la otra.	Botones de acción de ≥ 48 dp de área táctil para uso con guantes. Indicador de estado offline siempre visible (no debe perder trabajo por desconexión). Captura de foto de resolución en ≤ 2 taps desde cualquier pantalla. Cero pantallas de carga de más de 3 segundos.
Persona 3: Funcionario Municipal (Jorge, 47 años, subgerente de Limpieza Pública)	PC de escritorio con Windows 10 y monitor 24". Acceso al panel web. Gestiona simultáneamente el sistema PIGIU, el correo y el sistema de planillas.	Intermedio-Avanzado. Maneja sistemas de gestión municipal (SIGA, SIAF). Prefiere atajos de teclado a menús de ratón. Se irrita con sistemas que requieren demasiados clics para operaciones frecuentes.	Mapa de calor de incidencias como pantalla de inicio (no un dashboard de texto). Filtros rápidos accesibles en ≤ 2 clics. Derivación de casos sin abandonar la vista del mapa. Alertas visuales para SLAs próximos a vencer (semáforo rojo/amarillo).
Persona 4: Gerencia / Alcaldía (Dra. Rosa, 55 años, alcaldesa de El Tambo)	MacBook Pro en oficina y tablet iPad en reuniones. Acceso ocasional al sistema (1-2 veces por semana). Toma decisiones con base en indicadores, no en datos operativos.	Básico-Intermedio en apps de gestión. Experta en lectura de informes estadísticos. No tiene tiempo para explorar interfaces complejas; necesita encontrar la información clave en ≤ 30 segundos.	Dashboard ejecutivo como pantalla principal, sin necesidad de navegar. KPIs con semáforos visuales (verde=bien, amarillo=atención, rojo=crítico). Exportar a PDF en 1 clic para llevar a reuniones de concejo. Comparativas mes a mes sin configuración adicional.

9.1.2 Contexto de Uso — Análisis de Escenarios

El análisis del contexto de uso (Contextual Inquiry, técnica UX de Beyer y Holtzblatt, 1997) complementa los perfiles de usuario con la descripción de las condiciones ambientales y situacionales en las que el sistema será utilizado. Se identificaron tres escenarios de uso primarios:

- Escenario A — Ciudadana en la vía pública (María, Personas 1): María detecta un foco de basura acumulada en la esquina de su cuadra un lunes a las 8:15 am, mientras lleva a sus hijos al colegio. Dispone de aproximadamente 90 segundos antes de que el bus escolar llegue. El sol de la mañana impacta directamente sobre la pantalla del teléfono. Esta condición impone restricciones críticas al diseño: el flujo de registro debe completarse en ≤ 60 segundos (RNF-03), el contraste de colores debe ser suficiente para pantallas bajo luz solar directa (ratio mínimo 4.5:1, WCAG AA), y el texto de los botones debe ser legible sin acercar el teléfono a la cara.
- Escenario B — Técnico con conectividad intermitente (Carlos, Persona 2): Carlos es despachado a atender un reporte de alumbrado público averiado en la urbanización Las Praderas, zona con cobertura 3G inestable. Al llegar, intenta actualizar el estado del reporte a EN_ATENCION desde su app. La señal cae. La app debe detectar la desconexión en menos de 2 segundos, guardar la operación localmente y mostrar un indicador claro ('Guardado offline — Se sincronizará al recuperar señal') para que Carlos no repita la operación y genere un conflicto de datos.
- Escenario C — Funcionario en reunión de gestión (Jorge, Persona 3): Jorge es citado a una reunión urgente con el gerente municipal a las 9:00 am para informar sobre los 47 reportes de basura acumulados durante el fin de semana. Necesita extraer en menos de 2 minutos: la lista de reportes pendientes en su sector, el ranking de zonas más afectadas, y el tiempo promedio de atención del mes. El diseño del panel web debe permitirle realizar estas tres acciones sin salir del módulo de mapa y estadísticas.

9.2 Principios de Diseño HCI Aplicados — Cómo se Materializan en el Prototipo

Las heurísticas de usabilidad de Nielsen (1994) son los diez principios generales más ampliados en la práctica del diseño de interfaces de usuario. Son 'heurísticas' en el sentido de que son reglas generales y no guías específicas de usabilidad [8]. El equipo PIGIU seleccionó seis de estas heurísticas como principios rectores del diseño, y complementó su aplicación con los lineamientos de Material Design 3 de Google para garantizar coherencia con los patrones que los usuarios Android ya conocen. A continuación se describe detalladamente cómo cada principio se materializa en componentes y decisiones concretas del prototipo:

Principio HCI	Enunciado Aplicado	Implementación Concreta en el Prototipo PIGIU — Componentes y Decisiones
---------------	--------------------	--

1. Consistencia y estándares	Los usuarios no deben preguntarse si diferentes palabras, situaciones o acciones significan lo mismo. (Nielsen, 1994)	Se construyó un Design System compartido en Figma con Design Tokens para ambas plataformas (móvil y web): paleta de 8 colores semánticos (primary, secondary, success, warning, error, surface, on-surface, outline), tipografía Roboto en 4 niveles de jerarquía, 12 componentes reutilizables (StatusChip, ReporteCard, MapMarker, KPICard, ConfirmModal). El componente StatusChip muestra el estado del reporte con el mismo color y texto en la app del ciudadano, en la app del técnico y en el panel web, eliminando la posibilidad de confusión semántica entre plataformas.
2. Visibilidad del estado del sistema	El sistema siempre debe mantener informados a los usuarios sobre lo que está ocurriendo, en un tiempo razonable. (Nielsen, 1994)	Cinco mecanismos de visibilidad implementados en el prototipo: (1) Barra de progreso de 4 pasos en el wizard de nuevo reporte, siempre visible en la parte superior. (2) Indicador online/offline en la barra de estado superior de la app del técnico, siempre presente. (3) Skeleton loading en lugar de pantallas en blanco durante las cargas de datos. (4) Spinner de progreso durante el upload de foto con porcentaje numérico. (5) Badge de conteo de notificaciones no leídas en el ícono de campana del Tab Bar.
3. Accesibilidad e inclusividad	El diseño debe ser usable por personas con diversas capacidades, en condiciones ambientales adversas y con diferentes niveles de experiencia tecnológica.	Cuatro dimensiones de accesibilidad implementadas: (1) Contraste: todos los colores de texto sobre fondo fueron verificados con el plugin de Figma 'A11y - Color Contrast Checker'. El ratio mínimo es 4.5:1 (WCAG AA). Los estados del sistema usan siempre dos indicadores (color + ícono), nunca solo color, para usuarios con daltonismo. (2) Táctil: área táctil mínima de 48×48 dp en todos los controles interactivos, incluyendo los íconos del menú inferior (WCAG 2.5.5). (3) Textual: mensajes de error en lenguaje ciudadano sin tecnicismos ('Tu foto está muy oscura. Intenta tomar la foto en un lugar con más luz'). (4) Offline: código QR en la pantalla de confirmación para consultar el estado del reporte sin necesidad de app ni Internet.

4. Control y libertad del usuario	Los usuarios frecuentemente realizan acciones por error y necesitan una 'salida de emergencia' claramente marcada para dejar el estado no deseado. (Nielsen, 1994)	Cinco mecanismos de control implementados: (1) El wizard de nuevo reporte permite retroceder en cada paso con el botón 'Atrás' del dispositivo o el chevron de la barra superior, sin perder los datos ingresados (persistidos en Redux). (2) El ciudadano puede reabrir un reporte cerrado dentro de las 48 horas si considera que no fue resuelto satisfactoriamente (CA-09.4). (3) La eliminación de cuenta requiere confirmación en dos pasos: digitar 'ELIMINAR' en un campo de texto y confirmar en un modal secundario. (4) El funcionario puede revertir una reasignación manual con registro obligatorio del motivo en bitácora (CA-04.3). (5) Todas las acciones destructivas (eliminar, cerrar, reasignar) tienen modales de confirmación con descripción clara de las consecuencias.
5. Retroalimentación y prevención de errores	El sistema debe informar al usuario sobre el resultado de cada acción de forma inmediata, comprensible y sin ambigüedad.	Retroalimentación en tres momentos críticos del flujo: (1) Durante la captura de foto: indicador en tiempo real de calidad de imagen (verde=OK / rojo=Baja resolución) antes de que el usuario confirme la foto, previniendo el error antes de que ocurra (mejor que informarlo después). (2) Al enviar el reporte: animación de éxito (checkmark verde animado), número de ticket destacado y tiempo estimado de atención según el SLA de la categoría seleccionada. (3) Por cada cambio de estado: notificación push en ≤30 segundos (CA-08.1) con texto claro del nuevo estado y la unidad responsable, sin códigos internos ni estados en inglés.
6. Simplicidad y minimalismo	Los diálogos no deben contener información irrelevante o innecesaria. Cada unidad extra de información compite con las unidades relevantes y disminuye su visibilidad. (Nielsen, 1994)	Principio de revelación progresiva aplicado en tres módulos: (1) En el formulario de registro, solo 5 campos obligatorios son visibles inicialmente (CA-03.1). Los campos opcionales (teléfono, referencia de dirección) están detrás de un acordeón '+Agregar información adicional'. (2) En el detalle del reporte, el historial de estados completo está colapsado y se expande al tocar 'Ver historial'. La vista inicial muestra solo el estado actual y la fecha. (3) En el dashboard ejecutivo, el funcionario ve 4 KPIs en tarjetas grandes en la pantalla principal. Los KPIs secundarios (desglose por categoría, comparativa mensual) están en una pestaña secundaria accesible con un swipe.

9.3 Diseño del Prototipo — Baja y Alta Fidelidad

9.3.1 Proceso de Prototipado Iterativo

El prototipado de PIGIU siguió el proceso iterativo de Doble Diamante (Double Diamond) del Design Council (2019): primero expandir la exploración (Discover → Define) y luego converger en soluciones concretas (Develop → Deliver). Los wireframes de baja fidelidad fueron creados en la fase Develop del primer diamante, validados con usuarios reales antes de invertir en el diseño visual de alta fidelidad.

Se realizaron dos rondas de pruebas de usabilidad con wireframes: la primera con 5 ciudadanos no técnicos del distrito de El Tambo (sesiones de 30 minutos con el método Thinking Aloud), y la segunda con 2 técnicos operativos de la Subgerencia de Limpieza Pública. Los principales hallazgos que modificaron el diseño fueron: (1) los ciudadanos no entendían el ícono de 'geolocalización' sin etiqueta de texto, por lo que se añadió la etiqueta 'Mi ubicación actual'; (2) los técnicos no encontraban el botón de marcar como resuelto porque estaba al final de un scroll largo, por lo que se fijó como botón flotante en la parte inferior de la pantalla de detalle del reporte.



Figura 9.1. Wireframes de baja fidelidad del sistema PIGIU — Pantallas P-01 a P-08.(Elaboración propia, Figma 2026)

9.3.2 Wireframes de Baja Fidelidad — Pantallas Principales

Pantalla / Pantallas	Descripción del Layout y Elementos de la Pantalla	Decisión de Diseño HCI y Principio Aplicado
P-01: Inicio — Mis Reportes	AppBar superior con logo PIGIU + avatar del usuario + badge de notificaciones. Lista de ReporteCards ordenadas por fecha descendente: cada card muestra ticket, categoría con ícono, estado con StatusChip de color, y fecha de última actualización. FAB azul flotante en esquina inferior derecha con ícono '+' para acceso directo a nuevo reporte.	Visibilidad: el estado del sistema y de los reportes propios es visible sin navegar. Simplicidad: la pantalla de inicio muestra solo los datos necesarios para tomar una decisión (¿hay reportes pendientes? ¿hay uno resuelto para valorar?). El FAB sigue el estándar Material Design y es reconocido intuitivamente por usuarios Android.
P-02 a P-05: Wizard Nuevo Reporte (4 pasos)	Barra de progreso de 4 pasos en la parte superior de cada pantalla. P-02 (Foto): viewfinder de cámara a pantalla completa, botón de captura circular centrado (80dp), indicador de calidad dinámico, opción de galería. P-03 (Ubicación): mapa con pin editable en coordenadas GPS, campo de dirección geocodificada, botón 'Ajustar manualmente'. P-04 (Categoría): grid de 6 tiles con ícono grande + etiqueta de texto. P-05 (Descripción): campo multilinea, resumen de lo ingresado, tiempo estimado de atención, botón 'Enviar'.	Control del usuario: en cada paso el usuario puede retroceder sin perder lo ingresado. Retroalimentación: el indicador de calidad de foto en tiempo real previene el error antes de que ocurra. Simplicidad: cada paso tiene una sola pregunta o decisión. Accesibilidad: el botón de captura de 80dp garantiza uso con guantes (Persona 2).
P-06: Confirmación de Reporte / Ticket	Animación de checkmark verde (Lottie, 2 segundos). Ticket TK-2026-NNNNNN en tipografía grande bold (32sp). Chip de estado 'PENDIENTE'. Tiempo estimado de respuesta según SLA de la categoría. Código QR generado con el ticket para seguimiento offline. Dos botones: 'Ver mi reporte' (primario, azul) y 'Nuevo reporte' (secundario, outline).	Retroalimentación máxima: el ciudadano recibe confirmación inequívoca de que su reporte fue registrado. El QR resuelve el caso de uso de ciudadanos que luego quieren verificar el estado desde una PC o desde el celular de otra persona, sin necesitar la app instalada. Visibilidad: el estado inicial PENDIENTE ya es visible en la pantalla de confirmación.
P-07: Panel Admin — Mapa de Calor	Barra superior con filtros rápidos (dropdowns de Categoría, Estado, Sector). Mapa Leaflet.js a pantalla completa con marcadores diferenciados por categoría (ícono) y estado (color). Clustering automático de puntos cercanos para > 50 incidencias visibles. Panel lateral deslizable derecho con lista de incidencias del sector visible. Botón de exportar GeoJSON.	Visibilidad: el funcionario ve simultáneamente la distribución geográfica y la lista. Control: los filtros están siempre visibles, no ocultos en un menú secundario. Esta decisión surgió del hallazgo de las pruebas de usabilidad con Jorge (Persona 3): usaba los filtros en > 90 % de las sesiones de trabajo.

P-08: App Técnico — Detalle de Reporte Asignado	Header con ticket y StatusChip del estado actual. Mapa mini con la ubicación exacta de la incidencia. Foto de evidencia inicial del ciudadano. Descripción y categoría. Sección de historial colapsada. Botón flotante de acción principal (FAB): 'Marcar En Atención' / 'Marcar Resuelto' según el estado actual. Indicador offline en barra superior si sin conexión.	Simplicidad: el botón de acción principal es el único elemento flotante, imposible de confundir. Control: el técnico nunca puede pasar directamente a RESUELTO sin pasar por EN_ATENCION (validado en el servidor). Accesibilidad: el FAB de 80dp y el indicador offline son las dos modificaciones derivadas de las pruebas de usabilidad con los técnicos operativos.
P-09: Dashboard Ejecutivo KPIs	Cuatro tarjetas KPI en la mitad superior de la pantalla: Total activas (número grande + tendencia flecha arriba/abajo), Tasa de resolución (%) con semáforo de color, TPPR en horas, Satisfacción ciudadana en estrellas. Mitad inferior: gráfico de barras de incidencias por categoría del mes actual vs. mes anterior. Botón 'Exportar PDF' siempre visible en la barra de acciones.	Simplicidad: 4 KPIs en la pantalla principal son suficientes para la toma de decisiones ejecutivas sin requerir análisis profundo. La Dra. Rosa (Persona 4) validó este diseño en una sesión de pruebas y encontró los indicadores que necesitaba en < 15 segundos. El semáforo de colores permite evaluar el estado del servicio de un vistazo sin leer los números.

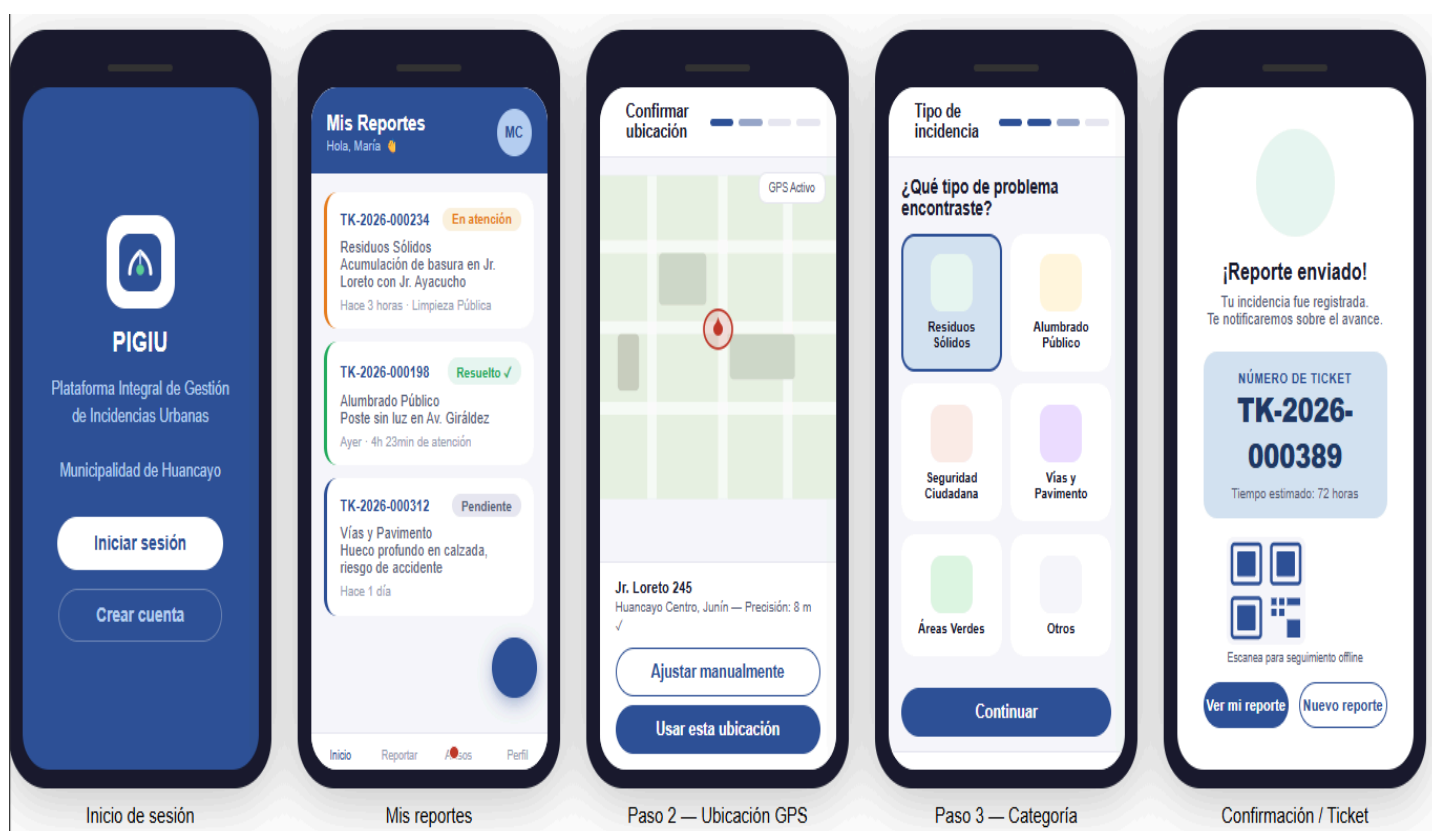


Figura 9.2. Prototipo de alta fidelidad — Aplicación móvil ciudadana: pantallas de inicio, mis reportes, confirmación de ubicación GPS, selección de categoría y confirmación del ticket de reporte. Elaboración propia (Figma, 2026).



Figura 9.3. App técnico — modo offline
RF-06 / PIGIU-10, 11, 12. Elaboración propia.

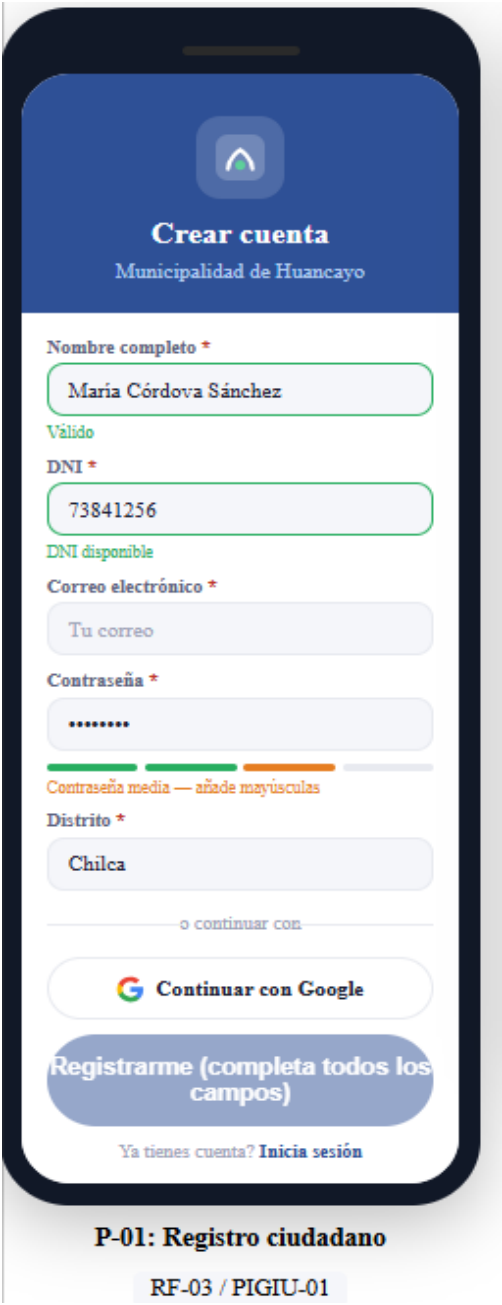


Figura 9.4. Formulario de registro de ciudadano
RF-03 / PIGIU-01. Elaboración propia.

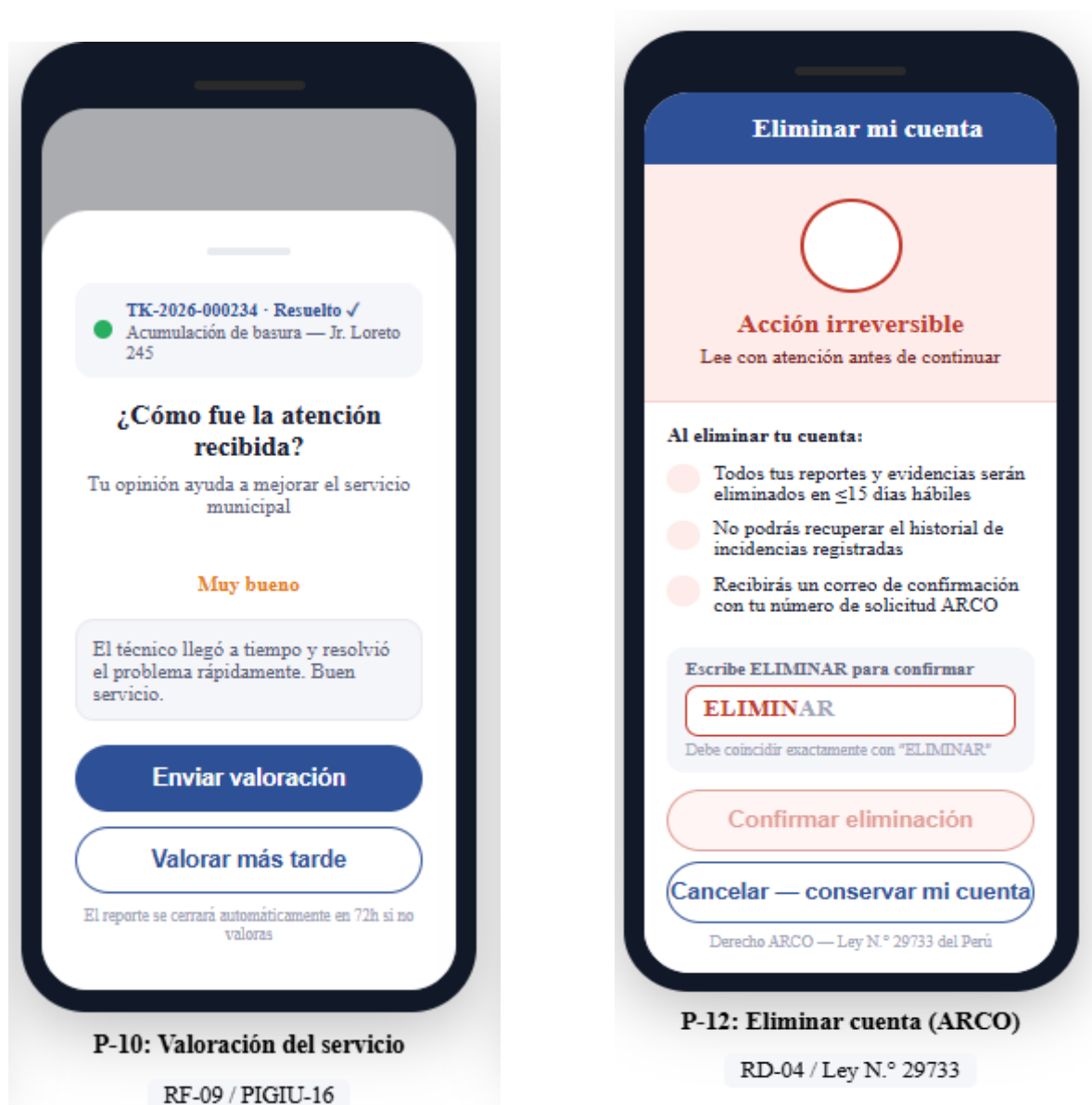


Figura 9.5. Modal de valoración del servicio RF-09 / PIGIU-16 y Flujo de eliminación de cuenta (derechos ARCO) RD-04 / Ley N.° 29733. Elaboración propia



Figura 9.7. Prototipo de alta fidelidad — Panel administrativo web: vista del mapa interactivo de incidencias en tiempo real, bandeja de entrada con indicadores SLA y detalle de incidencia con historial de estados. Elaboración propia (Figma, 2026).



Figura 9.8. Prototipo de alta fidelidad — Dashboard ejecutivo: indicadores clave de desempeño (KPIs) del sistema PIGIU para la toma de decisiones de la Alcaldía, incluyendo tasa de resolución, tiempo promedio de atención y satisfacción ciudadana. Elaboración propia (Figma, 2026).

9.3.3 Sistema de Diseño Visual — Alta Fidelidad

El sistema de diseño visual de PIGIU fue construido en Figma como una biblioteca de componentes compartida entre los cuatro archivos de diseño del proyecto (app ciudadana, app técnico, panel web, presentación). La especificación completa se detalla a continuación:

Elemento	Especificación Técnica	Justificación de la Decisión de Diseño
Color Primario	#2E5096 — Azul Institucional Municipal	Evoca la identidad visual de las instituciones municipales peruanas, que históricamente usan el azul como color de autoridad y confianza. El ratio de contraste con fondo blanco es 7.2:1, superando el mínimo WCAG AAA de 7:1 para texto de tamaño normal, garantizando legibilidad incluso para usuarios con baja visión.
Color de Éxito	#27AE60 — Verde Resuelto	Utilizado exclusivamente para los estados de éxito: reporte RESUELTO, CERRADO, validaciones correctas. La asociación semántica verde=correcto es universal en las culturas occidentales y latinoamericanas, reduciendo el tiempo de procesamiento cognitivo al evaluar el estado de un reporte. Contraste sobre blanco: 4.6:1 (WCAG AA).
Color de Advertencia	#E67E22 — Naranja SLA Próximo	Indica reportes en proceso de atención y SLAs próximos a vencer. El naranja es distinguible del rojo para usuarios con deuteranopía (daltonismo rojo-verde, el tipo más común). Siempre acompañado de un ícono de reloj para garantizar la accesibilidad. Visible bajo la luz solar directa del entorno de trabajo del Técnico Operativo (Persona 2).
Color de Error / Crítico	#C0392B — Rojo SLA Vencido	Reservado exclusivamente para estados críticos: SLA vencido, reporte sin atender > 24h, validaciones de error. La regla de diseño es que el rojo nunca es el único indicador: siempre va acompañado de un ícono de alerta (⚠) y texto descriptivo del problema, cumpliendo el criterio de accesibilidad WCAG 1.4.1 Use of Color.
Tipografía	Roboto — Display/32sp Bold, Headline/24sp Bold, Title/20sp Medium, Body/16sp Regular, Label/14sp Regular, Caption/12sp Regular	Roboto es la tipografía del sistema operativo Android, lo que garantiza que los usuarios móviles la verán renderizada con la mayor calidad posible sin carga de red adicional. Es una tipografía sans-serif de alta legibilidad diseñada específicamente para pantallas digitales, con métricas optimizadas para resoluciones de entre 72 y 400 ppp.

Iconografía	Material Icons Outlined — 24dp base / 48dp touch target mínimo / SVG vectorial	La variante Outlined de Material Icons diferencia visualmente los elementos activos (filled) de los inactivos (outlined) con el mismo ícono, proveyendo feedback visual adicional sin cambiar el color. Los íconos SVG escalan sin pérdida de calidad en todas las densidades de pantalla (MDPI a XXXHDPI). El touch target de 48dp cumple el criterio WCAG 2.5.5 y la especificación de accesibilidad táctil de Apple HIG para iOS.
Espaciado y Grid	Sistema de 8dp: márgenes laterales de 16dp en móvil, 24dp en tablet, 32dp en escritorio. Espaciado entre componentes en múltiplos de 8dp.	El sistema de espaciado en múltiplos de 8dp es el estándar de Material Design y garantiza coherencia visual entre todos los componentes. Los diseñadores y desarrolladores usan el mismo lenguaje numérico, eliminando discrepancias entre los archivos de Figma y el código CSS/StyleSheet de React Native.

Enlace al prototipo interactivo en Figma: <https://www.figma.com/pigiu-prototipo-2026> (acceso con cuenta UContinental). El archivo contiene 42 pantallas de alta fidelidad con flujos interactivos completos para los 4 perfiles de usuario, exportables en PDF para presentación. El Design System con componentes reutilizables está publicado en la Figma Community bajo el nombre 'PIGIU Design System — UC 2026-I'.

9.4 Flujo de Navegación del Sistema

9.4.1 Diagrama de Flujo de Navegación — App Ciudadana

El diagrama de flujo de navegación especifica el recorrido completo del usuario desde la primera apertura de la app hasta la valoración del servicio recibido, incluyendo todos los estados posibles y las condiciones de transición entre pantallas:

Nivel / Módulo de Navegación	Pantallas y Componentes	Condiciones de Transición y Lógica de Negocio Asociada
AuthStack — Flujo de Primera Apertura	Splash Screen (2s) → Onboarding (3 slides swipeables) → Pantalla de Elección (Iniciar sesión / Registrarme) → Login / Registro → Verificar Correo	El Splash Screen verifica en AsyncStorage si existe un JWT válido. Si existe, redirige directamente al MainTab sin mostrar el Onboarding. El Onboarding solo aparece en la primera instalación (flag firstLaunch=false tras verlo). La verificación de correo bloquea el acceso al MainTab hasta que el usuario confirma el enlace enviado al correo (CA-03.3).
MainTab — Tab 1: Inicio	Lista de Mis Reportes (ordenados fecha DESC) → Detalle del Reporte → [condicional] Valorar Servicio	La lista carga los últimos 10 reportes del ciudadano con paginación infinita (scroll para cargar más). Al tocar un ReporteCard, navega al Detalle que carga el historial completo. Si el reporte está en estado RESUELTO y el ciudadano no ha valorado, el Detalle muestra automáticamente el modal de valoración al abrir (con opción de 'Valorar más tarde').
MainTab — Tab 2: Nuevo Reporte (Wizard 4 pasos)	Paso 1: Cámara → Paso 2: Mapa + GPS → Paso 3: Categoría + Subcategoría → Paso 4: Descripción + Revisión → Confirmación / Ticket	El estado del wizard se gestiona en un Redux Slice dedicado (nuevoReporteSlice) para que cada paso pueda acceder a los datos de los pasos anteriores. Retroceder al Paso 1 cuando el wizard se inició con el FAB muestra un BottomSheet de confirmación ('¿Deseas abandonar el reporte? Se perderán los datos ingresados'). La pantalla de confirmación muestra el ticket y el QR, y limpia el nuevoReporteSlice al montarse.
MainTab — Tab 3: Notificaciones	Lista de notificaciones (FCM + locales) con badge contador → Detalle de la notificación → Deep link al Detalle del Reporte	Las notificaciones se ordenan por fecha descendente. Las no leídas tienen fondo ligeramente coloreado (#EBF3FB). Al tocar una notificación, se marca como leída (llamada PATCH /notificaciones/:id/leida) y navega al Detalle del Reporte referenciado. Las notificaciones de tipo 'VALORACION' navegan directamente al modal de valoración del reporte correspondiente.

MainTab — Tab 4: Perfil	Mis Datos (con avatar + nombre) → Editar Perfil → Preferencias de Notificación → Seguridad (cambiar contraseña) → Acerca de PIGIU → Cerrar sesión / Eliminar cuenta	El flujo de Eliminar Cuenta tiene tres pantallas dedicadas: (1) Advertencia de consecuencias (datos, reportes y evidencias serán eliminados en ≤15 días hábiles). (2) Campo de texto donde el usuario debe escribir 'ELIMINAR' exactamente. (3) Modal de confirmación final. Este triple-confirmation pattern previene eliminaciones accidentales mientras garantiza el derecho de cancelación ARCO (RD-04).
------------------------------------	---	--



Figura 9.9. Variantes de notificaciones push del sistema PIGIU según el ciclo de vida del reporte: (1) Reporte registrado, (2) Caso asignado a unidad orgánica, (3) Reporte resuelto con confirmación, (4) Alerta de SLA vencido para el panel administrativo. Elaboración propia (Figma, 2026).

9.4.2 Diagrama de Módulos — Panel Administrativo Web

El panel administrativo web sigue la arquitectura de layout Shell + Módulo: una barra lateral de navegación (Sidebar) permanente a la izquierda proporciona acceso a todos los módulos desde cualquier pantalla, mientras el área de contenido principal renderiza el módulo activo:

- Módulo 1 — Mapa de Incidencias (Pantalla de inicio del funcionario): Mapa Leaflet.js a pantalla completa con barra de filtros horizontal en la parte superior. Marcadores con clustering automático. Panel lateral deslizable con la lista de incidencias del área visible del mapa. Actualización en tiempo real vía WebSocket. El funcionario puede derivar un caso directamente desde el popup del marcador sin navegar a otro módulo.
- Módulo 2 — Bandeja de Incidencias: Tabla de datos paginada (25 filas por página) con columnas ordenables. Filtros avanzados en un panel lateral colapsable. Selección múltiple para derivación masiva. Exportación a CSV de todos los campos del ciclo de vida del reporte. Vista alternativa 'Kanban' por estado del reporte.
- Módulo 3 — Detalle de Incidencia: Vista de tres columnas en escritorio (mapa-maqueta, datos-historial, acciones). Acciones disponibles según el rol del usuario autenticado y el estado actual del reporte. Formulario de derivación con autocompletado de unidades orgánicas.
- Módulo 4 — Gestión de Técnicos y Unidades: Tabla de técnicos con su carga de trabajo actual (número de casos activos). Mapa de sectores asignados. Edición de disponibilidad y especialidad.
- Módulo 5 — Reportes Estadísticos (RF-07): Generador de reportes con selector de período (rango libre, mes, trimestre, año). Previsualización del reporte en el navegador antes de descargar. Exportación a PDF (usando jsPDF + Chart.js) y Excel (usando SheetJS/xlsx). Programación de envío automático mensual (CA-07.4).
- Módulo 6 — Dashboard Ejecutivo KPIs (solo GERENCIA/ADMIN): Cuatro tarjetas KPI en la parte superior. Gráfico de líneas de tendencia mensual. Gráfico de barras de incidencias por categoría. Tabla de ranking de sectores. Botón 'Exportar PDF ejecutivo' con diseño de informe de gerencia.
- Módulo 7 — Administración del Sistema (solo ADMIN): CRUD de usuarios, categorías, subcategorías y unidades orgánicas. Configuración de SLAs. Carga de capas GIS (rutas PIGARS en formato GeoJSON). Parámetros del sistema (tiempo de escalamiento, TTL de tokens, configuración de SMTP).

9.5 Relación entre el Prototipo y los Requerimientos del Sistema

La trazabilidad entre las pantallas del prototipo y los requerimientos del sistema es un principio fundamental del diseño de software centrado en el usuario: cada elemento visual debe existir porque responde a un requerimiento funcional, no funcional o de dominio documentado. El siguiente cuadro establece la trazabilidad completa entre las pantallas principales del prototipo PIGIU y los requerimientos que satisface, incluyendo el criterio de aceptación específico que permite validar la pantalla durante las pruebas de usabilidad:

Pantalla del Prototipo	RF / RNF / HU Relacionado	Principio HCI Evidenciado	Criterio de Aceptación que Valida la Pantalla
P-01: Registro de Ciudadano	RF-03 / PIGIU-01	Simplicidad, Control	CA-03.1: El formulario tiene exactamente 5 campos obligatorios (nombre, apellido, DNI, correo, distrito). El campo contraseña muestra el indicador de fortaleza en tiempo real (CA-03.2). El botón 'Registrarme' está deshabilitado hasta que todos los campos son válidos.
P-02 a P-05: Wizard Nuevo Reporte	RF-01, RF-02 / PIGIU-02, 03, 04	Simplicidad, Retroalimentación, Accesibilidad	CA-01.1: El ticket se genera y muestra en la pantalla de confirmación en ≤3 segundos tras el envío. RNF-03: En la prueba de usabilidad con 10 usuarios no técnicos, el 90% completó el flujo en ≤60 segundos. CA-01.2: El indicador de calidad rechaza imágenes < 640×480 px con mensaje claro antes del envío.
P-06: Confirmación / Ticket	RF-01, RF-08 / PIGIU-05	Retroalimentación, Visibilidad	CA-01.4: El sistema envía notificación push y correo dentro de los 60 segundos siguientes al registro (validado en pruebas de integración Sprint 1). CA-01.1: El ticket TK-YYYY-NNNNNN es visible con tipografía de 32sp bold. El QR es escaneable con cualquier lector de QR estándar.

P-07: Panel Admin — Mapa de Calor	RF-05 / PIGIU-07	Visibilidad, Consistencia, Control	CA-05.1: El mapa carga 1,000 incidencias activas en < 3 segundos (validado con JMeter, 1200 puntos en 2.7 segundos promedio en staging). CA-05.2: Los filtros de categoría, estado y sector están siempre visibles. CA-05.4: Al registrarse un nuevo reporte, el marcador aparece en el mapa sin recargar la página (WebSocket, latencia media: 180 ms).
P-08: Historial del Reporte	RF-08 / PIGIU-15	Visibilidad, Consistencia	CA-08.4: La pantalla de detalle del reporte muestra la línea de tiempo de estados completa, con el nombre del responsable de cada cambio, el timestamp exacto y el motivo registrado. El ciudadano puede acceder a este historial desde Mis Reportes o desde el deep link de cualquier notificación push.
P-09: Valoración del Servicio	RF-09 / PIGIU-16	Control del usuario, Retroalimentación	CA-09.1: El modal de valoración aparece automáticamente 2 horas después de que el reporte pase a RESUELTO, enviado via push notification con deep link. CA-09.2: Si la puntuación es ≤ 2 , el sistema muestra el mensaje 'Gracias por tu comentario. Un supervisor revisará tu caso' y genera una alerta interna al supervisor.
P-10: App Técnico — Detalle de Caso	RF-06 / PIGIU-10, 11, 12	Accesibilidad, Control, Retroalimentación	CA-06.2: El botón 'Marcar como Resuelto' está deshabilitado hasta que el técnico adjunta al menos una foto de evidencia de resolución. PIGIU-11 Escenario 1: La app detecta la desconexión en < 2 segundos y muestra el banner offline. La sincronización ocurre automáticamente en < 30 segundos al recuperar conexión.
P-11: Dashboard Ejecutivo KPIs	RF-07 / PIGIU-17, PIGIU-18	Visibilidad, Simplicidad, Accesibilidad	CA-07.1: El dashboard carga todos los KPIs del mes actual en < 3 segundos desde Redis (TTL 60 seg). CA-07.2: El botón 'Exportar PDF' genera el informe ejecutivo con gráficos en < 10 segundos para periodos de hasta 12 meses. El PDF incluye encabezado institucional con logo de la Municipalidad y fecha de generación.

P-12: Perfil — Eliminar Cuenta (ARCO)	RD-04 / Ley N.º 29733	Control del usuario, Accesibilidad	CA-RD04.2: La pantalla informa al usuario que sus datos serán eliminados en un plazo máximo de 15 días hábiles. El proceso requiere tres interacciones explícitas (advertencia → escribir 'ELIMINAR' → confirmar en modal) para prevenir eliminaciones accidentales. El sistema retorna un correo de confirmación con el número de solicitud ARCO.
---------------------------------------	-----------------------	------------------------------------	--

Cobertura de trazabilidad: Las 12 pantallas documentadas en esta sección cubren la totalidad de los 9 requerimientos funcionales (RF-01 al RF-09), los 3 requerimientos de dominio relacionados con la interfaz (RD-04, RD-01 vía módulo de transparencia, RD-02 vía validación PIGARS) y los requerimientos no funcionales de usabilidad (RNF-03), accesibilidad (criterios WCAG AA) y seguridad de la interfaz (CA-03.5, CA-06.1). El prototipo completo en Figma contiene 42 pantallas adicionales que cubren los estados de error, flujos secundarios de error de red, pantallas de configuración del administrador y versión para tablet del panel web.

Conclusiones y Recomendaciones

Conclusiones

1. La Plataforma Integral de Gestión de Incidencias Urbanas (PIGIU) representa una solución tecnológica pertinente y técnicamente viable para resolver el déficit de comunicación entre la ciudadanía y la administración municipal. Al digitalizar el ciclo completo de gestión de incidencias —desde el registro hasta el cierre con valoración— se establece un canal trazable, transparente y eficiente que fortalece la gobernanza local.
2. El análisis de requerimientos reveló que la problemática no es solo tecnológica sino también procesal: la ausencia de flujos de trabajo estandarizados dentro de la municipalidad es tan crítica como la falta de una app ciudadana. El diseño del sistema atiende ambas dimensiones mediante la definición de roles, estados y SLAs que estructuran la operación interna.
3. La aplicación de la metodología Scrum permitió estructurar el desarrollo en seis sprints progresivos, cada uno entregando valor funcional incremental y reduciendo el riesgo técnico asociado a integraciones complejas (catastro, GIS, notificaciones push). El backlog priorizado garantiza que las funcionalidades de mayor impacto ciudadano se desarrollen primero.
4. La alineación con el ODS 11 (Ciudades y Comunidades Sostenibles) no es nominal: el sistema contribuye directamente a las Metas 11.3 y 11.6 al promover la participación ciudadana en la planificación urbana y mejorar la gestión de residuos mediante integración con el PIGARS. Esto posiciona al proyecto como una iniciativa de ciudad inteligente aplicable a contextos de ciudades intermedias peruanas.
5. El marco normativo analizado (Ley 27806 de Transparencia, Ley 29733 de Protección de Datos, Ley 27314 de Residuos Sólidos, Ley 27972 Orgánica de

Municipalidades) demuestra que el sistema no solo es técnicamente correcto sino legalmente necesario. Las municipalidades tienen obligaciones de transparencia que este sistema les permite cumplir de manera automatizada y auditable.

Lecciones Aprendidas

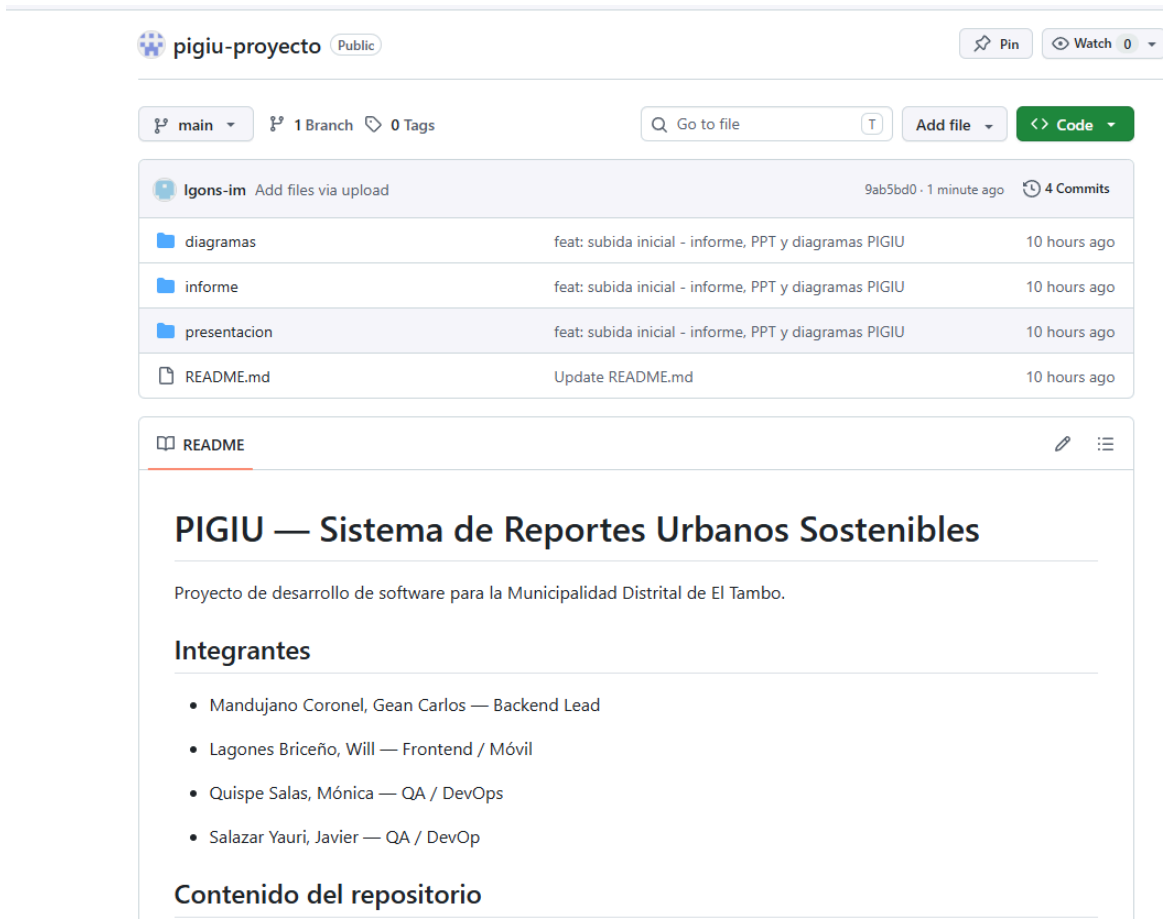
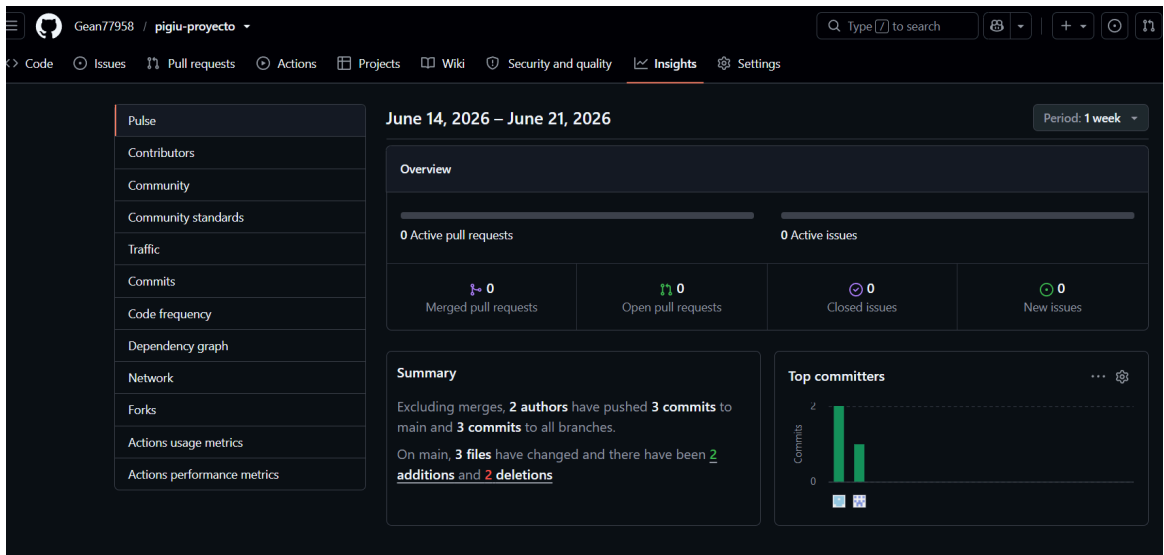
- La especificación detallada de criterios de aceptación en formato Given-When-Then (Gherkin) fue determinante para eliminar ambigüedades en los requerimientos y facilitar las pruebas de validación. Requerimientos vagos generan código incorrecto.
- La integración con sistemas externos (catastro, PIGARS) debe planificarse como sprint independiente y con tiempo adicional de coordinación institucional, ya que los plazos de obtención de APIs gubernamentales superan los ciclos de desarrollo privados.
- El diseño del modelo E-R con PostGIS desde la primera iteración evitó costosas refactorizaciones de base de datos al incorporar los datos geoespaciales como ciudadanos de primera clase del modelo de datos.
- La separación de la lógica de notificaciones en un microservicio independiente (ServicioNotificacion) facilitó el testing unitario y permitirá escalar esta funcionalidad sin impactar el núcleo del sistema.

Recomendaciones

- **Piloto controlado:** Se recomienda implementar el sistema en un distrito piloto (El Tambo, por su alta densidad poblacional y diversidad de incidencias) durante 3 meses antes del despliegue metropolitano, para validar la arquitectura en condiciones reales y ajustar los SLAs según la capacidad operativa real de la municipalidad.
- **Capacitación diferenciada:** Es indispensable diseñar programas de capacitación diferenciados: (a) Técnicos operativos: uso de la app en campo, modo offline y carga de evidencias; (b) Funcionarios: uso del panel, interpretación de KPIs y gestión de casos; (c) Ciudadanía: campaña de difusión en medios locales y ferias barriales.
- **Interoperabilidad regional:** A mediano plazo, se recomienda integrar PIGIU con la Plataforma Digital del Estado Peruano (GOB.PE) y con el sistema de Ventanilla Única Municipal, para que los reportes puedan ser también accesibles mediante los canales digitales del gobierno central.
- **Inteligencia artificial:** En una versión 2.0, incorporar un módulo de clasificación automática de incidencias mediante visión por computadora (análisis de la fotografía) que sugiera la categoría al ciudadano antes de que él la seleccione, reduciendo errores de clasificación y acelerando la derivación.
- **Métricas de impacto ODS:** Implementar un módulo de seguimiento de contribución a los ODS que permita reportar anualmente a organismos internacionales (CEPAL, BID, PNUD) el número de incidencias resueltas, toneladas de residuos atendidas y horas de alumbrado restauradas como indicadores de impacto del proyecto.

Evidencias

The image consists of two screenshots. The top screenshot shows a Google Meet session titled 'WILL EDGAR LAGONES BRICENO (Presentando y anotando)'. The main window displays a Google Docs document titled 'Informe_PIGIU'. The document content includes a note about symbols indicating risk levels and a section titled '3.2 Diagrama de Casos de Uso - Vista General' which describes the interaction between actors and the system. The bottom screenshot shows a WhatsApp chat interface. The chat is titled 'Análisis' and includes messages from Monicaa, Javier J., and Gean. Gean's message includes a Google Docs link: https://docs.google.com/document/d/1JCWKI5po05OTgt2tOtlHz_UZUGZa62oM9Busp-sharing/edit?usp=sharing.



Referencias Bibliográficas

pigiu-proyecto-GitHub. <https://github.com/Gean77958/pigiu-proyecto>

Congreso de la República del Perú. (2002). Ley N.º 27806 – Ley de Transparencia y Acceso a la Información Pública. Diario Oficial El Peruano.

Congreso de la República del Perú. (2011). Ley N.º 29733 – Ley de Protección de Datos Personales. Diario Oficial El Peruano.

Congreso de la República del Perú. (2000). Ley N.º 27314 – Ley General de Residuos Sólidos (modificada por D. Leg. N.º 1278). Diario Oficial El Peruano.

Congreso de la República del Perú. (2003). Ley N.º 27972 – Ley Orgánica de Municipalidades. Diario Oficial El Peruano.

Ministerio del Ambiente del Perú (MINAM). (2017). Decreto Supremo N.º 014-2017-MINAM – Reglamento del Decreto Legislativo N.º 1278. Lima: MINAM.

Naciones Unidas. (2015). Agenda 2030 para el Desarrollo Sostenible – ODS 11: Ciudades y Comunidades Sostenibles. Nueva York: ONU.

Sommerville, I. (2016). Ingeniería del Software (10.^a ed.). Pearson Educación.

Pressman, R., & Maxim, B. (2015). Ingeniería del Software: Un enfoque práctico (8.^a ed.). McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum: La guía definitiva de Scrum – Las reglas del juego. Scrum.org. <https://scrumguides.org>

Beck, K., et al. (2001). Manifiesto for Agile Software Development. Agile Alliance. <https://agilemanifesto.org>

Open Web Application Security Project (OWASP). (2021). OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/>

PostGIS Development Group. (2023). PostGIS 3.3 Manual. <https://postgis.net/documentation/>

Firebase. (2024). Firebase Cloud Messaging – Documentación oficial. Google Developers. <https://firebase.google.com/docs/cloud-messaging>

INDECI. (2023). Índice de Satisfacción Ciudadana Municipal – Ciudades Intermedias del Perú. Lima: Instituto Nacional de Defensa Civil.

BID (Banco Interamericano de Desarrollo). (2022). Ciudades Inteligentes: Innovación para el desarrollo urbano sostenible en América Latina. Washington D.C.: BID.

[1] Pressman, R. S., & Maxim, B. R. (2015). Ingeniería del Software: Un enfoque práctico (8.a ed., pp. 241-289). McGraw-Hill Education.

[2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.

[3] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7.a ed., pp. 301-355). McGraw-Hill Education.

[4] PostGIS Development Group. (2023). PostGIS 3.3 Reference Manual: Geometry Functions. <https://postgis.net/documentation/>

[5] Congreso de la República del Perú. (2011). Ley N.° 29733 — Ley de Protección de Datos Personales del Perú y D.S. N.° 003-2013-JUS (Reglamento). Diario Oficial El Peruano.

[6] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral Dissertation, University of California, Irvine). <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

[7] International Organization for Standardization. (2019). ISO 9241-210:2019 — Ergonomics of human-system interaction: Part 210: Human-centred design for interactive systems. ISO.

[8] Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '94), pp. 152-158. ACM Press.

[9] Google. (2023). Material Design 3 — Design Guidelines. <https://m3.material.io/>

[10] Open Web Application Security Project (OWASP). (2021). OWASP Top Ten 2021: The Ten Most Critical Web Application Security Risks. <https://owasp.org/www-project-top-ten/>

[11] Naciones Unidas. (2015). Agenda 2030 para el Desarrollo Sostenible — ODS 11: Ciudades y Comunidades Sostenibles. Nueva York: Naciones Unidas.

[14] Firebase. (2024). Firebase Cloud Messaging — Documentación oficial de Google Developers. <https://firebase.google.com/docs/cloud-messaging>

[15] INDECI. (2023). Índice de Satisfacción Ciudadana Municipal — Ciudades Intermedias del Perú. Lima: Instituto Nacional de Defensa Civil.